

University of Stuttgart
Institute of Industrial Automation and
Software Engineering

Develop a Hybrid Communication Protocol Framework for Multi-Robot Systems Using KUKA Robots and PLCs

Master Thesis 3923

Submitted at the University of Stuttgart by
Krutarth Patel

M.Sc. Electrical Engineering

Examiner:	Jun.-Prof. Dr.-Ing. Andrey Morozov
Supervisor:	Yuliang Ma, M.Sc.
External Supervisor:	Dr.-Ing Muhammad Saeed (ARENA2036 e.V.)

19.08.2025



Table of Contents

TABLE OF CONTENTS	II
TABLE OF FIGURES	IV
TABLE OF TABLES	VI
TABLE OF ABBREVIATIONS	VII
GLOSSARY	VIII
ABSTRACT	X
ZUSAMMENFASSUNG	XI
ACKNOWLEDGEMENTS	XII
1 INTRODUCTION	13
1.1 Background and Motivation	13
1.2 Problem Statement	15
1.3 Research Scope, Questions, Objectives, and Contributions	15
1.4 Thesis Structure	17
2 LITERATURE REVIEW	19
2.1 Industrial Communication Protocols	19
2.1.1 Traditional Protocols	19
2.1.2 Modern Industrial Communication Protocols	23
2.2 Multi-Robot Communication	27
2.2.1 Communication Architectures for Multi-Robot Systems	27
2.2.2 Real-time Coordination Requirements	29
2.3 Industrial Robotics Integration Challenges	30
2.3.1 ROS2 Industrial Applications and Limitations	30
2.3.2 PLC-Robot Integration Approaches	31
2.3.3 Hybrid Communication Framework Requirements	32
3 SYSTEM DESIGN AND METHODOLOGY	34
3.1 System Requirements and Design Objectives	34
3.2 Hybrid Communication Architecture Design	35
3.2.1 Overall Framework Architecture	35
3.2.2 Single Robot Communication Architecture	38
3.2.3 Multi-Robot System Architecture	39
3.3 Implementation and Evaluation Strategy	41
3.3.1 Technology Selection Rationale	41
3.3.2 Development and Testing Methodology	42
4 IMPLEMENTATION	44
4.1 Development Environment and Tools	44
4.1.1 Software Stack Overview	45

4.1.2	Workspace Structure for Single and Multi-Robot Systems	46
4.2	Single Robot System Implementation	48
4.2.1	OPC UA Server Implementation	49
4.2.2	Modbus-to-OPC UA Bridge	50
4.2.3	ROS2 Joint Publisher Node.....	52
4.2.4	Performance Logging Module	54
4.2.5	AI-Based Anomaly Detection Module.....	55
4.2.6	Discussion	56
4.3	Multi-Robot System Implementation	57
4.3.1	Multi-Robot OPC UA Server.....	57
4.3.2	Dual Modbus Bridge for Robot Control	59
4.3.3	Multi-Robot Controller Node.....	60
4.3.4	Multiple Robots Performance Monitoring System	64
4.3.5	AI Integration for Multi-Robot System	64
4.4	Integration and Execution.....	65
4.4.1	Single Robot Launch Pipeline	65
4.4.2	Multi-Robot Launch Pipeline.....	67
5	RESULTS AND EVALUATION.....	70
5.1	Evaluation of the Single-Robot System	70
5.2	Evaluation of the Multi-Robot System	76
5.3	Comparison Between Single and Multi-Robot Systems.....	81
5.4	Comparative Analysis with Existing Solutions	82
6	SUMMARY AND FUTURE SCOPE	84
	BIBLIOGRAPHY	86
	DECLARATION OF COMPLIANCE.....	92

Table of Figures

Figure 1.1: 6-Dof Industrial Robot KUKA KR360 [10]	14
Figure 1.2: Thesis Structure and Chapter Summary	18
Figure 2.1: Modbus application layer network with HMI, PLC devices [23].....	20
Figure 3.1: Hybrid Communication Framework Architecture	37
Figure 3.2: Single Robot Communication Architecture	38
Figure 3.3: Multi-Robot System Architecture	40
Figure 4.1: Single Robot Ros 2 Workspace Structure	47
Figure 4.2: Multi-Robot Ros 2 Workspace Structure	48
Figure 4.3: Terminal Output of Manual_Opcua_Server_Kr360.Py	49
Figure 4.4: Code Flow Chart of Script Manual_Opcua_Server_Kr360.Py	50
Figure 4.5: OpenPLC Web Interface for Robot Motion Commands	51
Figure 4.6: Control Flow of Modbus-To-OPC UA Bridge Logic	52
Figure 4.7: RViz Window With KR360 R2830 Robot Model With Static Pose	53
Figure 4.8: Rqt_Graph Visualizing the Live ROS2 Node Communication	53
Figure 4.9: Web Interface View of The KR360 Robot Control Simulator.....	54
Figure 4.10: Architecture Of AI-Based Anomaly Detection for Single KR360 Robot. .	56
Figure 4.11: Terminal Output For Initialization of Dual Robot OPC UA Objects	58
Figure 4.12: Terminal Output from Dual Modbus-OPC UA Bridge For Both Robots. .	60
Figure 4.13: Rviz Window with Two KR360 R2830 Robot Model With Static Pose...	62
Figure 4.14: Rqt_graph Showing ROS 2 Communication Nodes for Both Robots	62
Figure 4.15: Dual-Robot Web-Based Simulator Interface	63
Figure 4.16: Terminal Log of Simulator Showing Command Execution of GUI	63
Figure 4.17: Startup Sequence Diagram for Single Robot System Pipeline	67
Figure 4.18: Startup Sequence Diagram for Dual Robot System Pipeline	69
Figure 5.1: Latency Distribution Across Command Types in Simulation.....	70
Figure 5.2: Histogram of Total System Vs. PLC–OPC UA Latency.....	71
Figure 5.3: Latency Timeline Showing Stability Across Commands.....	71
Figure 5.4: Throughput Rate Over Time for Control Commands.....	72

Figure 5.5: Jitter Analysis and Histogram of Timing Variations	72
Figure 5.6: Cumulative Latency And Success Rate Across Session	73
Figure 5.7: Summary of System Latency and Performance Metrics	74
Figure 5.8: Confusion Matrix for Anomaly Detection Classification	74
Figure 5.9: Feature Importance from Random Forest Model.....	75
Figure 5.10: Classification Report Showing Accuracy and Precision	75
Figure 5.11: Multi-Robot Latency Distribution By Command Type.....	76
Figure 5.12: Multi-Robot Latency Distributions Analysis.....	77
Figure 5.13: Multi-Robot Latency Timeline Stability Across Commands.....	78
Figure 5.14: Multiple Robot Cumulative Performance Metrics.....	79
Figure 5.15: Summary of Multi-Robot System Latency and Performance Metrics	79
Figure 5.16: Jitter Analysis and Histogram for Multi-Robots.....	80
Figure 5.17: Multi-Robot Throughput Rate Over Time for Control Commands.....	80

Table of Tables

Table 2.1: Traditional Industrial Communication Protocols Comparison	22
Table 2.2: Modern Industrial Communication Protocols Comparison	26
Table 4.1: Software Stack Overview	45
Table 5.1: Summary of Single vs Multi-Robot Performance	82
Table 5.2: Comparison of Proposed Framework with Existing Solutions	83

Table of Abbreviations

AI	A rtificial I ntelligence
CPU	C entral P rocessing U nit
DDS	D ata D istribution S ervice
DoF	D egrees o f F reedom
GUI	G raphical U ser I nterface
HIL	H ardware-in-the- L oop
HMI	H uman M achine I nterface
HTTP	H ypertext T ransfer P rotocol
MQTT	M essage Q ueuing T elemetry T ransport
OPC UA	O pen P latform C ommunications U nified A rchitecture
PLC	P rogrammable L ogic C ontroller
PROFIBUS	P rocess F ield B us
PROFINET	P rocess F ield N etwork
Pub/Sub	P ublish/ S ubscribe
QoS	Q uality o f S ervice
ROS	R obot O perating S ystem
RTOS	R ea - Time O perating S ystem
RViz	R OS V isualization
TCP/IP	T ransmission C ontrol P rotocol / I nternet P rotocol
TSN	T ime- S ensitive N etworking
UDP	U ser D atagram P rotocol

Glossary

Data Distribution Service	Real-time communication middleware standard used in ROS 2 that enables scalable, high-performance data exchange between distributed systems with Quality of Service guarantees [13, 14].
Degrees of Freedom	The number of independent movements a robot can perform. A 6-DoF industrial robot can move in three translational directions (X, Y, Z) and rotate about three axes [8, 9].
End-to-End Latency	The total time required for a command to travel from user input through all system layers until robot execution begins. Measured from GUI command to robot trajectory execution [37, 38].
Hybrid Communication Framework	Multi-protocol architecture that integrates traditional industrial protocols (Modbus, PLC) with modern robotic middleware (ROS 2) through OPC UA middleware, enabling seamless interoperability [55, 56].
Jitter	Variation in timing between successive operations, measured as standard deviation of latency. Critical parameter for real-time system stability [37, 38].
Modbus TCP	Ethernet-based communication protocol that enables remote access to PLC data over TCP/IP networks, widely used for industrial device communication [20, 21, 23].
Multi-Robot Coordination	Synchronized control of multiple robotic systems working collaboratively, requiring real-time communication and namespace isolation to prevent conflicts [31, 33].
Namespace Isolation	ROS 2 technique that separates communication channels for different robots using unique prefixes (e.g., /robot1/, /robot2/), preventing topic conflicts [40, 42].
OPC UA	Platform-independent industrial communication protocol providing secure, reliable data exchange with semantic information modeling capabilities. Serves as middleware between PLC and ROS 2 systems [26, 27].

OpenPLC	Open-source programmable logic controller runtime supporting IEC 61131-3 standards and Modbus TCP communication, used for PLC simulation in this research [42, 51].
PLC	Industrial computer designed for real-time control of manufacturing processes, programmed using ladder logic and providing deterministic operation [2, 45].
ROS 2	Open-source robotics middleware framework providing communication infrastructure and hardware abstraction for robotic applications with improved real-time capabilities [16, 40, 41].
ros2_control	Hardware abstraction and control framework in ROS 2 providing standardized interfaces for robot controllers and trajectory execution [42, 53].
Throughput	Number of commands processed per unit time, measured in commands per second (cmd/s), indicating system processing capacity [21, 24].

Abstract

Industrial multi-robot systems often fail to integrate older PLC-based control with modern ROS 2 coordination. Existing OPC UA/ROS 2 bridges are slow (2–10 s delays), rigid, and not ready for precise, real-time multi-robot work. This gap limits scalability and prevents industrial deployment in scenarios where millisecond-level timing is critical.

This thesis presents a modular hybrid communication framework that combines OpenPLC with Modbus TCP for industrial logic, OPC UA middleware for standardized data exchange, and ROS 2 for robotic control. The design supports both single and dual KUKA KR360 robots in simulation, using namespace isolation to prevent conflicts, protocol-translation bridges for interoperability, and a web-based interface for executing standardized robot actions. An AI-based anomaly detection module monitors joint-level signals for abnormal behaviour.

The framework achieved latencies of ~113 ms in single-robot trials and ~104 ms in dual-robot trials, both with 100 % command success. Throughput remained ~1.6 commands/s in single-robot and ~1.5 commands/s in dual-robot setups. Synchronization jitter was ~31 ms for single-robot and ~68 ms for dual-robot configurations. Compared to common UA-Bridge solutions, the proposed framework reduced coordination delays from seconds to milliseconds while maintaining perfect reliability.

Unlike existing OPC UA/ROS 2 integration attempts, this framework offers sub-200 ms deterministic communication, scalable namespace-based multi-robot coordination, and built-in AI monitoring — all within a simulation-ready package aligned with industrial standards. These capabilities form a foundation for real-world Industry 4.0 automation, enabling faster commissioning, smoother interoperability between legacy PLCs and modern robotics, and a pathway towards influencing emerging industrial communication standards.

Key Words: ROS 2; OPC UA; OpenPLC; Modbus TCP; PLC–robot integration; Industrial robotics; Multi-robot coordination; KUKA KR360; Latency and jitter; Synchronization; Anomaly detection; Hybrid middleware; Web-based GUI; Simulation.

Zusammenfassung

Industrielle Multi-Roboter-Systeme scheitern häufig daran, ältere SPS-basierte Steuerungen mit moderner ROS 2-Koordination zu integrieren. Bestehende OPC UA/ROS 2-Brücken sind langsam (2–10 s Verzögerung), unflexibel und nicht für präzise Multi-Roboter-Arbeiten in Echtzeit geeignet. Diese Lücke schränkt die Skalierbarkeit ein und verhindert den industriellen Einsatz in Szenarien, in denen eine zeitliche Genauigkeit im Millisekundenbereich entscheidend ist.

Diese Arbeit stellt ein modulares hybrides Kommunikationsframework vor, das OpenPLC mit Modbus TCP für die industrielle Logik, OPC UA-Middleware für den standardisierten Datenaustausch und ROS 2 für die Robotersteuerung kombiniert. Das Design unterstützt sowohl einzelne als auch zwei KUKA KR360-Roboter in der Simulation und nutzt dabei Namespace-Isolation zur Vermeidung von Konflikten, Protokollübersetzungsbrücken für Interoperabilität und eine webbasierte Schnittstelle zur Ausführung standardisierter Roboteraktionen. Ein KI-basiertes Modul zur Erkennung von Anomalien überwacht Signale auf Gelenkebene auf abnormales Verhalten.

Das Framework erreichte Latenzen von ~113 ms in Versuchen mit einem einzelnen Roboter und ~104 ms in Versuchen mit zwei Robotern, beide mit 100 %iger Befehlsausführungsrate. Der Durchsatz blieb bei ~1,6 Befehlen/s in Konfigurationen mit einem einzelnen Roboter und ~1,5 Befehlen/s in Konfigurationen mit zwei Robotern. Der Synchronisationsjitter betrug ~31 ms für Einzelroboter- und ~68 ms für Doppelroboter-Konfigurationen. Im Vergleich zu gängigen UA-Bridge-Lösungen reduzierte das vorgeschlagene Framework die Koordinationsverzögerungen von Sekunden auf Millisekunden bei gleichbleibender perfekter Zuverlässigkeit.

Im Gegensatz zu bestehenden OPC UA/ROS 2-Integrationsversuchen bietet dieses Framework eine deterministische Kommunikation mit weniger als 200 ms, eine skalierbare, namensraumgestützte Multi-Roboter-Koordination und eine integrierte KI-Überwachung – alles in einem simulationsfähigen Paket, das den Industriestandards entspricht. Diese Funktionen bilden die Grundlage für die reale Automatisierung im Rahmen von Industrie 4.0 und ermöglichen eine schnellere Inbetriebnahme, eine reibungslosere Interoperabilität zwischen älteren SPSen und moderner Robotik sowie einen Weg zur Einflussnahme auf neue industrielle Kommunikationsstandards.

Key Words: ROS 2; OPC UA; OpenPLC; Modbus TCP; PLC-Roboter-Integration; Industrierobotik; Multi-Roboter-Koordination; KUKA KR360; Latenz und Jitter; Synchronisation; Anomalieerkennung; Hybrides Middleware; Webbasierte GUI; Simulation.

Acknowledgements

I would like to express my sincere gratitude to the University of Stuttgart for providing the academic environment and resources that made this thesis possible. I am especially grateful to ARENA2036 (Active Research Environment for the Next Generation of Automobiles), a research campus dedicated to the future of production and mobility, for its supportive ecosystem and commitment to Industry 4.0 innovation.

My deepest thanks go to Dr.-Ing. Muhammad Saeed (ARENA2036 e. V.) for his supervision, valuable feedback, and guidance throughout this work. I also wish to thank Mr. Peter Fröschle for the opportunity to work within the ARENA2036 ecosystem, which greatly enriched my academic and professional experience.

The German summary (Zusammenfassung) was drafted using DeepL Translator and then thoroughly reviewed and revised by the author to ensure technical accuracy and adherence to academic standards.

1 Introduction

The following chapter introduces the research topic, explains the motivation behind implementing a hybrid communication framework, explains the problem statement, and specifies the thesis objectives and structure.

1.1 Background and Motivation

Automation, real-time data exchange, and interconnected systems are becoming more common in modern industries faster due to Industry 4.0 [1]. Industrial robots are becoming more important in this change because they will be used to do repetitive tasks and work in dynamic, collaborative environments where many robots are frequently present [1]. Programmable Logic Controllers (PLCs), Human–Machine Interfaces (HMIs), and supervisory systems need to be able to send control signals and status updates to robots quickly and reliably [2, 3].

Historically, industrial automation has relied on predetermined protocols such as Modbus, Profibus, and Profinet. These protocols were ideally suited for one-to-one or one-to-many static control architectures. However, these protocols are not flexible enough; they cannot handle multiple robots operating in synchronization, and they cannot keep up with the growing need to expand robotics deployments. Industrial robotic arms are one of the most well-known and widely utilized varieties of robotic technology used in smart manufacturing. Robotic arms are now used frequently in many industries, like aerospace, automotive, and many more [4, 5]. These arms, like the KUKA KR series, are highly accurate, flexible, and durable, which means they can be used for operations like welding, inspecting, and assembly [5, 6]. Recent studies by S. Caro and A. Burghardt show that KUKA robots are very useful because they may operate in complex ways and are reliable in many different types of industries [6, 7].

A typical industrial robotic arm, like the KUKA KR360 used in the current research, has six degrees of freedom (DoF), which allow the end-effector to be entirely positioned and oriented in three dimensions [8]. One can move along the X, Y, and Z axes, and the other three can move in a circle around these axes, which are called rotational movements [9]. The idea of the 6-DoF structure is shown in Figure 1.1. Because these joints are flexible, the robot can do complicated tasks multiple times and in a wide range of industries.

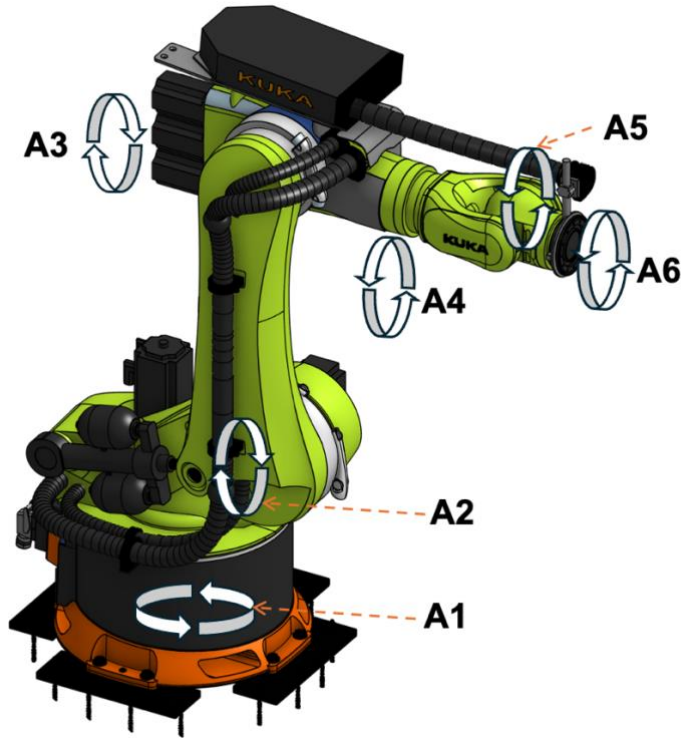


Figure 1.1: 6-Dof Industrial Robot KUKA KR360 [10]

In robotics, DoF indicates the number of independent movements a robot can perform [11]. Translational degrees of freedom pertain to linear movement along the coordinate axes, whereas rotational degrees of freedom pertain to rotation about those axes. The flexibility and motion complexity of six-axis industrial robots, such as the KUKA KR 360, enhance the necessity for accurate, real-time, and synchronized communication infrastructures. Monitoring a single robot is challenging, and coordinating multiple robots, particularly in simulation environments, requires an effective communication framework that effectively integrates both traditional and modern protocols [11].

Simultaneously, modern robotic systems—particularly those developed with the Robot Operating System (ROS2)—have implemented novel communication concepts, including Data Distribution Service (DDS) and publish–subscribe architectures. These systems enable decentralized control, precise synchronization, and enhanced modularity. The Open Platform Communications Unified Architecture (OPC UA) has become an essential middleware solution that offers platform-independent connectivity, semantic data modeling, and strong safety features for connecting traditional PLC systems with modern robotic frameworks. However, they are compatible with traditional PLC-based industrial infrastructures, resulting in an important technological gap.

The motivation for this thesis was inspired by the fact that this gap has become larger. Industrial robots become more intelligent and expand, but the communication infrastructure that they utilize is still divided, inflexible, and difficult to expand. This is particularly problematic when attempting to coordinate multiple robots in a shared task or workspace, utilizing simulation tools for testing.

1.2 Problem Statement

Integration problems are caused by the fact that modern ROS2 robotic platforms are not compatible with older PLC-based control systems [12]. Modbus and other older protocols are designed for controlling a single device and lack the flexibility required for complex robotic operations [12, 13]. ROS2 frameworks can work with scalable publish-subscribe architectures, but they do not work with older PLC systems [14]. The problem becomes more critical in multi-robot scenarios, where coordination, synchronization, and timing must be managed precisely [15]. Without a unified communication layer, the following challenges emerge:

- Challenges in synchronizing commands among PLCs and various ROS2-controlled robots [17].
- The lack of standardized interfaces bridging traditional and modern systems is a significant issue.
- Failure to simulate hybrid systems due to protocol conflict [18, 19].
- Increased latency due to misaligned communication frameworks.

Despite various advancements in both PLC systems and robotic middleware, there is still not a satisfactory, simulation-ready communication framework that connects traditional PLC control with ROS2-based robotic models that perform smoothly, especially in environments with more than one robot [15, 16]. This thesis solves this gap by developing and evaluating a hybrid PLC–OPC UA–ROS2 communication framework, designed to support scalable, real-time coordination between simulated industrial robots through these developed protocols in chapters 3 and 4.

1.3 Research Scope, Questions, Objectives, and Contributions

The main goal of the research is to create and test a communication framework that makes it simple for traditional industrial controllers and new robotic systems to work together. Particularly, it examines at how a hybrid architecture built up of PLC logic, OPC UA middleware, and ROS2-based robotic nodes can help control one or more industrial robots at the same time in a simulation scenario.

• Research Questions

The present investigation is motivated by the following research questions to address the essential communication and integration challenges:

1. How can OPC UA be used as an effective middleware to connect standard PLC communication protocols with ROS2-based robots?

2. Can multi-robot coordination be achieved in a simulated environment using a hybrid PLC–OPC UA–ROS2 communication architecture?
3. What are the performance characteristics such as latency, synchronization delay, and coordination accuracy of the proposed system in single-robot and dual-robot configurations?

- **Objectives**

To answer these questions, the research sets out to achieve the following objectives:

- Design a hybrid architecture that integrates simulated PLC control, OPC UA server nodes, and ROS2.
- Build a testable simulation environment for both single and dual KUKA KR360 robots that allows namespace-separated ROS2 nodes to work together in real time.
- Evaluate communication performance in terms of latency, message reliability, and synchronization.
- Validate the system under simulation-only constraints to verify that it can model industrial-scale control logic without requiring physical hardware.

- **Key Contributions**

This thesis makes the following original contributions:

- A dual-layer hybrid communication framework connecting PLC commands with advanced ROS2 robotic control using OPC UA as middleware.
- A fully functional simulation testbed supporting both single and multi-robot configurations through modular and scalable bridges.

- **Scope and Limitations**

While the proposed framework is intended to replicate real-world industrial conditions, this research is constrained to simulation-only testing using ROS2, RViz, and OpenPLC. All hardware components—including the PLC and robotic arms—are virtually modeled.

In this thesis implementation is limited to:

- A robot type (KUKA KR360) and Joint-level control (no trajectory planning).
- No more than two robots at a time because of system RAM, simulation tools, and memory limits (multi-robot scalability explored up to dual setup).

Despite these limitations, the work lays the groundwork for future real-world deployment and expansion into larger robotic fleets.

1.4 Thesis Structure

This thesis is organized into six chapters, each addressing a key stage in the research and development of a hybrid communication framework for simulation-based coordination of industrial robots. The structure of the thesis is illustrated in Figure 1.2 and summarized below.

Chapter 1 is the Introduction. It presents a motivation, background, and problem statement. It indicates the research questions, the scope and limitations, and the goals and contributions of this work. Chapter 2 is the literature review. It provides a structured literature survey for prior work on industrial communication protocols, multi-robot coordination methods, and integration strategies between PLC systems and modern robotic middleware. This chapter shows the problems with traditional methods and points out the gap that this research solves.

Chapter 3 is titled “System Design and Architecture”. This chapter presents the conceptual design of the hybrid PLC–OPC UA–ROS2 framework. It describes the communication layers, signal flow, namespace handling, and the architectural design for both single and dual robot systems. Performance evaluation strategies and metrics are also outlined in this chapter. Chapter 4 is about implementation of details the development and simulation environment. This includes the creation of bridge modules for Modbus-to-OPC UA and OPC UA-to-ROS2, the configuration of ROS2 nodes for single and dual KUKA KR360 robots, and the performance monitoring tools used during testing.

Chapter 5 is named Results and Evaluation. It presents both quantitative and visual insights into the way the system works with one robot or many robots. It displays and records latency, synchronization delay, and message flow in the form of graphs and logs, and it compares outcomes for single and dual robot setups. The last chapter summarizes the main results and evaluates whether the proposed system met the research goals. This chapter also outlines limitations and proposes future directions, such as real hardware integration.

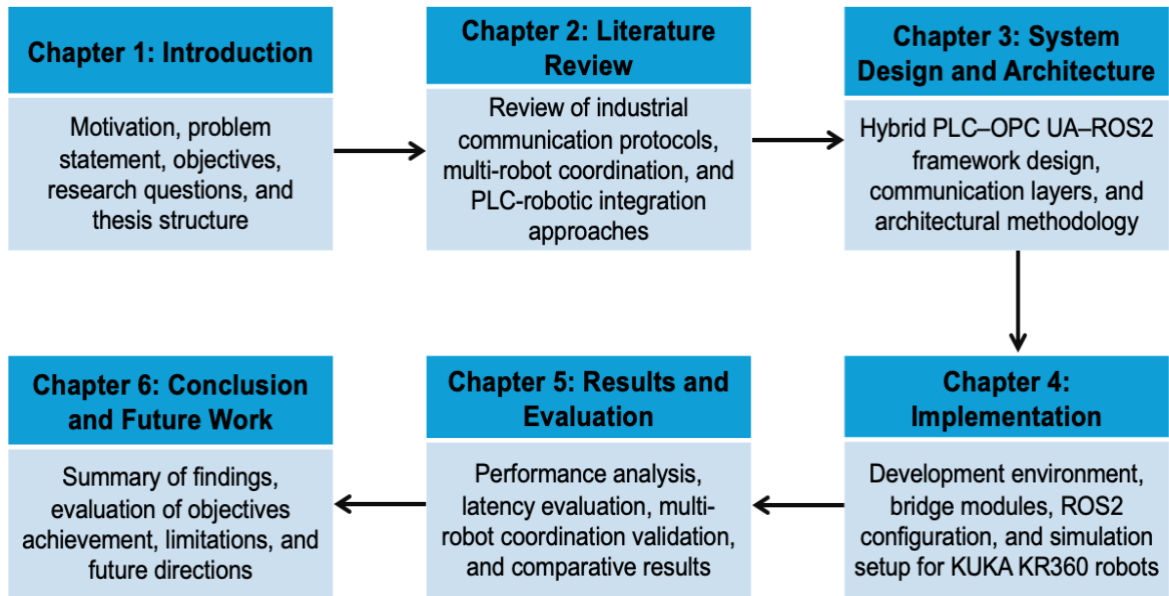


Figure 1.2: Thesis Structure and Chapter Summary

2 Literature Review

The following sections will try to establish the theoretical and methodological groundwork for different kinds of communication protocols. It also looks at frameworks that are already established that can be used for both single-robot and multi-robot communication systems. This chapter addresses the challenges frequently faced in the integration of industrial robotics. There is a comprehensive examination of all the important protocols and communication frameworks. The analysis includes an examination of recent technologies, limitations, and implementation problems. Additionally, the necessity for a novel hybrid communication framework appropriate for multi-robot systems is analyzed. This analysis supports the architectural design and methodological framework presented in Chapter 3.

2.1 Industrial Communication Protocols

The following section introduces traditional and modern industrial communication protocols, explains motivation for a hybrid framework, outlines comparisons Tables 2.1–2.2, and clearly specifies protocol roles: Modbus commands, OPC UA coordination.

2.1.1 Traditional Protocols

Industrial communication protocols (ICPs) are essential for interconnecting systems, and machines in industrial environments, especially with the rise of Industry 4.0 [20]. When discussing "traditional protocols" in industrial environments, the sources primarily highlight Modbus, and Profibus as an older standard that evolved into ProfiNet, alongside Ethernet/IP [20]. At the conclusion of section 2.1.1, we will compare all three protocols using table 2.1, which will be beneficial for section and protocol creation in chapter 3.

1) Modbus Protocol Family

Modbus, developed by Modicon Systems in 1979, is one of the earliest and most widely adopted industrial communication protocols [21]. Initially implemented over serial communication lines (Modbus RTU/ASCII), it was later extended to Ethernet networks via Modbus TCP, adapting to modern infrastructure requirements [21]. The architecture of the protocol sets limitations on the number of devices that can work with it. In theory, both serial and TCP implementations can support up to 247 slave devices, but in practice, only 32 devices per segment without repeaters are usually allowed [21]. Modbus operates using a client-server architecture and functions at the application layer (OSI Layer 7) [22] [23]. An illustration of a Modbus application layer example network is provided in Figure 2.1 Modbus connects with different industrial devices such as PLC, HMI and motor driver. Due to its small data frames and straightforward operation, it remains a practical choice for legacy systems and simple automation setups [23].

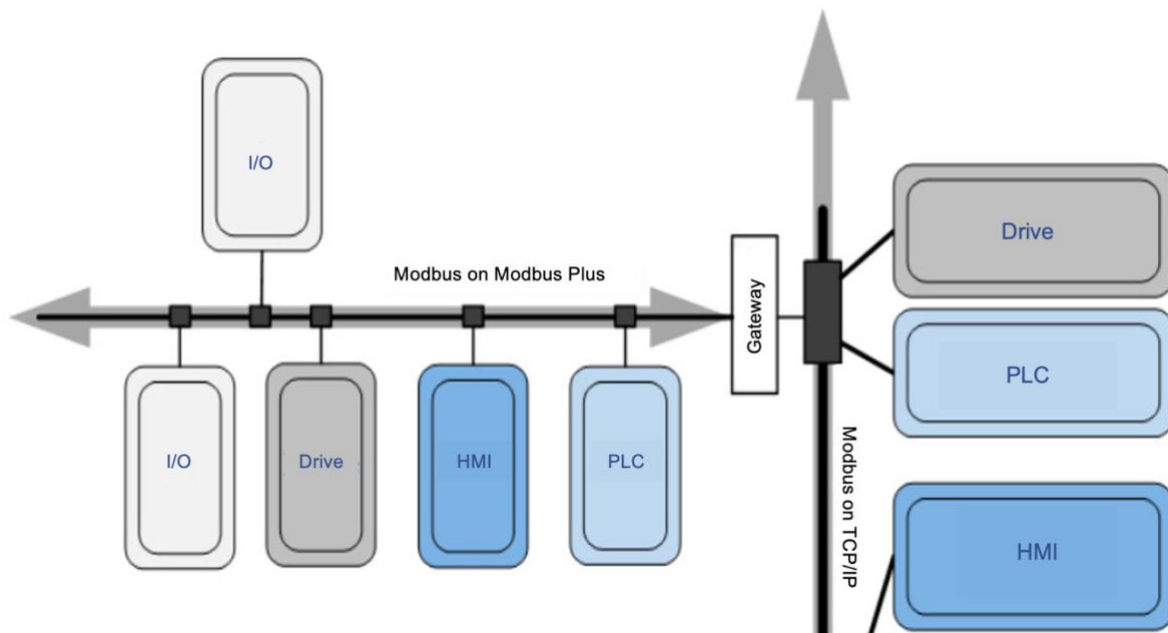


Figure 2.1: Modbus application layer network with HMI, PLC devices [23]

Advantages of Modbus:

- **Reliability:** Provides high dependability in industrial networks with short range and deterministic traffic [20]. Performance studies, on the other hand, show that when the network faces significant amounts of loads, successful request rates drop from 75 requests per second to 33 requests [21].
- **Open and Cost-Effective:** The protocol is publicly available and extensively utilized without associated licensing costs [21].
- **Optimized Message Structure:** The byte-oriented message format and the constrained frame size (maximum 255 bytes) provide it appropriate for predetermined applications, including PLC and automation systems [22]. Performance benchmarks indicate response times of 6-8ms for standard read/write operations under typical conditions [24].
- **Simplicity and Scalability:** The compact design makes it easier to quickly add more components and improve control systems [23].

Disadvantages of Modbus:

- **Limited Data Rate:** Modbus can only send data at much slower speeds than Ethernet-based protocols like Profinet and Ethernet/IP [24].
- **Lack of Built-in Security:** Modbus TCP lacks built-in encryption or authentication, so it can be hacked in networked conditions unless it is protected from the outside [25].

2) Profibus and Profinet

PROFIBUS, which stands for "Process Field Bus," is an older open field digital network standard that was created to connect sensors, actuators, and PLCs that are used in process automation [20]. The protocol was created to make communication reliable in industrial automation environments [20]. It has a client/server architecture. As Ethernet-based solutions became more popular, however, Profinet was created. Compared to PROFIBUS, Profinet offers faster data transmission rates, more flexibility, and better scalability [20].

Profibus and Profinet Advantages:

- Profibus is Widely used in industrial automation components [20].
- Profinet offers advanced security features, and it includes authentication Secure Real-Time Transport Protocol (SRTP) [20].
- Profinet has established itself as the standard for reliable and efficient communication in industrial environments [20].
- Profinet Provides faster data transmission rates, increased flexibility, and improved scalability. And it has a high data rate of up to 1000 Mbps [25].

Profibus and Profinet Disadvantages:

- Profibus has become outdated compared to modern fieldbus protocols like Modbus TCP [20].
- Profinet is not widely supports for simulation-based PLC and it supports only physically PLC for real time implantation [22].

3) Ethernet/IP

Ethernet/IP is an Industrial Communication Protocol (ICP) that uses standard Ethernet technology to allow devices from different brands and technologies send and receive data in real time [20]. Ethernet/IP operates at the application layer of the OSI model and builds on existing TCP/IP protocols. It has much better performance than traditional protocols, with data transfer rates of up to 1000 Mbps and frequencies of 1 to 100 MHz [20]. The protocol includes advanced security features, such as IPSec for data privacy and authentication, which fixes security problems that come with simpler protocols [20].

However, comparative analysis shows that there are major performance losses in certain operational situations. It has been tested and found that Ethernet/IP has the longest variable extraction processing times compared to Modbus and OPC-UA [20]. According to studies on resource utilization, Ethernet/IP uses 45% of the CPU and 26% of the RAM when processing batches of 50 variables, while Modbus only uses 27% of the CPU and 24% of the RAM for the same workloads [21]. Furthermore, Modbus TCP is also called a light protocol with a frame structure that works best for PLC-like systems [22].

Table 2.1 presents a detailed comparison of typical industrial communication protocols based on above research. The research identifies clear trade-offs among performance capabilities, implementation complexity, and resource requirements. Although Profinet and Ethernet/IP provide enhanced speed and sophisticated functionalities, their complexity and resource demands pose challenges for simulation-based research settings. Modbus TCP is the preferred option for straightforward command transmission, despite its scaling constraints, due to its low overhead (27% CPU compared to Ethernet/IP's 45%), superior simulation compatibility, and ease of integration. The protocol's latency performance of 6-8ms is sufficient for fundamental robot commands (HOME, PICK, PLACE), however its constraints in managing complex data warrant the adoption of a hybrid architecture utilizing OPC UA for advanced robotic coordination data. This is obvious from this comparison analysis that choosing a protocol should be based on the needs of the application, rather than just select the fastest solution. This supports the research method used in this thesis.

Table 2.1: Traditional Industrial Communication Protocols Comparison

Criteria	Modbus TCP	Profinet / Profibus	Ethernet/IP
Maximum Data Rate	10 Mbps	100 Mbps	1000 Mbps
Typical Latency	6-8 ms (normal)	1-10 ms	Not available
Device Capacity	247 theoretical and 32 practical	512 devices	1000+ devices
Implementation Complexity	Low - Simple setup	High - Complex configuration	Medium - Standard Ethernet
Licensing Costs	Free - Open protocol	Licensed - Vendor dependent	Licensed – Proprietary
Security Features	None	Advanced - Authentication, SRTTP	Medium - IPSec, authentication
CPU Usage	27%	Not specified	45%
RAM Usage	24%	Not specified	26%
Simulation Support	Excellent - OpenPLC compatible	Limited - Physical hardware only	Limited - Complex infrastructure
Integration with ROS	Good - Via OPC UA bridge	Difficult - Proprietary protocols	Complex - Additional middleware
Best Use Case	Simple PLC commands	Real-time industrial control	High-volume data applications
Main Limitations	Security vulnerabilities	Complexity, cost, simulation limitations	Processing overhead, resource consumption
Justification	Optimal for basic PLC-robot commands in hybrid architecture	Complexity outweighs benefits in simulation environment	Performance overhead unsuitable for simple command transmission

2.1.2 Modern Industrial Communication Protocols

The traditional protocols' limitations outlined in Section 2.1.1, specifically vendor-dependence, inadequate interoperability, and platform-specific constraints, prove insufficient for the demands of Industry 4.0 [26]. Traditional protocols such as Modbus and Profinet were developed for one-to-one client-server communication, resulting in inefficiencies in modern industrial applications that necessitate one-to-many or many-to-one data exchanges among multiple devices [26]. Modern protocols like OPC Unified Architecture (OPC UA) and Message Queuing Telemetry Transport (MQTT) have been developed to fill significant gaps, offering platform-independent interoperability, semantic data modeling, improved security features, and scalable publish-subscribe communication models [26]. Advanced protocols facilitate flexible data exchange among interconnected industrial systems, while also addressing the scalability and security limitations identified in traditional protocol implementations for Industry 4.0 [26]. At the conclusion of section 2.1.2, we will compare both protocols using table 2.2, which will be beneficial for section and protocol creation in chapter 3.

1) Open Platform Communications Unified Architecture (OPC UA)

The main problems with current industrial protocols are explained in Section 2.1.1. OPC UA showed up as a complete solution to these problems. The protocol was created as a service-oriented, technology- and platform-independent standard for implementing Industry 4.0. It enables devices on the manufacturing environment securely communicate to systems in the working environment [26]. As a follow-up to the earlier OPC Component Object Model (COM), OPC UA fixes important problems like Microsoft DCOM dependencies, platform-specific restrictions, and issues with accessing through firewalls [27].

Advantages of OPC UA:

- **Platform Independence:** Enables Linux/Unix systems, embedded controls, and different device architectures to communicate to each other, eliminating some of manufacturer limitation problems that come with outdated protocols [28]. Performance studies show that OPC UA has less than 5ms of latency for direct communication between a client and a PLC, which is faster than Modbus's 6–8ms baseline [29].
- **Complex Data Modeling:** Allows complex the data to be used for semantic data description, which means that processes and assets can be shown in manners that proceed beyond the boundaries of protocols like Modbus [26]. This solves handle of issues with data complexity that were mentioned in Section 2.1.1.
- **Enhanced Security:** It has features like encryption, digital signatures, and certificate verification which are not found in standard protocols like Modbus [26]. Security features solve the identified faults in Modbus implementations that make them unsafe.

- **Scalable Communication:** The publish/subscribe modification helps multicast communication for one-to-many data patterns simpler. This improves the problems with client-server bottlenecks that were found in traditional protocol [26].

Disadvantages of OPC UA:

- **Implementation Complexity:** Complex software infrastructure and many of parameters that can be changed require specialized abilities that make tuning difficult than it is with simpler protocols [26]. This is different from Modbus's advantage of being simple, which was explained in Section 2.1.1.
- **Resource Overhead:** A study by M. Ladegourdie and J. Kua indicates that OPC UA requires up to 18.5% of CPU and 74.3% of RAM on edge devices. This contrasts with Modbus, which only uses 27% of CPU [30]. Additional network overhead is created when 215KB of bandwidth is used instead of 118KB for Modbus [30].
- **Performance Trade-offs:** When compared to simpler protocols, cloud-based implementations have an average response time of 162ms, but safety features might add up to 15.9 seconds to the time it takes to start a session [26]. Performance drops when there are a great number of clients (1.2 seconds for 1000 clients), which makes it hard to scale up in some situations [26].

2) Message Queuing Telemetry Transport (MQTT)

MQTT is a simple, application-level messaging protocol that operates over TCP/IP. It was created by IBM and is now standardized by OASIS for communication between machines and the Internet of Things (IoT) [26]. Because MQTT is designed for environments with limited resources that need a small code footprint and low network bandwidth usage, it is fundamentally different from OPC UA because it uses a central broker to distribute messages in a publish/subscribe approach [26]. While OPC UA focuses on semantic data modeling and complex information structures, MQTT is only concerned regarding how rapidly messages are transmitted and has no concerns what message has been transmitted [28].

Advantages of MQTT:

- **Lightweight Performance:** Performance evaluations indicate that MQTT has faster transmission times than OPC UA, particularly when sending messages to many subscribers (471ms for 1000 clients against 1.2 seconds for OPC UA) [26]. In environments with limited bandwidth, higher efficiency is achieved with less connection initialization and data message overhead [28].
- **Scalable Architecture:** The publish/subscribe architecture based on brokers is excellent at scaling, and it handles communication effectively in IoT deployments with many devices and digital twin systems [27]. This improves

the problems with scalability that were found in old client-server protocols compared to OPCUA.

- **Cloud Integration:** MQTT provides simple to collect and send data from edge devices to cloud servers, which meets the needs of Industry 4.0 for more sensor data and measurements [28]. The design of the protocol allows feasible to achieve IoT and cloud integration which is not possible with traditional industrial protocols.
- **Network Efficiency:** Quality of Service (QoS) levels (0, 1, 2) make sure that messages are delivered reliably and securely while remaining lightweight [26]. It operates effectively on connections which have significant amounts of latency or are not reliable, so it can be used in distributed manufacturing environments.

Disadvantages of MQTT:

- **Insufficient for Industrial Robotics:** Regardless of these advantages, MQTT has certain drawbacks. It is usually not enough to manage complex industrial robots [28]. In contrast to OPC UA's complex information model advantages it indicates that MQTT is not an appropriate choice for the robotic synchronization demands identified in this research.
- **Real-time and Deterministic Limitations:** subsequently attempt to connect industrial hardware like Siemens PLCs, it operate to issues with timing because the native clocks are limited to up to 10 Hz, which can cause 100ms of delay and make the system unstable [28]. Performance studies show that when the CPU is occupied, operations slow down significantly (about 300 μ s), which means that execution has been delayed [29].
- **Security Deficiencies:** : By default, MQTT lacks all standard security features, so it sends authentication information in plaintext and utilizes outside protocols like TLS to send data securely [26]. This represents a significant disadvantage compared to OPC UA's integrated encryption, authentication, and authorization features documented in OPCUA.

In Table 2.2, it finds a complete comparison of modern industrial communication protocols that have been tested for applications involving coordinating multiple robots. The study finds two different approaches to architecture: MQTT focuses on efficient lightweight messaging for IoT applications, while OPC UA focuses on complete data modeling and robust industrial security. Even though it uses more resources (18.5% CPU, 74.3% RAM), OPC UA is the ideal choice for industrial robotics because it has advanced semantic data modeling features that are necessary for managing complex robot coordination data. The protocol achieves 5ms latency in direct PLC communication and includes a full security framework, meeting the complex requirements of the multi-robot systems described in this research.

MQTT, on the other hand, is not suitable for predetermined robot control applications because of its lightweight architecture and strong cloud integration features. The rate at which messages are sent is faster with MQTT (471ms for 1000 clients) than with OPC UA (1.2 seconds). However, because it lacks concern about the content and has timing limits with industrial hardware, it is hard to use for real-time robotic coordination. Based on this study, OPC UA's complexity trade-offs indicate industrial robotics uses that need precise information semantics, reliable communication, and strong security. The chosen protocol fits with the hybrid architecture method, which is explained in Section 2.1.1. It uses OPC UA for complex robotic data and keeps Modbus for basic PLC commands

Table 2.2: Modern Industrial Communication Protocols Comparison

Criteria	OPC UA	MQTT
Maximum Data Rate	Implementation dependent up to 100 Mbps	100 Mbps
Typical Latency	5–162 ms	300 μ s–471 ms
Device Capacity	1000+ clients (with degradation)	Unlimited (broker-dependent)
Implementation Complexity	High- Complex configuration	Low - Lightweight setup
Licensing Costs	Free	Free
Security Features	Advanced - Encryption, certificates, digital signatures	Basic - External TLS required
CPU Usage	Varies: • 2.06% (Unity) • 2.32% (PLCSIM) • 0.27% (Node-RED) • Up to 18.5% (RPi 3B, 30 nodes)	Varies: • 4.65% (Unity) • 1.75% (PLCSIM) • 0.18% (Node-RED)
RAM Usage	Varies: • 9.37 MB (Unity) • 1.46 MB (PLCSIM) • 1.16 MB (Node-RED) • Up to 74.3% (RPi 3B)	Varies: • 9.37 MB (Unity) • 1.48 MB (PLCSIM) • 1.09 MB (Node-RED)
Simulation Support	Excellent – Compatible with Digital Twin (Unity), Node-RED, PLCSIM	Excellent suited for simulation environments and cloud platforms
Integration with ROS	Excellent – Needs bridges (e.g., OPC UA-ROS2), more effort for real-time systems	Good – ROS-MQTT bridges widely available; simpler than OPC UA
Network Bandwidth	High - 215KB overhead	Low - Minimal bandwidth usage

Best Use Case	Complex industrial automation, robotics	IoT messaging, sensor data distribution
Main Limitations	Resource overhead, complexity	Insufficient for real-time control, security gaps
Justification	Optimal for complex robot data modeling and industrial integration	Inadequate for deterministic robot control requirements

2.2 Multi-Robot Communication

While Sections 2.1.1 and 2.1.2 discussed several significant issues with traditional and modern methods for communicating, multi-robot systems make coordination, synchronization, and scale requirements even more difficult. Multiple robots need to be coordinated at the same time, which requires real-time data exchange patterns, secure execution, and scalable communication structures that extend single-protocol solutions to multirobot [31]. The second section studies multi-robot communication architectures, real-time coordination requirements, and scaling challenges to illustrate why existing approaches are not appropriate for industrial multi-robot applications and why a novel hybrid communication framework developed in this research is required.

2.2.1 Communication Architectures for Multi-Robot Systems

Multi-robot coordination occurs when a group of robots works together in the same space to achieve a specific objective. The robots consider what other robots do to make the system work effectively together [31]. Communication architecture is essential for multi-robot coordination because robots need to communicate with each other safely and accurately to operate together. It is such a "bridge" that helps multiple robots communicate with one another [32]. If the system lacks a proper communication design, adding multiple robots together can cause issues such as crashes, conflicts, and system failures because there are not sufficient methods for all of them to communicate with each other. Based on their information exchange patterns and control methods, multi-robot communication architectures can be generally divided into three primary types [33]:

1) Centralized Communication Architecture:

In centralized architecture, every part of the robot operates by a single decision-making device, or robot node [34]. This central controller receives data from all the robots, processes it, and then utilizes that data to send commands to each robot [34].

Advantages of centralized systems:

- **Simplified Coordination** Central control helps make it simple for robots to operate together on operations, which decreases the risk of confusion and unnecessary work [34].

- **Ease of Implementation:** This architecture has simpler algorithms because only one powerful device operates all commands, which makes it easy to use in smaller groups [34].
- **Global Optimization:** Complete information access helps improve the overall efficiency of the group and enables informed decisions that benefit the whole group [35].

Disadvantages of centralized systems:

- **Single Point of Failure:** The reliability of the entire system depends upon a single component; if the central controller fails, the entire system stops operating [31, 34].
- **Scalability Issues:** As the number of robots increases, the central controller receives excessive amounts of input, which slows down decision-making, resulting in slower operations [34].
- **Communication Overhead:** Continuous communication between robots and a central controller slows operations down in areas where connectivity fails [34].

2) Decentralized Communication Architecture:

Decentralized control allows various robots to decide based on local information and interactions with neighbourhood robots, making the entire structure work like a swarm of robots without a single leader [31]. There is no central monitoring node in this framework, allowing each robot to send and receive messages from other robots on its network [34].

Advantages of decentralized systems:

- **Robustness:** This type of architecture is more fault-tolerant because there is no single point of failure; each robot maintains operation even if others suffer from issues [36].
- **Scalability:** Adding more robots is easy because each unit operates on itself, resulting in simple computation and communication [36].
- **Flexibility and Adaptability:** Robots can quickly change the environment or adapt to different operations based on the events going on within the surroundings [36].

Disadvantages of decentralized systems:

- **Local optimization:** when there is no global monitoring, systems might not be able to determine the optimal solutions for all, which could lead to less effective results [34].

- **Complex Coordination:** To achieve coordinated behaviour without a central controller, complex methods are required that are difficult to develop and execute [34].
- **Inconsistent Behaviour:** When robots make independent decisions, it can cause unpredictable behavior [34].

3) Hybrid Communication Architecture:

Combining aspects of centralized and decentralized control, hybrid approaches aim to utilize the advantages of both while reducing the drawbacks [33]. These systems can combine decentralized communication within a group with centralized communication between groups through group heads. This makes it possible to achieve synchronization and significant levels of scalability at the same time [33]. Both explicit communication (sending and receiving messages) and implicit communication (learning from changes in the surroundings or robot behaviour) can be used in combined hybrid architectures to make them more flexible and allow them to work together [33].

2.2.2 Real-time Coordination Requirements

Multi-robot coordination needs extremely high real-time performance that is much higher than what a single robot is capable of, as tasks are distributed and coordination data terminates quickly. While Sections 2.1.1 and 2.1.2 demonstrated individual protocol limitations, multi-robot systems amplify these through exponential timing synchronization and deterministic behavior requirements [37].

• Industrial Multi-Robot Timing Requirements

When controlling and operating multiple robots, strict deadlines are needed for the perception-actuation loops. If data arrives late, it can damage the robots or make it impossible to finish the task [37]. Multi-robot industrial applications impose specific quantitative timing constraints essential for safe coordination. Deterministic behavior requires communication cycles within 1ms periods with jitter below 200 μ s for smooth joint motion coordination [38].

• Protocol Performance in Real-time

Y. Ye et al. claim that performance evaluations indicate important differences between communication protocols when real-time demands are placed on multiple robots. ROS1 has major timing issues, with a mean jitter of 20.1ms and message delivery rates dropping to 7.8% in three-node setups. The maximum scheduling delays can reach 1280 μ s, which is exceptionally high for the industry standard of 1ms [38]. ROS2, on the other hand, was designed to solve the issues that existed with ROS1. ROS2 with DDS middleware has a maximum latency of 0.3ms, while ROS1's occurred >20ms

under the same conditions. When the Real-Time Operating System (RTOS) is improved, ROS2 has a maximum scheduling latency of only 26 μ s while meeting 1ms temporal requirements [39]. TCP-based communications have significant issues, such as losing a significant number of messages and Nagle's algorithm creating period distributions that fail with strict control loops. Priority-based methods, on the other hand, have a mean jitter of 2.0ms and 99.4% message delivery rates [39].

- **Timing Failure Consequences**

When real-time standards fail, failures occur one after the other, which has extremely harmful effects. Robot crashes, path deviations, and system instability are all caused by significant delay, and network congestion from choosing an incorrect protocol increases the chance of robots colliding with each other [38]. This study shows that different protocols fail to satisfy all the objectives of multiple robots. For example, Modbus has simple, deterministic commands but no complex data structures. In contrast, OPC UA has industrial features. However, additional computing power is required, and ROS2 has better real-time performance but fails to integrate with industrial protocols [39]. These results show that industrial applications with multiple robots need hybrid communication architectures that utilize the advantages of each protocol's strengths while reducing its limitations. These create foundations for the research technique utilized in this thesis.

2.3 Industrial Robotics Integration Challenges

While Section 2.2 established the complex timing and coordination requirements of multi-robot systems, the integration of robotic platforms with established industrial control introduces additional challenges that current solutions insufficient. This section discusses the challenges associated with current robotic middleware in industrial environments, traditional methods of integrating PLCs and robots, and the requirements for effective hybrid communication frameworks.

2.3.1 ROS2 Industrial Applications and Limitations

ROS2 includes numerous useful features for industrial robotics applications. It has been used widely in numerous industry domains, such as autonomous material handling, aerospace operations, and defense applications [40]. Among its advantages for industry are Data Distribution Service (DDS) middleware that enables multiple robots to communicate with each other, QoS protocols that ensure data flows accurately, and built-in security systems that can authenticate and encrypt data [40]. Performance evidence shows ROS2's superior scalability, with OTTO Motors successfully scaling from 25 robots under ROS1 limitations to over 100 robots through ROS2's distributed architecture [41].

However, ROS2 has issues communicating with traditional industrial control systems, especially in places where PLCs are the primary kind of control system. Studies reveal that ROS2 is not natively supported by most current PLC manufacturers, requiring

custom frameworks or intermediary protocols such as Modbus TCP or OPC UA for data exchange [42]. Data type compatibility issues exist between ROS2's flexible unbounded arrays and IEC 61131-3 PLCs requiring fixed-length structures, necessitating manual truncation and adaptation procedures [42]. Furthermore, activating ROS2 security features increases operational costs and consumes more resources, which is a problem in industry environments that are wireless and limited in resources [43].

Even though ROS2 has better real-time features than ROS1, it is still hard to achieve stable execution in complex industrial multi-robot situations. Callback scheduling is challenging to predict, and execution times change significantly. These can cause distributed manufacturing systems to miss deadlines and have buffer overflows [43]. There is a significant amount of uncertainty in highly developed systems because of network delays and jitter. Message loss can happen even with Stable QoS environments when the network is extremely occupied [44]. When combined with the precise timing requirements listed in Section 2.2.2, these limits become critical. For example, industrial coordination requires accuracy to the millisecond level, with communication cycles that are always 1ms.

This study indicates that ROS2 allows significant improvements to robotic middleware, but it cannot satisfy all the integration requirements of industrial multi-robot systems on its own. There are clear gaps in the compatibility of existing systems, the reliability of performance guarantees, and the simplicity of integrating industrial protocols, indicating a need for hybrid communication methods. These challenges make it critical to examine traditional methods of integrating PLCs and robots to determine whether they can be utilized on a larger scale. These will help us understand the industrial communication requirements we discuss in Section 2.3.2.

2.3.2 PLC-Robot Integration Approaches

Section 2.3.1 highlights the limitations of ROS2 in industrial integration. Traditional Programmable Logic Controller (PLC) methods have historically ensured reliable control of industrial robots; however, they encounter specific challenges related to scalability and coordination in contemporary multi-robot systems.

Programmable Logic Controllers have served as the backbone of industrial robot control for decades, providing deterministic, robust operation in tough manufacturing environments through direct I/O wiring and established fieldbus protocols including Modbus TCP, EtherNet/IP, and PROFINET [45]. Standard integration approaches include direct sensor-actuator wiring to PLC I/O cards, remote communication via Ethernet using server-client models, and external coordination systems where PLCs stream commands to robot controllers [46]. These methods utilize the advantage that PLCs are accessible, simple to program using known ladder logic, can be controlled from anywhere, and are more reliable in industries [47, 48].

Traditional PLC-robot integration methods, on the other hand, had significant challenges in multi-robot coordination situations. Communication bandwidth limits become significant bottlenecks. For example, fieldbus systems can only send data at a certain speed, like 2.5 Mb/s. High-frequency encoder data from a single robot can use up to 1.8 Mb/s, quickly using up all available bandwidth when multiple robots are working at the same time [49]. As more robots struggle for access to a shared workspace, coordination becomes much difficult due to the need to prevent collisions and determine whose receives what resources. This is challenging for traditional PLCs to execute because it works in cycles [50].

When PLCs are utilized to achieve the high real-time standards established in Section 2.2.2 for multi-robot coordination, they operate into significant timing issues. Sequential input reading, logic processing, and output updating are all slowed down by PLC cycle times. For example, it has been shown with 200ms input reading cycles and 25ms speed limits because of I/O card limitations [51]. Distributed architectures add unexpected jitter and latencies across communication levels, which makes it challenging to achieve the precise millisecond timing needed for industrial multi-robot synchronization [52]. These timing issues make it impossible to correctly apply high-frequency disturbance compensation and dynamic trajectory adaptation, which are needed for precise multi-robot operations [53].

Analysis indicates that PLCs are more effective at industrial robustness and stable control than the ROS2 limitations listed in Section 2.3.1. However, they cannot fulfill the entire coordination, and flexibility demands of current multi-robot systems on their own. Scaling beyond single-robot control leads to bandwidth losses, synchronization problems, and the need for complicated solutions, including localizing control loops to avoid the conflict [54]. This creates a fundamental gap where neither pure ROS2 approaches nor traditional PLC methods can satisfy the complete spectrum of industrial multi-robot integration requirements, establishing the critical need for hybrid communication frameworks examined in Section 2.3.3.

2.3.3 Hybrid Communication Framework Requirements

As per S. Chitikeshi's recent investigations, the problems with integration discussed in Sections 2.3.1 and 2.3.2 have been addressed by using hybrid communication frameworks that combine industrial protocols with robotic software. Among these methods are Industrial Internet of Things (IIoT) Gateway frameworks that use OPC UA, Modbus, MQTT, and JSON protocols to connect PLCs to robot platforms [55]. Moreover, ROS2-based Industrial Control Systems (RICS) integrate OpenPLC with ROS2 motion control systems using special function blocks [43]. Also, UA Bridges create logical connections between ROS and OPC UA Publish/Subscribe (Pub/Sub) networks so that multiple robots may operate together [56]. A UA bridge is like a logical gateway because it has two interfaces: one for communicating with other devices in the same ROS space, and another for communicating with other OPC UA devices through an OPC UA Pub/Sub interface. If it receives an OPC UA Pub/Sub message, it changes it to a ROS message and sends it to the ROS space, and vice versa.

According to the Eclipse Arrowhead Framework, distributed integration is demonstrated when PLCs with OPC UA servers work as separate systems and ROS-based robots work in a different server [57].

The examination of these hybrid systems' performance indicates they have significant issues for industrial multi-robot applications. The IIoT Gateway is highly available and stable, and the CPU usage remains at 25% regardless of the number of OPC UA nodes connected [58]. RICS, on the other hand, operates reliably with less than 8% CPU usage during continuous monitoring situations [43]. However, when there are multiple robots, timing issues become crucial. UA Bridge implementations have delays of 2–10 seconds for data transfer between ROS spaces and OPC UA networks, which is significantly greater than the millisecond-level accuracy standards set out in Section 2.2.2 [56]. These delays become more difficult when connecting multiple protocol layers. Implementations not properly optimized add infinite latency that is not compatible with the needs of predetermined industrial coordination.

When looking at what has previously been accomplished, it shows that for hybrid industrial-robotic frameworks to work, they are required to achieve precise technical standards in the areas of interoperability, performance, and scalability. Some important requirements are that different PLC devices and enterprise IT systems should be able to communicate with each other without any problems, that different operational technology (OT) protocols (such as OPC UA, Modbus, and EtherCAT) should have unified interfaces, and that manufacturing components should be able to be deployed on any platform [59]. Real-time performance requirements include millisecond data delivery that meets the deterministic timing requirements from Section 2.2.2, a reliability level of more than 99% uptime for continuous manufacturing, and scalable architectures that can support changing multi-robot configurations without affecting performance [59]. Robust authentication, fault-tolerant operation with automatic failure recovery, and data integrity preservation across protocol translation layers are all requirements for security.

Even with these advancements, there are still important vulnerabilities in the current hybrid methods that make it difficult for industrial multi-robots to communicate effectively. There is evidence that current solutions cannot deal with both the problems with ROS2 industrial integration listed in Section 2.3.1 (native PLC support, data type compatibility, and security overhead) and the problems with PLC multi-robot coordination listed in Section 2.3.2 (bandwidth saturation, synchronization failures, and timing limitations) at the same time, while fulfilling the extremely high real-time requirements listed in Section 2.2.2. The current frameworks are complex to configure and operate, so they require manual OPC UA node mapping. The current frameworks also have network latency that fails with timing, as well as not enough AI/ML integration for smart coordination. The research presented here indicates the need for a novel hybrid framework that enables stable multi-robot coordination through improved protocol bridging, automated configuration methods, and built-in performance monitoring. These novel frameworks of architectural design and methodology are briefly discussed in Chapter 3.

3 System Design and Methodology

This chapter explains how to develop a novel hybrid communication framework based on the in-depth literature review in Chapter 2, which observed critical gaps in existing industrial multi-robot communication methods. The suggested method solves the issues with ROS2 industrial integration listed in Section 2.3.1, the difficulties with PLC multi-robot coordination listed in Section 2.3.2, and the performance challenges with current hybrid approaches listed in Section 2.3.3. This chapter establishes the system requirements based on previous research. This chapter presents the architectural design that connects industrial control systems with robotic middleware and the implementation strategy that enables it to be practical for multiple robots (dual robots) to operate together in single simulation environment.

3.1 System Requirements and Design Objectives

The extensive examination of the literature in Chapter 2 defines specific functional and performance benchmarks that a hybrid industrial multi-robot communication framework must fulfill. Given the limitations of traditional and modern industrial protocols, the system needs to be able to rapidly communicate via traditional industrial protocols (like Modbus TCP and PROFINET) to modern robotic middleware (like ROS2 and OPC UA) while predictably maintaining its operation. The problems with integrating ROS2 into industry that are described in Section 2.3.1 require native PLC compatibility, data type adaptation methods, and security optimization to maintain latency as low as possible in simulation environments [42, 43]. The PLC and multi-robot coordination constraints from Section 2.3.2 additionally indicate, for bandwidth optimization, synchronization methods, and a scalable design that can support the coordination of multiple robots without communication bottlenecks [49, 52].

The precise timing limits provided in Section 2.2.2 for industrial multi-robot coordination directly affect the performance standards. To make sure that joint motion coordination occurs effectively, the framework needs to be able to achieve established communication cycles every 1ms with jitter below 200ms. This fixes the timing problems that have been found in previous protocol implementations [38, 39]. To meet industrial responsiveness standards, system delay must always be less than 200ms from end to end. This is an important advancement compared to the 2–10 second delays seen in current hybrid solutions, which are described in Section 2.3.3. Scalability requires at least two robots to work together, and it is possible for the system to expand to include more robots. It still requires maintaining accurate timing to within a millisecond and sending messages 99.4% of the time, as shown in optimized ROS2 implementations [44].

The integration requirements address issues with compatibility that have been identified in Chapter 2 between different types of industrial and robotic systems. IEC 61131-3-compliant PLCs and ROS2 nodes must be enabled to communicate to each

other in both directions, and the solution must be possible to deal with data type mismatches between unlimited ROS2 arrays and fixed-length PLC structures [42]. To support previous technologies, it requires Modbus TCP connectivity so that a variety of industrial equipment can work with each other, OPC UA middleware for modeling semantic data and standardizing the industry [26, 27], and the capacity to implement across a wide range of manufacturing environments without thinking about the platform. The framework must also support namespace isolation for situations with multiple robots, automated configuration to reduce the time required for manual mapping (as described in Section 2.3.3), and modular design that helps robots operate in workspace and in future it will add more robots in this setup.

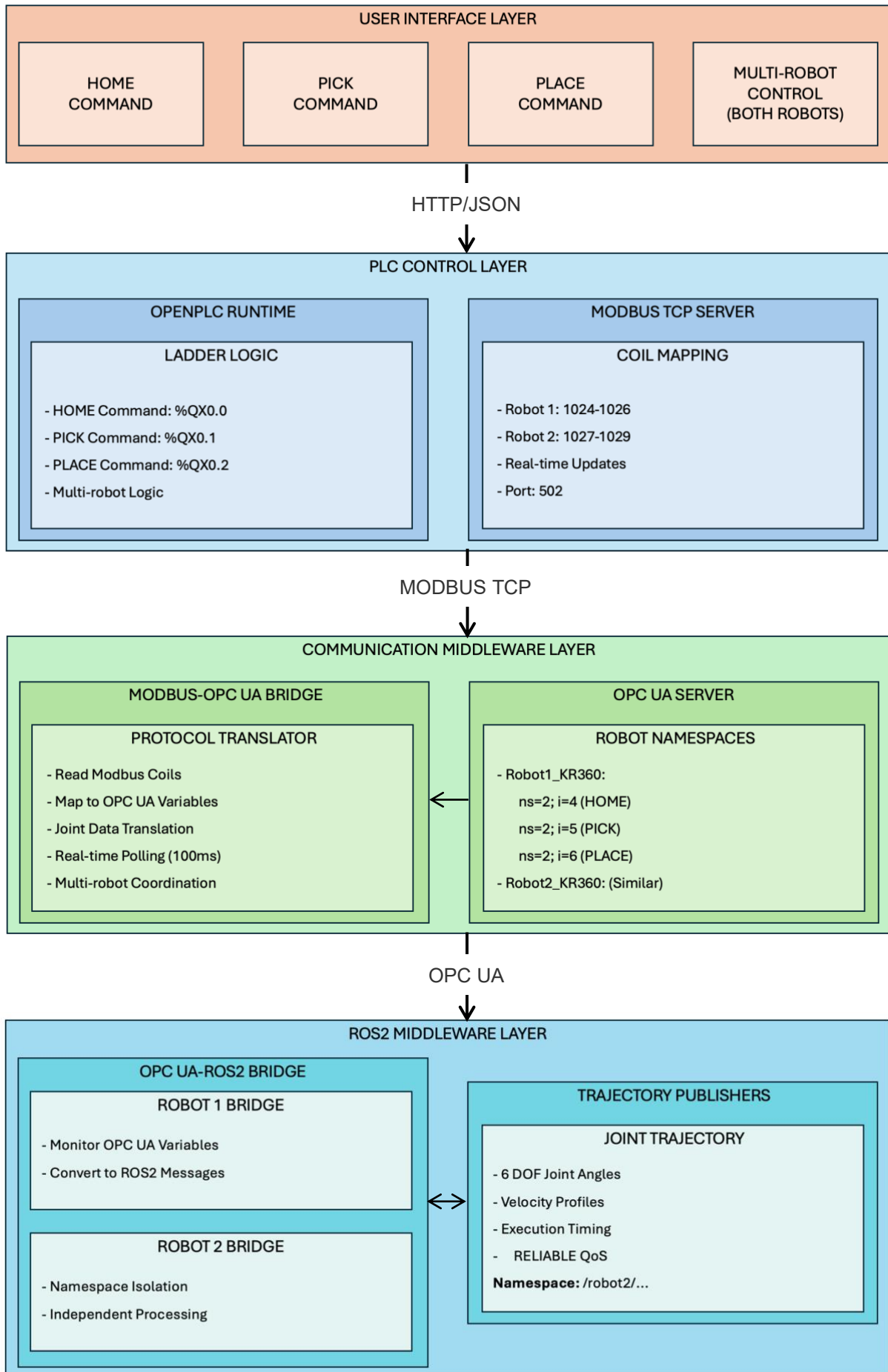
These requirements lead to four key design objectives: integrate PLC control logic with ROS 2 via standard OPC UA to bridge existing gaps; ensure deterministic multi-robot coordination that meets industrial timing requirements and scales beyond typical PLC limits; adopt a modular architecture that allows independent development, testing, and deployment while preserving performance and reliability; and optimize performance to achieve sub-200 ms response times with minimal resource use, addressing the latency and efficiency shortcomings of current hybrid systems. These objectives form the foundation of the architectural design presented in Section 3.2.

3.2 Hybrid Communication Architecture Design

The hybrid communication architecture design demonstrated in Section 3.2 provides a comprehensive solution to the difficulties that arise when integrating multiple robots in industry. The five-layer modular framework connects PLC industrial control to ROS2 robotic software through standard OPC UA communication. This helps the entire process work together efficiently and maintains the capacity to expand if required. This architectural foundation addresses gaps in existing solutions' performance and integration, and it provides the design with the structure required for the systematic implementation and validation discussed in Section 3.3.

3.2.1 Overall Framework Architecture

The hybrid communication framework architecture addresses the design objectives established in Section 3.1 through a five-layer modular design that bridges industrial control systems with robotic middleware. Figure 3.1 provides an in-depth overview of the architecture, showing how each layer contributes to making protocol integration simple while achieving the demanding performance standards of less than 200ms response times and 1ms communication cycles. The architecture is divided according to a hierarchy, with user interface commands moving through industrial control logic, protocol translation middleware, robotic communication layers, and robot execution systems. This makes sure that everything can be tracked from beginning to end and that performance is optimized.



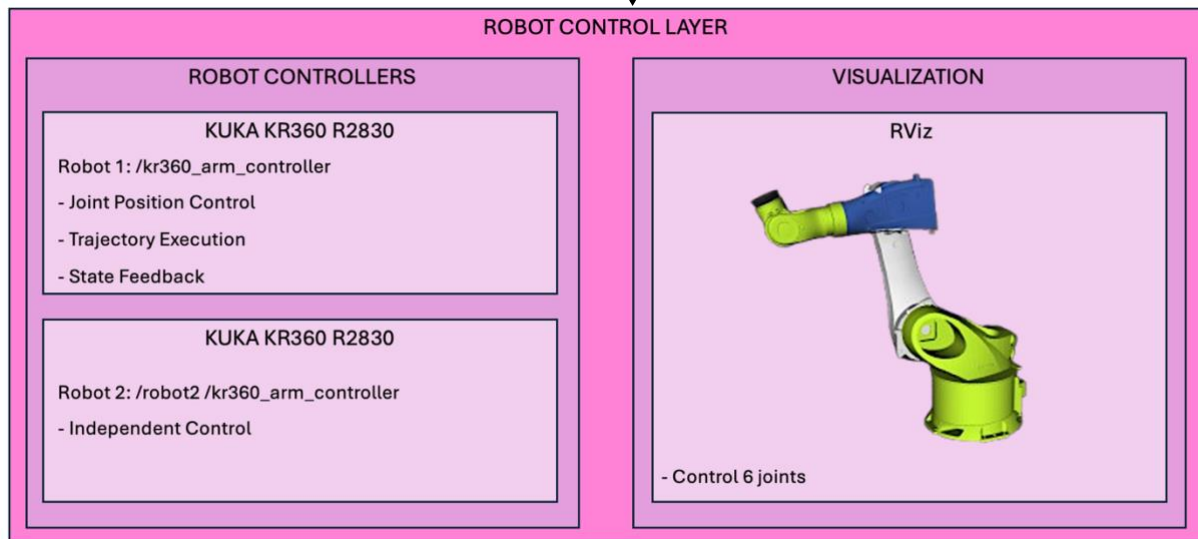


Figure 3.1: Hybrid Communication Framework Architecture

The five architectural layers each have their own set of functions, but they are not firmly attached. This allows the system to be flexible and for each component to be developed independently, depending on the design objectives. The User Interface Layer offers its issue commands through web-based controls that support HOME, PICK, and PLACE actions for both single-robot and multi-robot setups. This fulfills the requirement from Section 3.1 for automated configuration. The PLC Control Layer uses OpenPLC runtime and IEC 61131-3 ladder programming to perform industrial logic. It sends control signals via Modbus TCP protocol to ensure existing systems can still work with it. The Communication Middleware Layer connects the industrial and robotic domains by implementing an OPC UA server and translating protocols between Modbus and OPC UA namespaces in real time. This fills in the gaps in existing solutions' integration. The ROS2 Middleware Layer changes OPC UA variables into ROS2 trajectory messages using specific bridge nodes that separate namespaces to support multiple robots. This makes sure that the design can expand without communication problems. The Robot Control Layer uses the `ros2_control` framework to execute motion commands. It also has RViz for monitoring efficiency and visualization, enabling the deterministic multi-robot coordination objective to be achieved.

This layered architecture fulfills the goal of integrating hybrid protocols by giving each layer a well-defined set of interfaces while maintaining the optimizations for specific protocols within layers. The third design goal is achieved by the modular design, which allows components to be tested and deployed separately. The performance optimization goal is also achieved by minimizing the communication overhead between layers and making the data structures as efficient as possible. Scalability of the architecture is demonstrated by namespace isolation mechanisms that enable easy switching from single robot to multi-robot configurations without making any changes to the architecture. This achieves the requirements for scalability set out in Section 3.1.

3.2.2 Single Robot Communication Architecture

The single robot communication pipeline establishes the foundational data flow that addresses the ROS2 industrial integration challenges identified in Section 2.3.1 while meeting the sub-200ms latency requirements from Section 3.1. Figure 3.2 details the complete communication sequence from user command initiation to robot trajectory execution, demonstrating how each protocol translation maintains data integrity while optimizing performance.

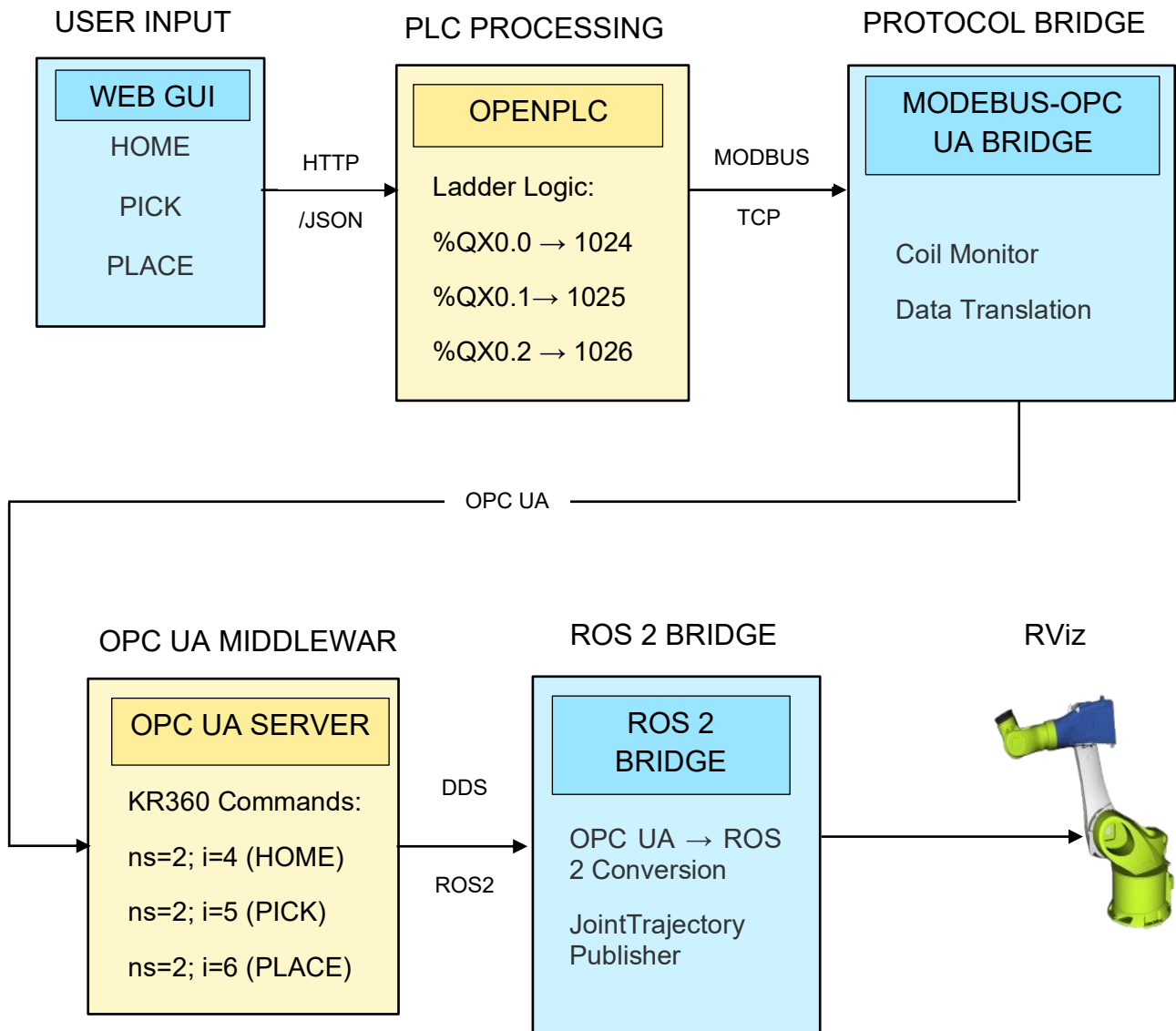


Figure 3.2: Single Robot Communication Architecture

The communication pipeline begins with user command input through the web interface, where operators select predefined robot actions (HOME, PICK, PLACE) transmitted via HTTP/JSON protocol to the OpenPLC runtime. The OpenPLC system processes these commands through IEC 61131-3 ladder logic, converting high-level commands into specific digital output signals mapped to Modbus TCP coil addresses

(1024 for HOME, 1025 for PICK, 1026 for PLACE). This approach addresses the native PLC compatibility requirement while maintaining the deterministic behavior specification from Section 3.1.

The Modbus-to-OPC UA bridge component continuously monitors PLC output coils at 100ms polling intervals to meet the 1ms communication cycle requirement, detecting state changes and translating them to corresponding OPC UA variable updates. Upon detecting coil activation, the bridge writes predefined joint angle arrays to OPC UA nodes (ns=2; i=4, ns=2; i=5, ns=2; i=6) representing the three robot positions, addressing the data type adaptation mechanism requirement. This translation process handles the data type mismatches between Modbus Boolean signals and ROS 2 float64 arrays while maintaining semantic consistency through the OPC UA information model.

The ROS 2 bridge node monitors OPC UA variables through subscription mechanisms, immediately detecting joint angle updates and converting them to JointTrajectory messages containing six-degree-of-freedom position commands with velocity profiles and execution timing. The trajectory publisher transmits these messages via DDS middleware to the robot controller topic (/kr360_arm_controller/joint_trajectory), ensuring the 99.4% message delivery ratio requirement through RELIABLE QoS settings. The robot controller processes trajectory commands through ros2_control framework, executing smooth motion interpolation while providing real-time feedback through joint state publishers, completing the bidirectional communication requirement and enabling performance monitoring throughout the execution pipeline.

3.2.3 Multi-Robot System Architecture

The multi-robot system architecture extends the single robot pipeline to support coordinated dual-robot operations while maintaining the performance and scalability requirements established in Section 3.1. Figure 3.3 illustrates the namespace isolation and parallel communication pathways that enable simultaneous robot control without the communication bottlenecks identified in traditional PLC approaches from Section 2.3.2.

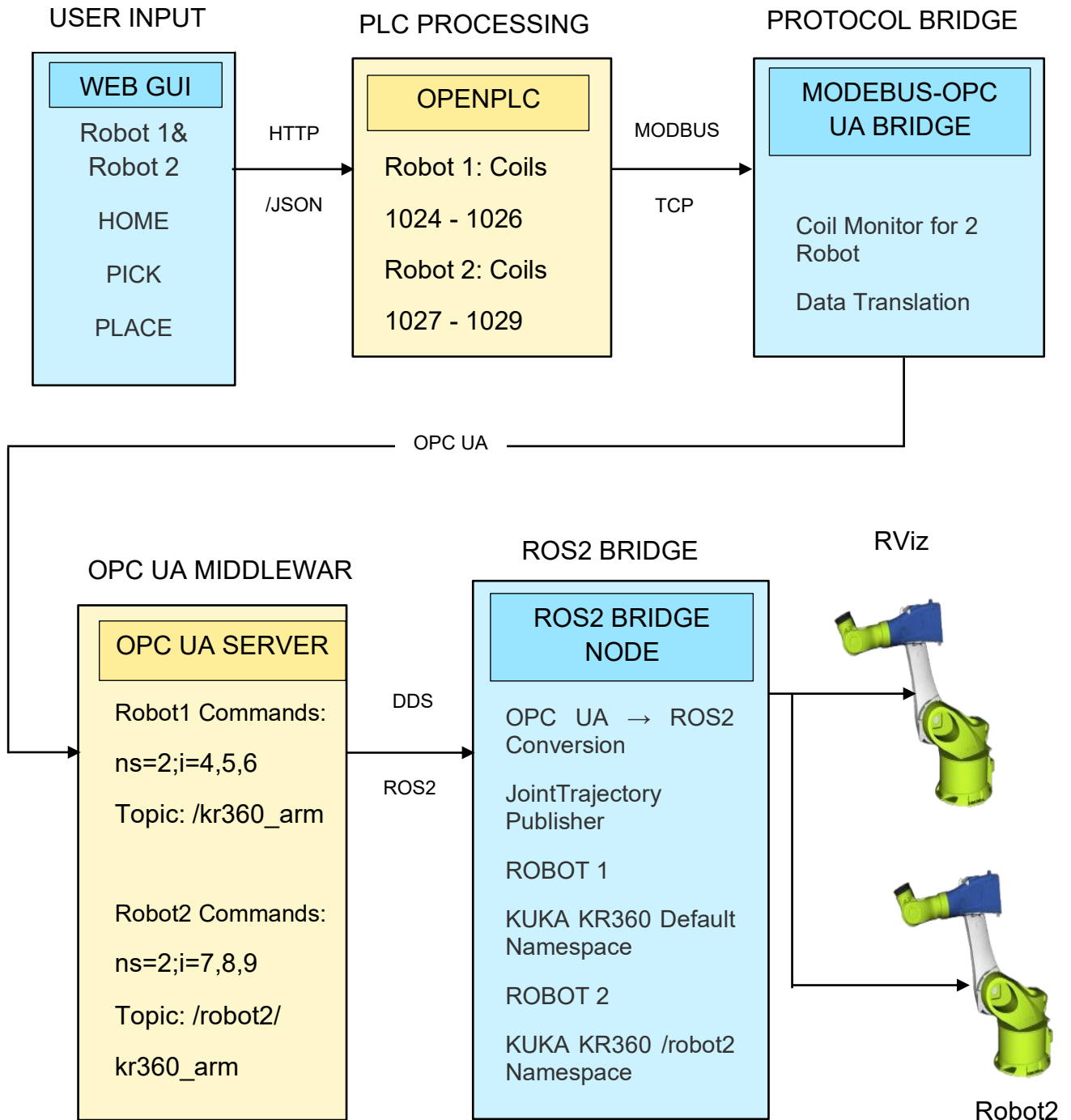


Figure 3.3: Multi-Robot System Architecture

The multi-robot architecture implements namespace isolation through dedicated communication channels for each robot while sharing common infrastructure components. Robot 1 utilizes Modbus coil addresses 1024-1026 mapped to OPC UA namespace Robot1_KR360, while Robot 2 employs coil addresses 1027-1029 mapped to Robot2_KR360 namespace, ensuring independent command processing without signal interference. This approach addresses the bandwidth optimization requirement by distributing communication load across separate channels while maintaining the sub-millisecond timing precision through parallel processing architectures.

The OPC UA server maintains separate object hierarchies for each robot, enabling independent joint position variables and command states while providing centralized monitoring capabilities. The dual Modbus bridge monitors all six coil addresses simultaneously, updating corresponding OPC UA variables for each robot namespace without blocking operations, addressing the synchronization mechanism requirement. This design eliminates the bandwidth saturation issues identified in Section 2.3.2 where single PLC systems struggle with multiple robot coordination due to sequential processing limitations.

ROS 2 integration employs dedicated bridge nodes for each robot (`robot1_ros_bridge.py`, `robot2_ros_bridge.py`) operating in separate namespaces (`/kr360_arm_controller` for Robot 1, `/robot2/kr360_arm_controller` for Robot 2) to prevent topic conflicts and enable independent trajectory execution. Each bridge node subscribes to its respective OPC UA variables and publishes to robot-specific topics, ensuring the namespace isolation requirement while maintaining system-wide coordination capabilities. The architecture supports coordinated operations through synchronized command sequences while preserving individual robot autonomy, demonstrating scalability beyond the dual-robot configuration to larger multi-robot systems through additional namespace allocation and bridge node instantiation. Performance monitoring extends to both robots through independent logging systems while providing unified dashboard visualization, satisfying the performance optimization objective with minimal resource overhead through efficient namespace management and optimized data structures.

3.3 Implementation and Evaluation Strategy

The implementation and evaluation strategy presented in Section 3.3 provides a systematic approach for validating the hybrid communication framework design. The evidence-based technology selection and phased development methodology ensure comprehensive verification of the architectural objectives established in Sections 3.1 and 3.2. This strategic foundation enables the practical system implementation and performance evaluation detailed in Chapters 4 and 5, demonstrating the framework's effectiveness in addressing industrial multi-robot communication challenges.

3.3.1 Technology Selection Rationale

The implementation of the hybrid communication framework requires careful selection of technologies that address the integration challenges identified in Chapter 2 while meeting the performance requirements established in Section 3.1. Technology selection criteria include industrial compatibility, open-source availability for research purposes, performance capabilities, and alignment with the modular architecture design objectives.

OpenPLC was selected as the PLC runtime environment due to its IEC 61131-3 compliance, ensuring compatibility with industrial programming standards while providing open-source accessibility for research implementation [42]. Unlike exclusive

PLC systems that limit research access, OpenPLC enables direct modification and integration with custom communication protocols, addressing the native PLC compatibility requirement from Section 3.1. The platform supports standard ladder logic programming familiar to industrial practitioners while offering Modbus TCP communication capabilities essential for the hybrid architecture.

OPC UA middleware selection addresses the protocol bridging requirements identified in Section 2.3.3, providing standardized industrial communication with semantic data modeling capabilities [56]. OPC UA's platform independence and vendor neutrality eliminate the integration limitations of proprietary protocols while supporting the real-time performance requirements through efficient binary encoding and subscription mechanisms. The technology's widespread industrial adoption ensures compatibility with existing manufacturing systems, addressing the legacy system support requirement.

ROS 2 selection for robotic middleware addresses the multi-robot coordination limitations identified in Section 2.3.1 while providing the scalability and real-time capabilities required by the architecture design [43]. ROS 2's DDS-based communication infrastructure supports the 99.4% message delivery ratio requirement while enabling namespace isolation essential for multi-robot coordination. The framework's `ros2_control` integration provides standardized robot interfaces compatible with industrial manipulators, ensuring seamless integration with the OPC UA middleware layer.

3.3.2 Development and Testing Methodology

The implementation methodology employs a phased development approach that validates each architectural layer independently before system integration, ensuring systematic verification of the design objectives established in Section 3.1. The methodology addresses the complexity challenges identified in current hybrid solutions through modular component development and comprehensive testing protocols.

Phase 1 implementation focuses on individual component development and validation, beginning with OpenPLC ladder logic programming for robot command processing, followed by Modbus-OPC UA bridge development with protocol translation verification. Each component undergoes unit testing with performance benchmarking to validate less than 200ms response time requirements and 1ms communication cycle specifications. Phase 2 integrates PLC and middleware components through systematic interface testing, validating data type adaptation mechanisms and protocol translation accuracy under various load condition.

Phase 3 implements ROS 2 integration with comprehensive multi-robot testing, validating namespace isolation and parallel communication pathways through coordinated motion sequences. Performance evaluation employs systematic timing analysis measuring end-to-end latency, communication reliability, and synchronization accuracy across single and multi-robot configurations. Testing protocols include scalability assessment through incremental robot addition.

Validation criteria align with the design objectives from Section 3.1 and successful dual-robot coordination without communication bottlenecks. Performance benchmarking compares achieved metrics against the limitations documented in Chapter 2, validating the hybrid architecture's effectiveness in addressing existing solution gaps. The methodology ensures systematic verification of all architectural components while providing comprehensive performance data for the evaluation presented in Chapter 5.

4 Implementation

This chapter presents the implementation of the hybrid industrial communication framework introduced in Chapter 3. The focus is on translating the architectural design into a functioning simulation that integrates ROS 2, OPC UA, and OpenPLC protocols to control one or more simulated KUKA KR360 robotic arms.

The implementation is carried out in two stages: first, a single-robot control pipeline is established and validated in a simulated environment; then, the system is extended to support multi-robot coordination by replicating and isolating communication layers. Each component PLC logic, OPC UA middleware, ROS2 actuation, and visualization—is modularly implemented using open-source tools and simulated interfaces.

Section 4.1 introduces the development environment and toolchain, while Section 4.2 details the implementation of the single-robot workspace, including the OPC UA server, the Modbus bridge, and ROS 2 integration logic. Section 4.3 describes the scalable extension to a multi-robot environment, focusing on namespace isolation and mirrored pipelines. Section 4.4 concludes with the system integration and launch process that brings together all components into a fully functional simulation.

4.1 Development Environment and Tools

The proposed system was implemented and validated in a simulated environment using open-source tools. The goal was to prototype a flexible control pipeline capable of coordinating robotic tasks using standardized communication layers across simulated PLC and robot components. No physical hardware was used during implementation.

The development environment was based on Ubuntu 22.04 LTS, chosen for its stability, long-term support, and ROS 2 compatibility. The ROS 2 Humble Hawksbill distribution was selected for its enhanced real-time features, DDS-based middleware, and long-term maintenance status. Source code was developed using Python 3 in the Visual Studio Code (VS Code) IDE, and the system was built using Colcon, the standard ROS 2 build tool.

Simulation of industrial control logic was achieved using OpenPLC, which supports IEC 61131-3 ladder logic and exposes control signals over Modbus TCP. These signals were periodically read using pyModbusTCP and mapped to corresponding OPC UA nodes created using the python-opcua library. ROS 2 bridge nodes, implemented in Python using rclpy, subscribed to these OPC UA nodes and converted joint data into ROS 2 JointTrajectory messages.

Robot behavior was simulated using the `ros2_control` framework with fake hardware interfaces, and joint positions were visualized in RViz. Additional monitoring scripts were developed using `pandas`, `datetime`, and `matplotlib` to track control latency and

execution timelines. These scripts generated timestamped logs used later in Chapter 5 for performance analysis.

4.1.1 Software Stack Overview

Each component of the system was selected to ensure platform independence, open-source licensing, and modular deployment. The stack includes the following technologies:

- **Operating System:** Ubuntu 22.04 LTS — widely adopted in robotics research.
- **OpenPLC + Modbus TCP:** Implements IEC 61131-3 ladder logic, exports digital outputs via Modbus protocol.
- **pyModbusTCP:** A Python client used to read coil values from the OpenPLC runtime and feed them into OPC UA variables.
- **python-opcua:** Used to implement a custom OPC UA server. This exposes the control commands (as Boolean flags) and joint targets (as numerical arrays) for external systems.
- **ROS 2 (rclpy):** The Python ROS 2 API was used to create publishers that convert OPC UA joint values into JointTrajectory messages for the robot controller.
- **ros2_control + RViz:** Handles the actuation and visual feedback of the simulated KR360 robot.
- **Performance Tools:** Python libraries such as pandas, datetime, and matplotlib were used to log and visualize latency, execution times, and trajectory update intervals.

Table 4.1: Software Stack Overview

Component	Role in the System	Library / Tool Used
OS / IDE	Development and testing environment	Ubuntu 22.04, VS Code
PLC Logic Simulation	Generates Modbus control signals	OpenPLC
Modbus Client	Reads Modbus coils from PLC	pyModbusTCP
OPC UA Server	Middleware layer exposing robot commands	python-opcua
ROS 2 Control Interface	Publishes robot trajectories from OPC UA input	rclpy
Robot Simulation & Visualization	Simulates motion and displays joint feedback	ros2_control, RViz
Performance Monitoring	Logs system latency and events	pandas, matplotlib

4.1.2 Workspace Structure for Single and Multi-Robot Systems

This research utilizes two separate ROS 2 workspaces to enable modular development and improve scalability: one for single KR360 robot simulation and another for dual-robot deployment. Each workstation exhibits a methodical arrangement, with a distinct separation of essential elements such as robot specifications, middleware interfaces, performance monitoring tools, and control scripts. Section 4.1.2.1 delineates the architectural framework for the single robot workspace, whereas Section 4.1.2.2 analyzes the structural implementation for multi-robot simulation scenarios.

4.1.2.1 Single Robot Workspace Structure

The simulation environment for the KR360 robot has been established within a specific ROS 2 workspace named `kuka_ws_2`. This workspace includes all necessary elements for industrial robot connection via Modbus, OPC UA, and ROS 2 protocols. The directory structure enforces a clear distinction among hardware descriptions, communication middleware, and performance monitoring modules.

The workspace architecture employs a modular design approach to provide clarity and maintainability, as depicted in Figure 4.1. The `kr360_description` package includes the Unified Robot Description Format (URDF) model, mesh files, and control configuration parameters essential for defining the KR360 robot in the simulation environment. The `Kuka_kr360_middleware` package facilitates middleware operations by linking OPC UA variables with ROS 2 topics. Performance monitoring and runtime assessment are conducted via Python scripts in the `performance_testing` package. The `scripts` package offers independent utilities for testing OPC UA servers, translating Modbus inputs, and transmitting commands directly to robots.

The ROS 2 workspace employs Python-based launch files (`.launch.py`) found in each package's `/launch` directory to facilitate effective launch sequence management and uphold clear runtime separation. These launch files automate the initialization of nodes, encompassing controller instantiation, ROS 2 bridge configuration, and OPC UA server deployment. This single-robot workspace architecture allows for separate middleware assessment, supports systematic debugging processes, and encourages modular simulation development.

```

kuka_ws_2/
├── src/
│   ├── kr360_ros2/
│   │   ├── kr360_bringup/
│   │   └── kr360_description/
│   ├── kuka_kr360_middleware/
│   └── performance_testing/
├── scripts/
│   ├── direct_robot_controller.py
│   ├── manual_opcua_server_kr360.py
│   └── plc_opcua_bridge.py
├── install/
├── build/
└── log/

```

Figure 4.1: Single Robot Ros 2 Workspace Structure

4.1.2.2 Multi-Robot Workspace Structure

The dual KR360 robots' simulation environment is created in a different ROS 2 workspace called `multiple_kuka_ws_2`. This structure is similar to the modular principle of the single robot workspace, but it has been expanded to allow two robots to work at the same time. Every component is either a duplicate of a different component or keeps separate from the others to avoid conflicts. This is achieved through launch files and middleware logic.

Figure 4.2 shows how this workspace is set up to handle the simulation of two KR360 robots with separate namespaces and shared middleware. The `kr360_description` folder has URDFs and configuration files for both robots (Robot 1 and Robot 2). These files include the mesh data and controller settings for each robot. The `kr360_bringup` package has launch files for each robot (`kr360_robot1.launch.py` and `kr360_robot2.launch.py`) and a combined RViz visualization (`multi_kr360_rviz.launch.py`). A shared `kuka_kr360_middleware` package takes care of middleware tasks like OPC UA bridging. There are Python programs in the `scripts` folder that act like the OPC UA server and connect Modbus signals for both robots. There are also separate ROS bridge nodes for each robot. The `performance_testing` folder has scripts and data files that keep track of the latency, timing, and synchronization between the two robots while they are running at the same time.

```

multiple_kuka_ws_2/
├── src/
│   ├── kr360_ros2/
│   │   ├── kr360_bringup/
│   │   └── kr360_description/
│   └── kuka_kr360_middleware/
├── scripts/
│   ├── multi_robot_opcua_server.py
│   ├── multi_robot_plc_bridge.py
│   ├── multi_robot_controller.py
│   ├── robot1_ros_bridge.py
│   └── robot2_ros_bridge.py
├── performance_testing/
│   ├── multi_robot_performance_monitor.py
│   ├── generate_multi_robot_graphs.py
│   └── generate_multi_robot_additional_graphs.py
└── install/, build/, log/

```

Figure 4.2: Multi-Robot Ros 2 Workspace Structure

Each robot has its own ROS 2 namespace so that topics do not clash. Robot 1, for instance, uses the default namespace `/kr360_arm_controller`, while Robot 2 uses a prefixed namespace like `/robot2/kr360_arm_controller`. This way of naming things keeps topics, parameters, and controller states separate for each robot, so they can all work at the same time in the same ROS2 environment. The system's scalable design is demonstrated by this multi-robot workspace, which allows for mirrored launch control, synchronized execution, and modular simulation of both robots via shared interfaces and well-separated namespaces.

4.2 Single Robot System Implementation

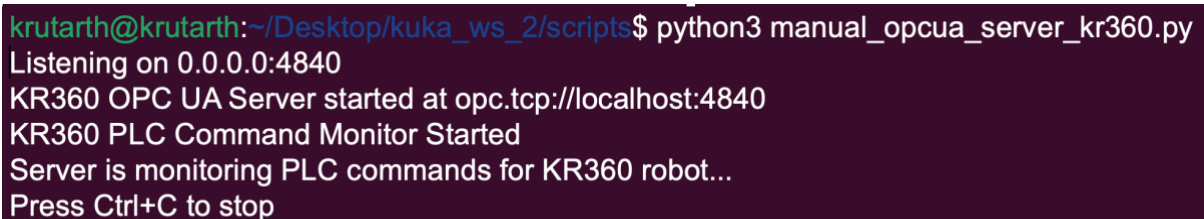
This section presents the implementation of a single KR360 robot system within a simulated industrial environment. The objective is to construct a hybrid communication pipeline in which a software-based PLC issues control commands using Modbus TCP, which are then translated to OPC UA variables and interpreted by ROS2 nodes. The full pipeline includes: a custom OPC UA server, a Modbus-to-OPC UA bridge, a ROS2 joint trajectory publisher, a performance logger, and an AI-based anomaly detection module.

The entire system operates in real-time using simulated interfaces and modular Python scripts, allowing the KR360 robot model to respond to control commands such as HOME, PICK, and PLACE. Visualization and trajectory execution are managed using RViz and the `ros2_control` framework with fake hardware. The implementation is organized into ROS2 packages, with launch files managing initialization sequences. Each component is described in the following sections.

4.2.1 OPC UA Server Implementation

The OPC UA server acts as the central middleware layer between the PLC logic and the ROS2 robot controller. Its main function is to expose robot command variables (HOME, PICK, PLACE) in a standardized format that can be accessed by both PLC and ROS2 components. This modular design enables real-time data sharing, platform independence, and flexible integration with future systems. To implement this, the `python-opcua` library was used to create a customized server named `manual_opcua_server_kr360.py`. This script initializes the server at the endpoint `opc.tcp://0.0.0.0:4840/`, registers a custom namespace (`KR360_Command_Server`), and adds Boolean control nodes. Each command variable is marked writable so that external clients—such as a Modbus bridge—can write control signals to them in real time.

Figure 4.3 is terminal output of Opcua server. It shows that OPC UA server running, and it is ready to monitor command for KR360 robot. It runs on port 4840. Opcua server monitors commands from plc and robot.



```
krutarth@krutarth:~/Desktop/kuka_ws_2/scripts$ python3 manual_opcua_server_kr360.py
Listening on 0.0.0.0:4840
KR360 OPC UA Server started at opc.tcp://localhost:4840
KR360 PLC Command Monitor Started
Server is monitoring PLC commands for KR360 robot...
Press Ctrl+C to stop
```

Figure 4.3: Terminal Output of Manual_Opcua_Server_Kr360.Py

This following flowchart represents the setup process for an OPC UA server that controls a robot. The process begins by creating a server instance and configuring it to listen on port 4840. After registering a namespace called "`KR360_Command_Server`", the code retrieves the objects node and adds a KR360 object to it. Three Boolean variables are then created under this object: HOME, PICK, and PLACE, each with a default value of false. These commands represent predefined robot positions with specific joint values already set - HOME moves the robot to its home position, PICK positions it for object pickup operations, and PLACE positions it for object placement operations. Finally, all three variables are configured to be writable, allowing external clients to send commands to control the robot's pre-programmed movements through the OPC UA protocol.



Figure 4.4: Code Flow Chart of Script `Manual_Opcua_Server_Kr360.Py`

4.2.2 Modbus-to-OPC UA Bridge

The second major component in the single-robot system is the Modbus-to-OPC UA bridge. This module acts as the translation layer between industrial control logic, simulated through OpenPLC, and the OPC UA server. Implemented as an independent Python process using the `pyModbusTCP` and `python-opcua` libraries, the bridge ensures platform decoupling and modular communication. At runtime, the bridge continuously polls digital coil outputs from the OpenPLC runtime. These coils are mapped to robot motion commands and updated periodically. When a specific coil becomes active (set to True), the bridge updates the corresponding OPC UA variable. This event-based model ensures real-time responsiveness while minimizing redundant data transmission.

In this configuration:

- %QX0.0 maps to Modbus coil address 1024 (HOME)
- %QX0.1 maps to Modbus coil address 1025 (PICK)
- %QX0.2 maps to Modbus coil address 1026 (PLACE)

The bridge continuously checks these coil states and writes updated values to OPC UA nodes with IDs ns=2; i=4, ns=2; i=5, and ns=2; i=6, respectively. These node IDs correspond to the KR360_Command_Server namespace registered earlier in 4.2.1. Figure 4.5 OpenPLC Web Interface showing the ladder logic configuration and active Modbus coil addresses used for issuing robot motion commands. When a coil (e.g., 1024 for HOME) is triggered, the bridge script detects the signal and relays the corresponding position to the OPC UA variable.

Monitoring

Refresh Rate (ms):

Point Name	Type	Location	Write	Value
HOME_BUTTON	BOOL	%IX0.0		☐ FALSE
PICK_BUTTON	BOOL	%IX0.1		☐ FALSE
PLACE_BUTTON	BOOL	%IX0.2		☐ FALSE
HOME_ACTIVE	BOOL	%QX0.0	<input checked="" type="button" value="True"/> False	☐ FALSE
PICK_ACTIVE	BOOL	%QX0.1	<input checked="" type="button" value="True"/> False	☐ FALSE
PLACE_ACTIVE	BOOL	%QX0.2	<input checked="" type="button" value="True"/> False	☐ FALSE

Figure 4.5: OpenPLC Web Interface for Robot Motion Commands

Figure 4.6 flowchart shows the runtime behavior of the bridge script, which commences with initialization and enters a continuous polling loop. In each repetition, it verifies the status of the HOME, PICK, and PLACE Modbus coils. Upon activation of a coil, the relevant joint data is recorded in the corresponding OPC UA variable, after which the coil is reset to avoid unnecessary updates. A brief delay of 0.1 seconds is incorporated at the conclusion of each loop cycle to mitigate CPU demand and prevent excessive polling. This polling method guarantees elevated responsiveness and operational efficiency. The bridge's modular design facilitates future growth by incorporating more coils and establishing new control variables as required.

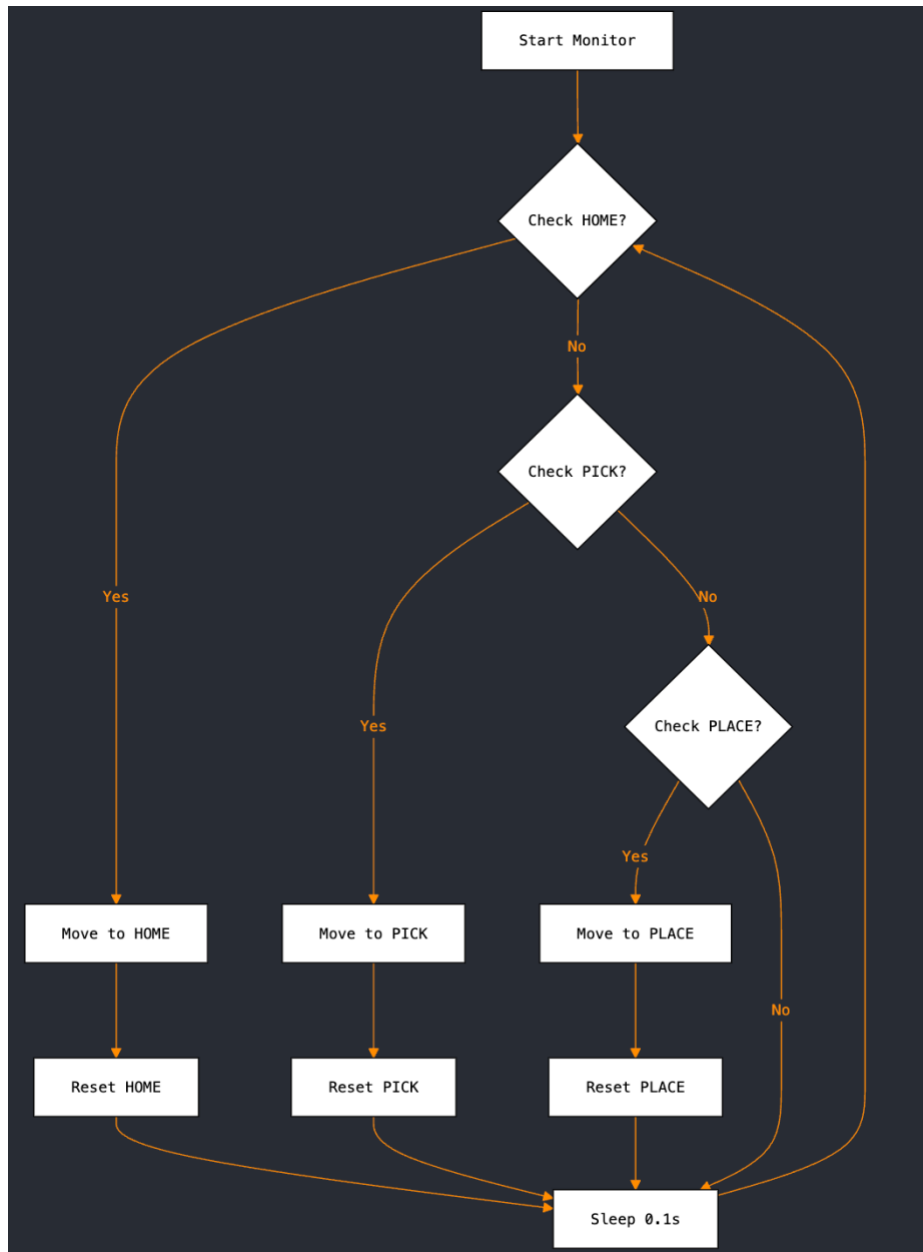


Figure 4.6: Control Flow of Modbus-To-OPC UA Bridge Logic

The subsequent section discusses how ROS2 acquires the updated OPC UA variables and converts them into motion commands for the robot controller. These revised OPC UA variables are converted into motion commands for the robot controller.

4.2.3 ROS2 Joint Publisher Node

The ROS2 joint publisher node is the final stage in the control pipeline of the single-robot system. It receives target joint positions from the OPC UA server and publishes them as trajectory commands to the robot controller within the ROS2 environment. This ensures seamless coordination between the industrial control logic and robotic execution. The node is implemented using the rclpy Python client library, enabling flexible construction of ROS2 publishers and integration with the rest of the framework.

It continuously monitors the OPC UA variable holding joint angle data. Upon detecting an updated command, the node generates a JointTrajectory message containing six joint positions and publishes it to the topic `/kr360_arm_controller/joint_trajectory`.

To prevent unnecessary message traffic, the node includes logic to compare incoming values with the previously executed command. Only new or changed commands trigger motion updates. This minimizes latency and maintains system efficiency. Figure 4.7: RViz window showing the KR360 R2830 robot model in its initial static pose. The robot is visualized using its URDF and mesh files. Motion is executed when the ROS2 node publishes a trajectory command based on OPC UA input.

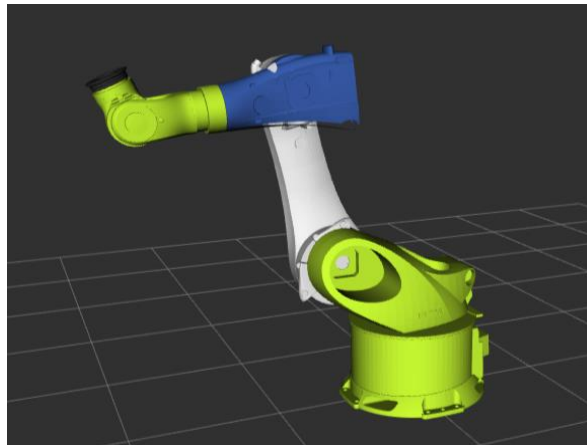


Figure 4.7: RViz Window With KR360 R2830 Robot Model With Static Pose

Figure 4.8: ROS2 communication graph (rqt_graph) displaying live interactions between core ROS2 nodes. The `/kr360_opcua_bridge` node publishes data to `/kr360_arm_controller/joint_trajectory`, which is why robot will move in rviz. Supporting nodes such as `/joint_state_broadcaster` and `/robot_state_publisher` handle feedback and visualization.

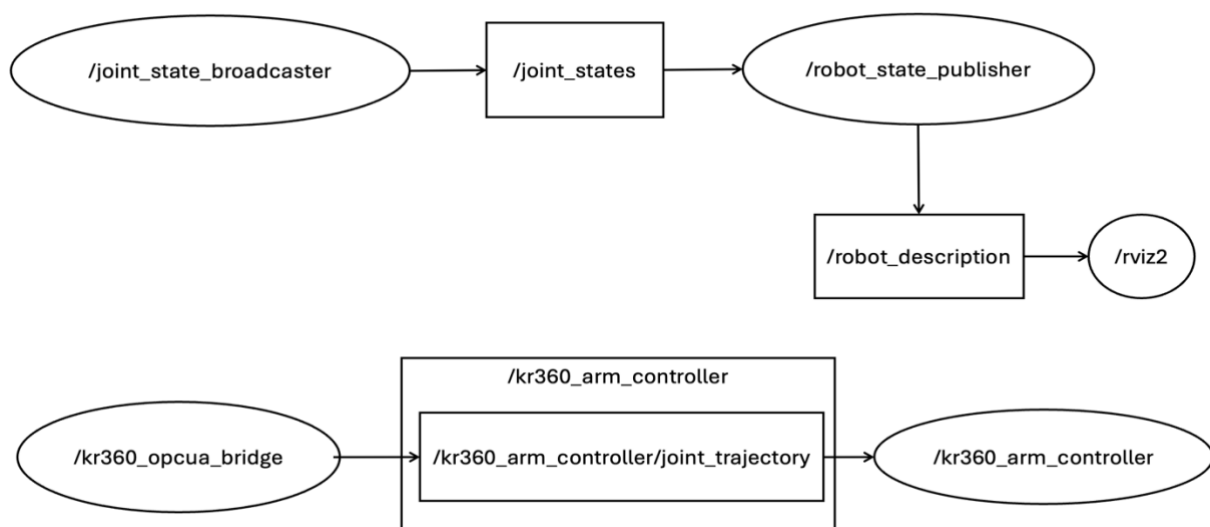


Figure 4.8: Rqt_Graph Visualizing the Live ROS2 Node Communication

Figure 4.9 Web-based control simulator used to issue commands such as HOME, PICK, and PLACE to the robot. The GUI updates the OPC UA variables accordingly, which are then read by the ROS2 node to generate motion.

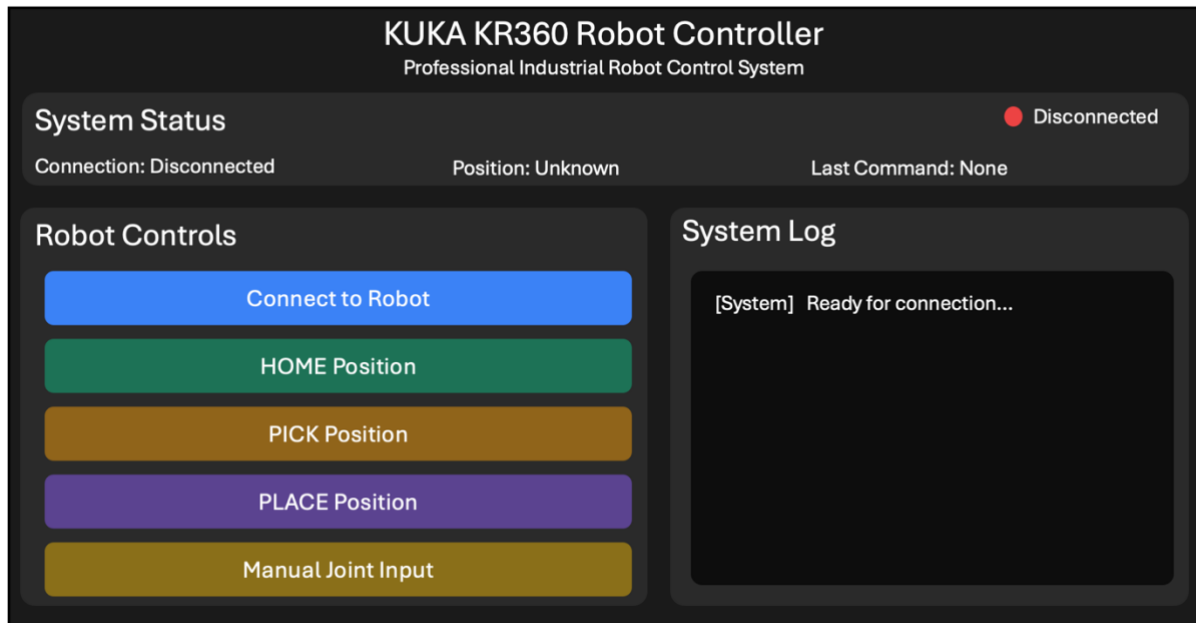


Figure 4.9: Web Interface View of The KR360 Robot Control Simulator

This bridge completes the closed-loop cycle between PLC logic and simulated robot actuation. The commands HOME sent from PLC and that commands receive to the OPC UA bridge and again sent to ROS2, so it closed whole the cycle. The modularity and independence of the publisher node allow it to be reused or expanded in multi-robot configurations, with minimal changes limited to namespaces or topic remapping.

4.2.4 Performance Logging Module

A specialized performance logging module was established to quantitatively evaluate the system's responsiveness and execution precision. This module records timestamped data at essential points in the control pipeline and archives the results in organized CSV log files. Chapter 5 subsequently analyzes the gathered data to assess critical parameters including command latency, signal propagation delay, and system responsiveness. The logging tool was created as a standalone Python script to guarantee its operation independent of the ROS2 and OPC UA processes. This distinction avoids interference with system performance or the introduction of measurement errors.

Upon execution, the script checks for the existence of a logs/ directory. If absent, it generates one. It subsequently creates a uniquely designated CSV file utilizing the current date and time to archive log entries. Each item documents the exact UTC

timestamp of a relevant system event (e.g., command received, trajectory published) accompanied by a descriptive label. A typical log entry is produced as follows:

```
timestamp = datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

action = "ROS2 trajectory published"

writer.writerow([timestamp, action])
```

This high-resolution logging guarantees that all critical control activities are traceable with millisecond accuracy. The script incorporates exception handling and termination logic to appropriately end the file in the event of user interruption, hence preventing data loss.

The performance logger operates concurrently with the primary control system without modifying or postponing any pipeline procedures. Its passive characteristic facilitates precise evaluation of genuine system performance under standard operating settings. The logging functionality is crucial for verifying the proposed hybrid communication system. This establishes the basis for the quantitative assessment of the system's timing behavior, facilitating a systematic and objective performance study in Chapter 5.

4.2.5 AI-Based Anomaly Detection Module

To enhance system reliability and introduce intelligent monitoring, an AI-based anomaly detection module was integrated into the single KR360 robot simulation. This module operates as a passive observer, analyzing incoming joint command vectors in real time to detect patterns that deviate from expected behavior. By classifying commands as either "normal" or "anomalous," it adds a safety layer above the OPC UA–ROS2 motion pipeline. The anomaly detection logic was implemented using a supervised learning approach. A labeled dataset of joint angles was collected using the `kr360_data_logger.py` script. Each row in the dataset was tagged as either normal (0) or anomalous (1). The training process was carried out using a Random Forest classifier from the `scikit-learn` library. The resulting model was saved as a serialized `.pkl` file (`kr360_realistic_model.pkl`) and later used in real-time inference.

The script `kr360_realtime_detector.py` loads the trained model and continuously reads joint command values from the system. These six-dimensional vectors are reshaped and passed into the classifier, which returns a binary label. The script runs in a non-blocking loop parallel to the control system and logs predictions to the terminal. A sample detection routine is summarized below:

```
data = np.array(joint_input).reshape(1, -1)

prediction = model.predict(data)[0]

label = "ANOMALY DETECTED" if prediction == 1 else "NORMAL"

print(f"[{label}] | joints: {joint_input}")
```

To better illustrate how the module fits within the system architecture, an integration diagram is provided in Figure 4.10. This diagram shows the detection module operating in parallel to the control flow. It reads joint values published via OPC UA, processes them through the machine learning model, and outputs a classification label. Importantly, the module does not block or interfere with ROS2 execution. It is designed for monitoring and alerting purposes.

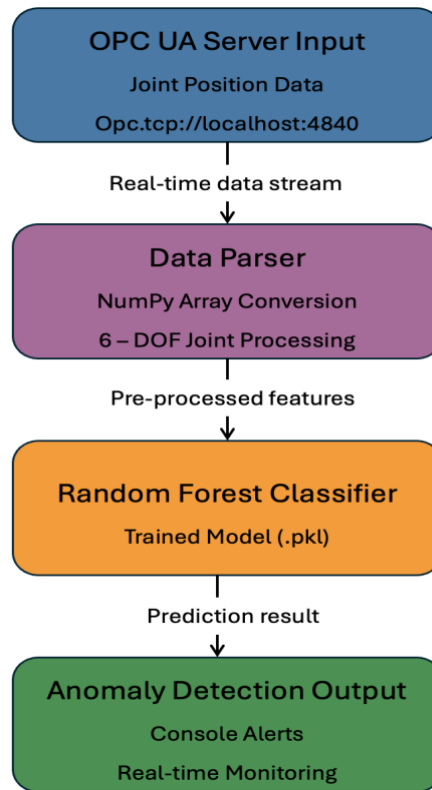


Figure 4.10: Architecture Of AI-Based Anomaly Detection for Single KR360 Robot

While this module was implemented only in the single-robot workspace, its architecture is modular and can be adapted to multi-robot systems in future work by assigning robot-specific detection threads and classifiers. Quantitative results, including model accuracy, confusion matrix, and classification performance, are presented in Chapter 5. These validate the effectiveness of anomaly detection in identifying outlier commands without disrupting real-time control.

4.2.6 Discussion

The single KR360 robot system set up a control pipeline that is both flexible and useful. It uses industrial protocols, ROS2 actuation, and AI-based monitoring. The OPC UA server, the Modbus bridge, and the ROS2 publisher were all made into distinct modules. This made it easier to see how the data moved and to link all the parts together. It was easy to change and keep the system running when OPC UA was the main way for the PLC simulation and ROS2 to talk to each other.

Adding extra modules, including the performance logger and the anomaly detection script, made the simulation's monitoring and safety features better. The AI module made it possible for the robot to effectively sort movement commands, so there was no need for input during any phase of its operation. The web-based simulator also made it easier to operate. These technologies work together to create a complete and adaptable framework for safely and traceably replicating how industrial robots perform. The next section improves this design so that two robots can work together. It does this by introducing new logic for control and management of namespaces.

4.3 Multi-Robot System Implementation

The single-robot KR360 simulation architecture was expanded to make the multi-robot system more robust and scalable. The main communication pathway, which includes Modbus communication, OPC UA middleware, and ROS 2 messaging, remained but changed so that two KR360 robotic arms could work at the same time. Each robot was given its own namespace to keep command conflicts from occurring and to keep ROS topics, service calls, and middleware mappings separate. This design allows both robots be controlled at the same time, but separately, in a single simulation environment.

To allow for dual-robot operation, the OPC UA server was changed to create separate OPC UA nodes so that the status of both robots could be shared and monitored. In the same way, the Modbus-to-OPC UA bridge was changed so that the OPC UA address space of each robot was linked to its own OpenPLC signals. It became accessible by dedicated ROS 2 nodes to send trajectory commands to each robot based on its joint state data. The performance monitoring modules were also updated so that it can now measure execution time, latency, and the operational status of each robot separately. The launch files were changed to make sure that all system parts correctly registered their own namespaces. This made it possible for all parts of the pipeline to work together without any issues.

The AI-based anomaly detection module was not included to the two-robot setup, but the single-robot setup can still be used for this purpose. Each robot will continue to use its own OPC UA channels for communication as well as maintain its own joint state logs. The current detection and classification model could be added to either robot, not both, without making major changes to the way they are built. In the final section of this thesis, it explains about how adding this kind of anomaly detection could be added to the system in the future.

4.3.1 Multi-Robot OPC UA Server

The single KR360 robot simulation architecture served as the basis for the multi-robot system, thereby strengthening it. The OPC UA server used in the single-robot setup was modified to support a dual-robot configuration, allowing two industrial robots to be

simulated simultaneously. This meant developing two different OPC UA object nodes, one for each KR360 robot. The nodes are set up in the same manner as the single-robot architecture, which maintains modularity and consistency across implementations.

There is an object for each robot in the OPC UA namespace, like Robot1_KR360 and Robot2_KR360. These objects have writable variables for joint positions and PLC control commands. Some of these variables include JointPositions and boolean flags like MoveHome, MovePick, and MovePlace. Below is a simpler version of the code that was used to create one robot object:

```
self.robot1_obj = self.server.nodes.objects.add_object(self.idx,
"Robot1_KR360")

self.robot1_joints = self.robot1_obj.add_variable(self.idx,
"JointPositions", [0.0]*6)

self.robot1_joints.set_writable()

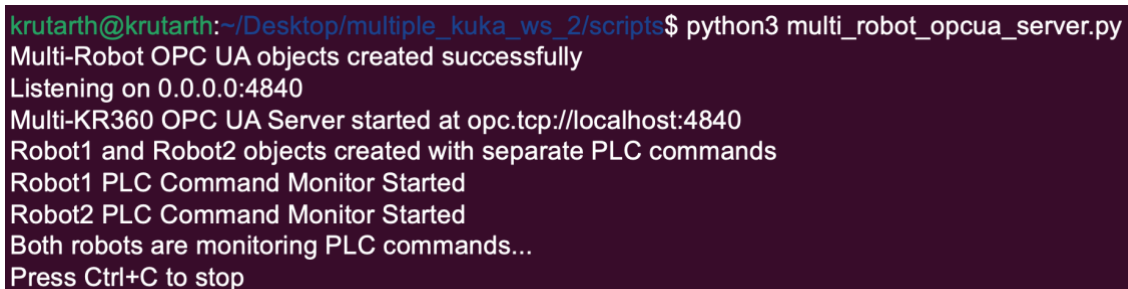
self.robot1_plc = self.robot1_obj.add_object(self.idx, "PLCCommands")

self.robot1_home_cmd = self.robot1_plc.add_variable(self.idx, "MoveHome",
False)

self.robot1_home_cmd.set_writable()
```

This logic applies to Robot 2; however, the variable names have been modified to fit. The result is that the OPC UA server provides each robot with its control interface, allowing two PLC sources or Modbus bridges to operate simultaneously without interfering with each other. The implementation also includes a built-in monitoring system that utilizes separate threads to monitor PLC commands and convert them into joint changes continuously.

The terminal screenshot in Figure 4.11 indicates that the robot objects were made successfully and that the server is running on `opc.tcp://localhost:4840`. The log messages indicate that the JointPositions and control commands for both robots were set up successfully and made readable.



```
krutarth@krutarth:~/Desktop/multiple_kuka_ws_2/scripts$ python3 multi_robot_opcua_server.py
Multi-Robot OPC UA objects created successfully
Listening on 0.0.0.0:4840
Multi-KR360 OPC UA Server started at opc.tcp://localhost:4840
Robot1 and Robot2 objects created with separate PLC commands
Robot1 PLC Command Monitor Started
Robot2 PLC Command Monitor Started
Both robots are monitoring PLC commands...
Press Ctrl+C to stop
```

Figure 4.11: Terminal Output For Initialization of Dual Robot OPC UA Objects

The design of this OPC UA server is based on the single-robot structure, with only a small duplication that works well. It will be easier to add a third or fourth robot to the system in the future if each robot's joint data and command inputs are stored in separate objects. For these kinds of extensions, the only adjustments that would need to be made are:

- Adding additional object blocks, such Robot3_KR360
- Copying the structure of the joint variable and PLC command
- Updating the monitoring threads or putting them together in a loop that can handle numerous robots at once
- This modular server design makes sure that the OPC UA interface can expand so it may be used with higher simulated or real-world robotic cells.

4.3.2 Dual Modbus Bridge for Robot Control

It included support for two robots to the original Modbus-to-OPC UA bridge, so both robots received different motion commands from OpenPLC. This bridge receives values for the coils from a Modbus TCP server and changes the OPC UA nodes for each robot separately. This connection is created feasible by the script `multi_robot_plc_bridge.py`, which discusses between the PLC layer and the middleware in real time.

The single-robot system used three Modbus coils to send the MoveHome, MovePick, and MovePlace motion commands. These coils were duplicated and remapped so that it could communicate with both robots in the dual-robot system. Coils 1024, 1025, and 1026 were distributed to Robot 1, and coils 1027, 1028, and 1029 were distributed to Robot 2. The system polls all of these coils all the time. When a coil is turned on, the OPC UA server's variable for that coil is changed to reflect that. This makes sure that each robot receives the right motion command through the coil mapping that is specific to it. The Modbus client reads the coil states and changes the OPC UA variables for Robot 1 as shown in the next section of code from the Modbus-to-OPC UA bridge script:

```
coil_home = client.read_coils(1024, 1).bits[0]
coil_pick = client.read_coils(1025, 1).bits[0]
coil_place = client.read_coils(1026, 1).bits[0]

ua_robot1_home.set_value(coil_home)
ua_robot1_pick.set_value(coil_pick)
ua_robot1_place.set_value(coil_place)
```

Using different coil addresses, the same logic applies to Robot 2. The separation of control signals ensures that each robot receives instructions without any issues. This parallel bridging method allows both robotic arms to perform their tasks simultaneously, keeping them separate from each other.

The bridge script keeps track of its status and shows which coils each robot is looking at. Figure 4.12 shows that the terminal output validates a successful Modbus connection to OpenPLC and the startup of all six coils. It also shows that polling is ongoing and that both Robot1_KR360 and Robot2_KR360 OPC UA objects are being updated.

```
krutarth@krutarth:~/Desktop/multiple_kuka_ws_2/scripts$ python3 multi_robot_plc_bridge.py
Connected to OpenPLC via Modbus
Connected to Multi-KR360 OPC UA Server
OPC UA nodes mapped successfully for both robots
Multi-Robot PLC to OPC UA Bridge Ready!
=====
Starting Multi-Robot PLC to OPC UA bridge...
Monitoring OpenPLC outputs for Robot1 and Robot2
Robot1: Coils 1024-1026 (QX0.0 - QX0.2)
Robot2: Coils 1027-1029 (QX0.3 - QX0.5)
Press Ctrl+C to stop
```

Figure 4.12: Terminal Output from Dual Modbus-OPC UA Bridge For Both Robots

This architecture for the bridge permits the system develop further. Adding a third robot would simply require giving it the next set of Modbus coils (for example, 1030, 1031, 1032), making a new OPC UA object (Robot3_KR360), and copying the signal translation logic into the same script or a separate thread. This modular signal mapping makes things more flexible and will help with future implementations that need to simulate how multiple robots work together.

4.3.3 Multi-Robot Controller Node

To control each robot separately in a dual-robot simulation system, you need unique ROS 2 bridge nodes for each robot. These bridge nodes take joint values from the OPC UA server and turn them into ROS 2 trajectory commands. The standard JointTrajectory message format is used to send the trajectory commands to robot-specific ROS 2 topics. Using ROS 2 namespaces, this setup lets Robot 1 and Robot 2 work at the same time without interfering with each other's data. In ROS 2, a namespace is a label that comes before any topic and node names. For instance, the trajectory `/robot2/kr360_arm_controller/joint_trajectory` is only available to Robot 2 because it is in the `/robot2` namespace. This separation is necessary in environments with more than one robot so that the control systems don't get in harm's method.

Two different architectural methods were looked at for placing together the ROS 2 bridges. There are two independent Python nodes in the modular approach: `robot1_ros_bridge.py` and `robot2_ros_bridge.py`. Each node operates in its own ROS

2 namespace and gets its information from its own OPC UA object, which is either Robot1_KR360 or Robot2_KR360. The combined solution employs one script, `opcua_kr360_bridge.py`, that has logic for both robots and runs them separately with two ROS 2 timers. A ROS 2 timer lets a node set up a certain function to run at a set time interval and then run it repeatedly. One timer in the combined script handles Robot 1's commands, and the other timer handles Robot 2's commands. This method drops the number of nodes that are active, but it makes it challenging to expand and fix errors. The modular method, which has one bridge node for each robot, is easier to manage and add to. The code below comes from the file `robot2_ros_bridge.py`. It shows how Robot 2's bridge node gets joint values from the OPC UA server and makes a `JointTrajectory` message.

```
traj = JointTrajectory()

traj.header.stamp = self.get_clock().now().to_msg()

traj.joint_names = ['robot2_joint_a1', 'robot2_joint_a2',
'robot2_joint_a3', 'robot2_joint_a4', 'robot2_joint_a5', 'robot2_joint_a6']

point = JointTrajectoryPoint()

point.positions = joint_values

point.velocities = [0.0] * 6

point.time_from_start.sec = 2

traj.points.append(point)

self.publisher_.publish(traj)
```

This code makes a `JointTrajectory` message. The list `joint_names` specifies the names of Robot 2's six joints. The point object sets the target joint positions, the speed to zero, and the time it takes to move to 2 seconds. The finished message is then sent to the subject `/robot2/kr360_arm_controller/joint_trajectory`. It works the same way with Robot 1's `robot1_ros_bridge.py` node, but the namespace and joint names are not started with `/robot2`.

The third option, `opcua_kr360_bridge.py`, combines both publishing pipelines into a single script. It has two publishers, two OPC UA readers, and two timers, one for each robot. It works fine, but it's difficult to keep up with when the system grows to three or more robots. For robustness and modularity, it is best to use separate bridge nodes in real-world systems.

Figure 4.13 confirms that the robot model is loaded correctly in the simulation. The robot is visualized using its URDF and mesh configuration from the `kr360_description` package. This visualization is generated when the launch file `multi_kr360_rviz.launch.py` is executed using the command “`ros2 launch kr360_bringup multi_kr360_rviz.launch.py`”. After this launch, the 2 robots appear in its initial static pose in RViz. Motion is not triggered at this point; instead, the robot waits

for a control command such as HOME, PICK, or PLACE. These commands are now sent using a custom web-based robot control simulator as per figure 4.15, which updates the OPC UA variable. Once a change is detected, the ROS2 bridge node publishes the corresponding trajectory message to move the robot.

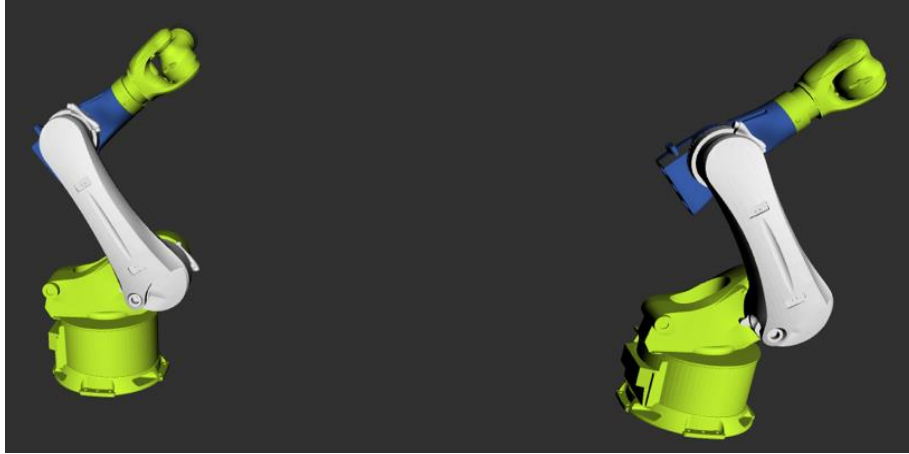


Figure 4.13: Rviz Window with Two KR360 R2830 Robot Model With Static Pose

Figure 4.14 shows the live data flow between nodes in the ROS 2 environment. Each robot has its own `kr360_opcua_bridge` node, which publishes joint trajectory messages to its respective controller. These publishers are isolated under `/` and `/robot2` namespaces. The presence of `controller_manager`, `joint_state_broadcaster`, and `robot_state_publisher` confirms that both robots are active and being visualized independently.

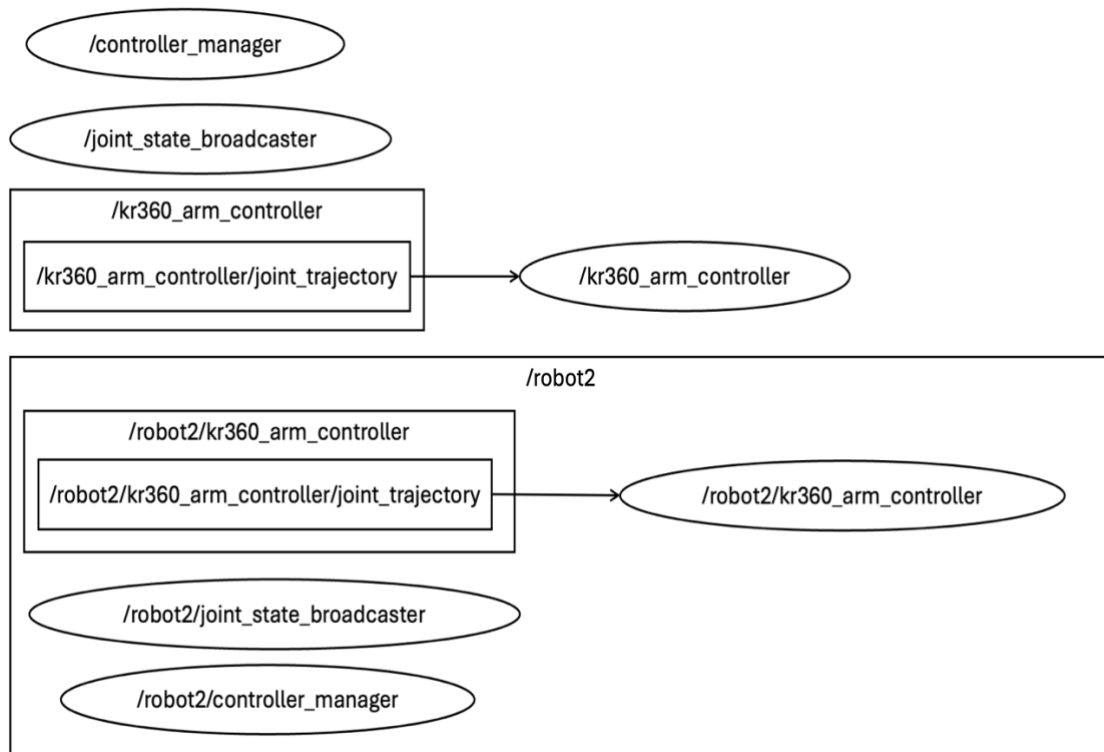


Figure 4.14: Rqt_graph Showing ROS 2 Communication Nodes for Both Robots

Figure 4.15 illustrates the control GUI that allows it transmit commands to both robots. There are different buttons on the interface for Robot 1 and Robot 2. Each button sends a distinct command for movement, like HOME, PICK, or PLACE. When click on the buttons, it updates the right OPC UA variable. The bridge node sees this and sends it to the controller as a trajectory instruction.

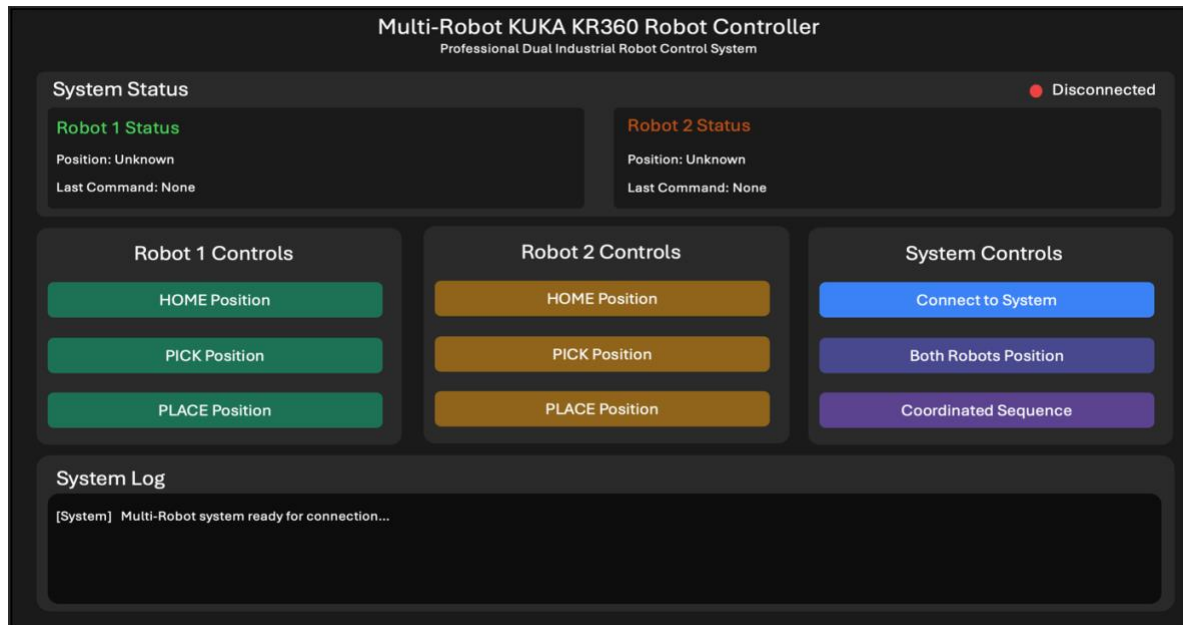


Figure 4.15: Dual-Robot Web-Based Simulator Interface

Figure 4.16 provides the backend log of the simulator's Flask server. It shows the system listening for input on port 5000 and confirms that commands were received and passed to the bridge. This log output helps verify that the GUI is correctly linked to the OPC UA middleware and ROS 2 control layer.

```
krutarth@krutarth:~/Desktop/multiple_kuka_ws_2/scripts/robot_controller$ python3 app.py
Starting Multi-Robot KR360 Controller Web Interface...
Open your browser to: http://localhost:5000
Press Ctrl+C to stop
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server for simulation testing. Do not use it in a production
deployment
Press CTRL+C to quit
```

Figure 4.16: Terminal Log of Simulator Showing Command Execution of GUI

This arrangement for the controller node is made to be scalable. The developer would clone one of the current bridge files and give it the name `robot3_ros_bridge.py` to add a third robot. The OPC UA path and ROS 2 namespace would be changed to `/robot3/` in the new script. A new launch file would be made to start the node in the new namespace. This strategy keeps each robot separate while they work in the same ROS 2 environment.

4.3.4 Multiple Robots Performance Monitoring System

To evaluate the way the dual-robot system responds to control inputs, a performance monitoring module was created. This module, which is in the file `multi_robot_performance_monitor.py`, keeps separate records of Robot 1 and Robot 2's key timing metrics. Its main goal is to maintain track of and compare both robots' execution latency without messing with their control pipelines. The module is built as a stand-alone Python program that does not require either ROS 2 or OPC UA. This design makes sure that performance logging works on its own, so it does not impact how the system works in real time. Standard Python libraries like `datetime`, `csv`, and `os` are used for data logging. This allows the module record operational events with millisecond accuracy.

When the script runs, it first checks to see if there is a `logs/` directory in the workspace. If the directory does not exist, it will be made automatically. The module then makes two CSV log files, one for each robot, and names them with the date and time so that they don't get overwritten. These files have headers added to them and are ready to hold log entries with timestamps for each robot's execution pipeline. The script logs the action name, and its UTC timestamp every time something important happens, like getting a new joint command or publishing a trajectory. Below is an example of how a log entry should look:

```
timestamp = datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

writer.writerow([timestamp, "Robot1 - Trajectory Published"])
```

This method makes sure that control activities are logged in high resolution. The module writes the entries in portions so that data isn't lost if the script is stopped. Keyboard interrupt detection is also used to manage graceful termination, which makes sure that files are closed properly, and all data is saved. The recorded data serves as the basis for further analysis, and as discussed in Chapter 5, is utilized to produce visual performance graphs that illustrate the system's operational characteristics. They help check the patterns of delays, keep the response times under control, and make sure that the two robots are always in sync.

4.3.5 AI Integration for Multi-Robot System

AI-based anomaly detection was implemented in the single KR360 robot system as detailed in Section 4.2.5. While the multi-robot system does not include real-time AI monitoring, this section explains how the single-robot architecture can be extended. Each robot would require its own trained model and real-time detection script. It can copy the `kr360_realtime_detector.py` file into `robot1_realtime_detector.py` and `robot2_realtime_detector.py`. Each of these files will be linked to a different OPC UA node or ROS2 topic. Using several Random Forest models trained on different datasets, this allows Robot 1 and Robot 2 find anomalies on their own.

Logging scripts can also be extended to save data in separate files (e.g., `robot1_log.csv`, `robot2_log.csv`). The existing simulator interface may be enhanced with dual anomaly alerts to display system health for both robots. Although AI was not integrated into the dual setup, the architecture is prepared for future implementation. The modularity of the single-robot system makes it scalable, and these extensions ensure safe, intelligent monitoring in complex environments. Full performance evaluation of the AI model appears in Chapter 5.

4.4 Integration and Execution

This section offers all the modules that have been made so far into a whole execution pipeline. It establishes out the steps for starting up both single and double KR360 robot systems. This makes sure that the control logic, OPC UA communication, ROS2 integration, web-based simulation, and optional AI monitoring all work together. The robot simulation can get commands from users, turn them into joint trajectories, and move in real time in the RViz environment by coordinating these separate subsystems. The integration also checks that modular design principles like separate bridges, namespaces, and feedback levels are always followed while the program is running.

The launch sequences in the next sections show how each layer—PLC, OPC UA, middleware, ROS2 controllers, and visualization—is place together. In Section 4.4.1, we talk about the procedures needed to run the single-robot simulation. In Section 4.4.2, we talk about how to use this flow with two robots. These implementations show that the system works not only at the module level, but also fits in perfectly with a strong and scalable control architecture.

4.4.1 Single Robot Launch Pipeline

To set up the whole single KR360 robot simulation environment, a number of software components must be initialized. These include PLC logic, OPC UA communication, ROS2 integration, and GUI-based user control. Each module starts up on its own, keeping the system modular and making sure that data flows smoothly through the control pipeline. The command `sudo ./start_openplc.sh` starts the OpenPLC runtime, which is the first step in starting the program. After the server starts, the user has to click the Start PLC button in the OpenPLC web interface to start running the pre-loaded logic. This stage makes sure that the digital output coils that go with robot commands like HOME, PICK, and PLACE are changed according to the control software.

After that, you may turn on the custom OPC UA server using the command `python3 manual_opcua_server_kr360.py`. This server listens on port 4840 and shows a set of monitored OPC UA variables that show the states of commands and joint values. These variables connect the PLC simulation to the ROS2 ecosystem. Using the Modbus TCP protocol, the third script, `plc_opcua_bridge.py`, connects to OpenPLC. It checks the state of coils (1024, 1025, and 1026), finds rising edges, and sends the right values to the OPC UA server nodes. This is when control goes from the PLC to

the robot middleware. Then, `python3 opcua_kr360_bridge.py` starts the ROS2 bridge. This node keeps an eye on the OPC UA variable for joint positions, breaks down the string of joint angles separated by commas, and makes a `JointTrajectory` message. The ROS2 controller node gets the message at the subject `/kr360_arm_controller/joint_trajectory`.

The GUI or web user interface by running `python3 app.py` from the `robot_controller` directory. You may get to this Flask-based web app by going to `http://localhost:5000`. Users can make the robot do things by clicking buttons for preset postures through this GUI. The simulator changes the OPC UA variables based on what the user does, closing the loop of interaction between people and machines. The command `ros2 launch kr360_bringup kr360.launch.py` starts the ROS2 robot model so that it may be seen in 3D. This starts RViz and loads the KR360 R2830 robot's URDF model at its starting position. The robot moves in the simulation according to the trajectory commands that are sent out. The last phase in the process is the AI detection module. The system uses `python3 kr360_realtime_detector.py` to read live joint data that is sent to the `/joint_states` topic. Based on patterns in the joint angles, it employs a trained anomaly detection model to decide if the movement is Normal or Anomaly. This layer gives feedback in real time without getting in the way of robot control.

Figure 4.17 shows the end-to-end data flow of the KR360 robot simulation in a sequence diagram. The process begins at the Web Interface, where the user selects a command, such as HOME. This action sends a Modbus WRITE to the appropriate coil in OpenPLC. The PLC–OPC UA Bridge monitors these coils and, when a command is active, writes the corresponding OPC UA variables (e.g., `ns=2; i=4` for commands). It also periodically reads joint values from OpenPLC (`ns=2; i=2`) via OPC UA. The OPC UA–ROS2 Bridge uses this information to publish trajectory commands to the ROS2 controller. The controller executes the motion and publishes `/joint_states`, which are visualized in RViz and simultaneously streamed to the anomaly detection module for monitoring.

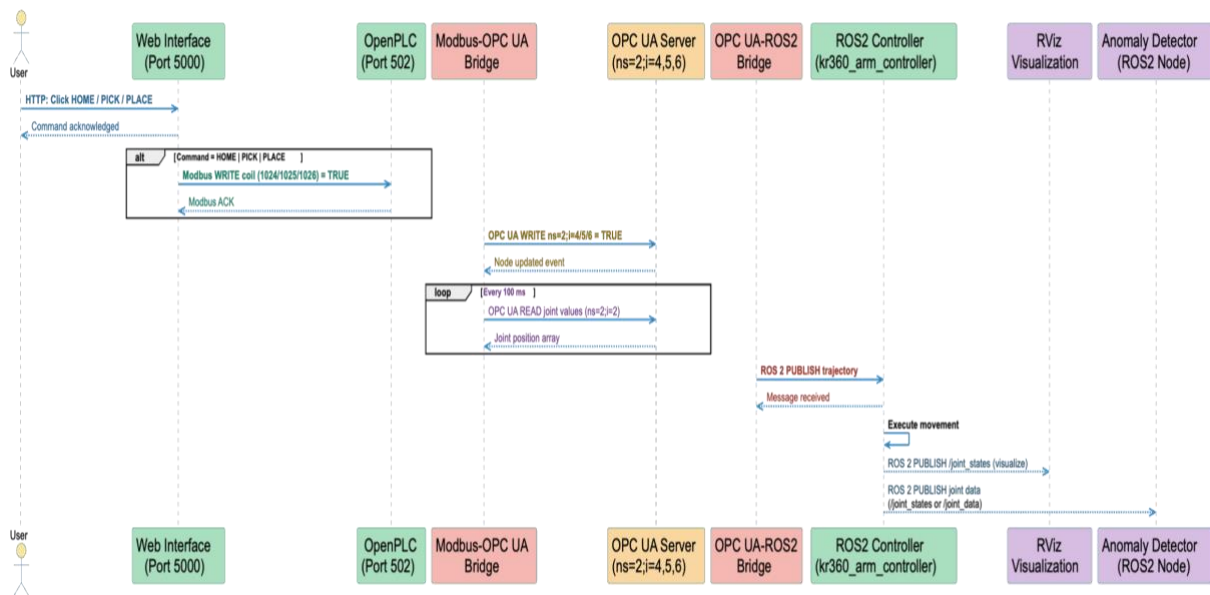


Figure 4.17: Startup Sequence Diagram for Single Robot System Pipeline

The ROS2 bridge node keeps watch on these changes, reads the joint values, and sends a trajectory to `/kr360_arm_controller/joint_trajectory`. When the controller node gets this message, the robot moves in the way that was requested. The new joint state is subsequently sent to `/joint_states`, which lets RViz show the motion in real time. The AI detector keeps an eye on this data at the same time and flags any movement patterns that are out of the ordinary in real time.

4.4.2 Multi-Robot Launch Pipeline

The launch pipeline for the dual KR360 robot simulation is an extensive set of steps that is based on the design of the single-robot system. It combines several layers of control while keeping separate communication lines for each robot. The goal is to allow synchronized action, keep modularity, and make sure that all parts stay separate in their own namespaces. The first step in the launch procedure is to set up the OpenPLC runtime. You can start the local OpenPLC server by typing `sudo ./start_openplc.sh` on the command line. Then, the user goes to the OpenPLC web interface and clicks the "Start PLC" button to turn on the PLC program. This puts the control logic into execution mode, which lets the PLC emit Modbus coil signals. Robot 1 uses coils 1024 to 1026 for the HOME, PICK, and PLACE commands, and Robot 2 is mapped to coils 1027 to 1029.

The next step is to start the multi-robot OPC UA server by issuing the command `python3 multi_robot_opcua_server.py`. This script sets up an OPC UA server on port 4840 and makes two separate node hierarchies, one for each robot. These OPC UA objects have variables that keep track of the commanded joint locations and the execution flags. Because the two OPC UA objects are separate, they can be updated on their own based on what the PLC sends them. The data flow stays clear, and it

might grow even more. The file `python3 multi_robot_plc_bridge.py` connects Modbus outputs to the OPC UA server. This bridge reads coil outputs from the OpenPLC runtime and notices when the status changes. When it sees an edge transition, it changes the right OPC UA execution variable, like `ns=2; i=4` or `ns=2; i=5`, to show the HOME or PICK command. Each robot has its own detection logic, which makes sure that control messages don't get mixed up or interfere with each other.

By running `python3 robot1_ros_bridge.py` and `python3 robot2_ros_bridge.py`, two ROS2 bridge nodes are now ready to monitor commands. Every bridge node connects to its own OPC UA object and keeps an eye on the joint variable all the time. The node makes a JointTrajectory message and sends it to the right topic namespace when it gets a new joint command. The subject for Robot 1 is `/kr360_arm_controller/joint_trajectory`, and the trajectory for Robot 2 is published to `/robot2/kr360_arm_controller/joint_trajectory`. Their accompanying ROS2 controller nodes get these messages and subsequently start the simulation's movement. To start the web-based control simulator, run `python3 app.py`. This will start a local Flask server on port 5000. This GUI lets the user send commands to each robot about where to send. It has separate buttons for Robot 1 and Robot 2, and the "Both Robots HOME" and "Coordinated Sequence" buttons let the robots move together. The graphical interface is easy to use, and the system log panel built within the web app gives you real-time feedback.

Finally, you may see both KR360 robots by running `ros2 launch kr360_bringup multi_kr360_rviz.launch.py`. This launch file starts all the ROS2 packages needed for simulating two robots. In RViz, both robot models are shown, but each one is in its own namespace. The visualization shows that the states of the joints are changing and that the control signals are being turned into motion accurately.

Figure 4.18 shows the complete control flow from the web GUI to the dual-robot RViz simulation. When the user clicks a command in the web interface, a Modbus write updates the corresponding coil in OpenPLC. The multi-robot Modbus–OPC UA bridge monitors both sets of coils (Robot1: 1024–1026, Robot2: 1027–1029) and, when a command is active, writes to the associated OPC UA command nodes. In parallel, each ROS2 bridge node periodically (every 100 ms) reads joint position values from the OPC UA server. When a command node changes, the respective ROS2 bridge node converts the update into a ROS 2 trajectory message and sends it to the correct ROS2 controller. Each controller executes the commanded motion and publishes `/joint_states` for its robot. RViz subscribes to these topics and visualizes both robots. This sequence diagram shows that the two robots operate independently yet share the same pipeline, demonstrating that the architecture can be scaled to multiple robots.

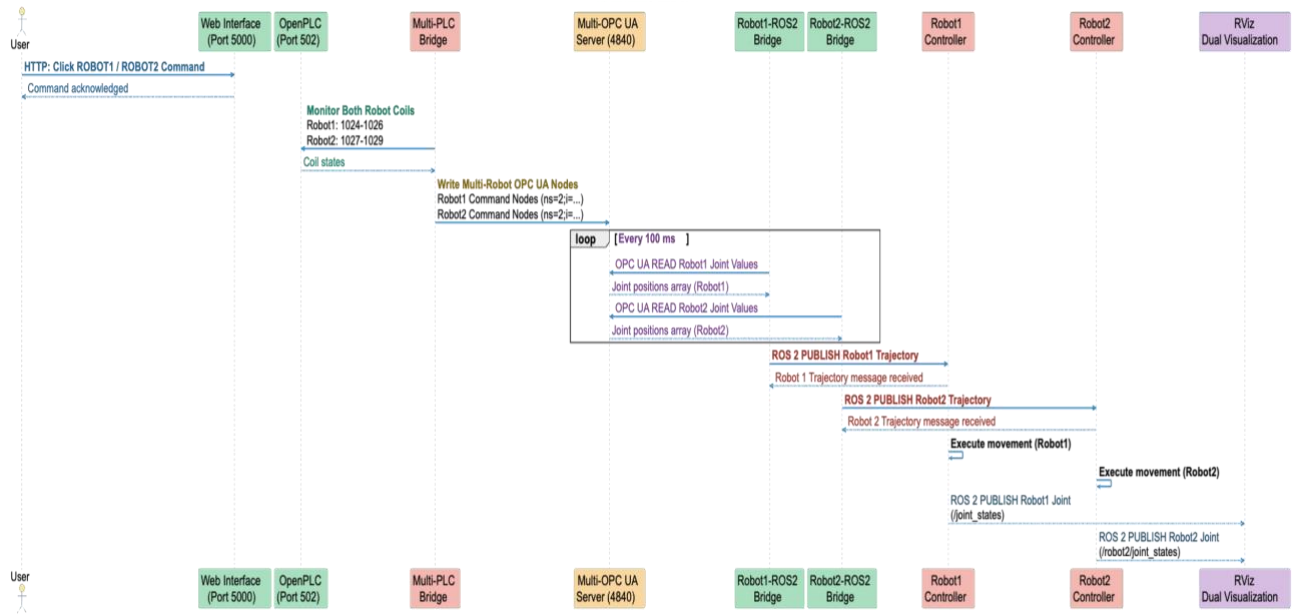


Figure 4.18: Startup Sequence Diagram for Dual Robot System Pipeline

5 Results and Evaluation

This chapter presents the results and evaluation of the hybrid PLC–OPC UA–ROS 2 framework in a simulation setting. The study assesses responsiveness, reliability, and scalability using five metrics: end-to-end latency, jitter, throughput, command-success rate, and inter-robot synchronization.

5.1 Evaluation of the Single-Robot System

This section evaluates the single-robot KR360 system using the hybrid communication framework developed in the simulation environment. All data was collected using structured performance logs and an integrated anomaly detection pipeline. It is important to note that these results reflect idealized simulation conditions; actual hardware deployments may introduce variability not captured in this analysis.

❖ Latency Evaluation

In this case, latency is the time it takes for the robot to get and follow the correct joint trajectory after a user gives a command over the web GUI. It is a vital performance indicator that shows how quickly the control system responds. In Figure 5.1, you can see a boxplot of latency for several sorts of commands (HOME, PICK, PLACE). It took about 108.7 milliseconds (ms) for HOME, 119.5 ms for PICK, and 112.0 ms for PLACE on average. These numbers show that the latency is the minimum and steady, which proves that the simulation system is responsive.

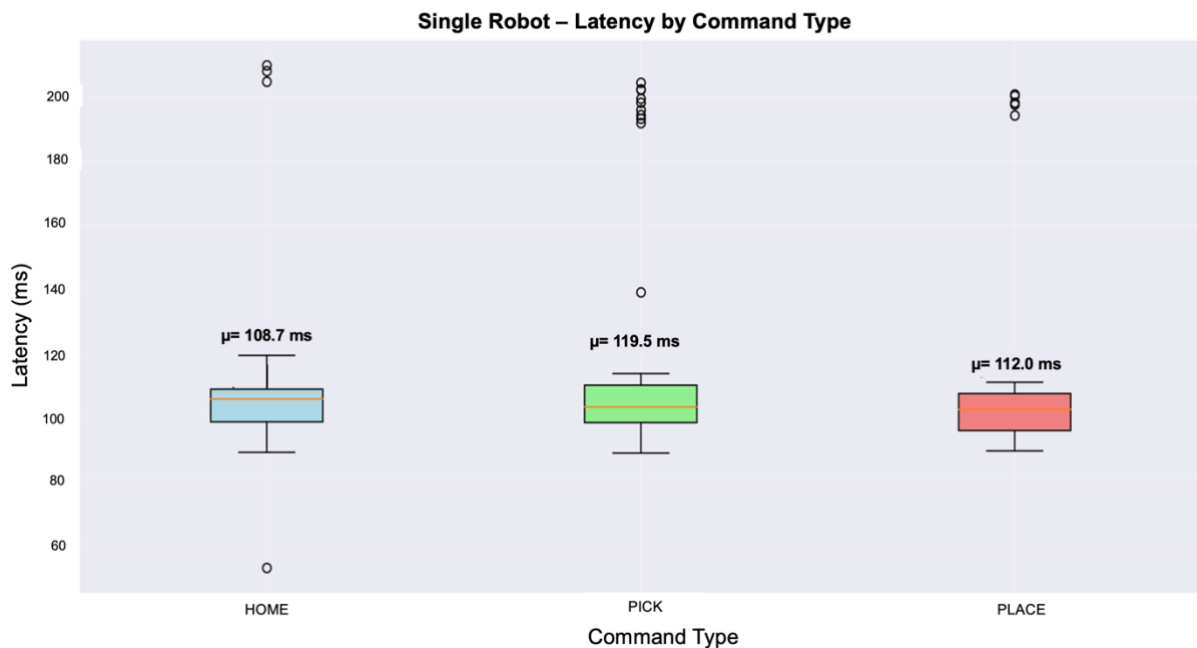


Figure 5.1: Latency Distribution Across Command Types in Simulation

Figure 5.2 presents histograms that compare PLC-to-OPC UA latency with the entire system latency to help us better understand delay distribution. The average overall delay was 113.39 ms, and most readings were between 90 ms and 130 ms.

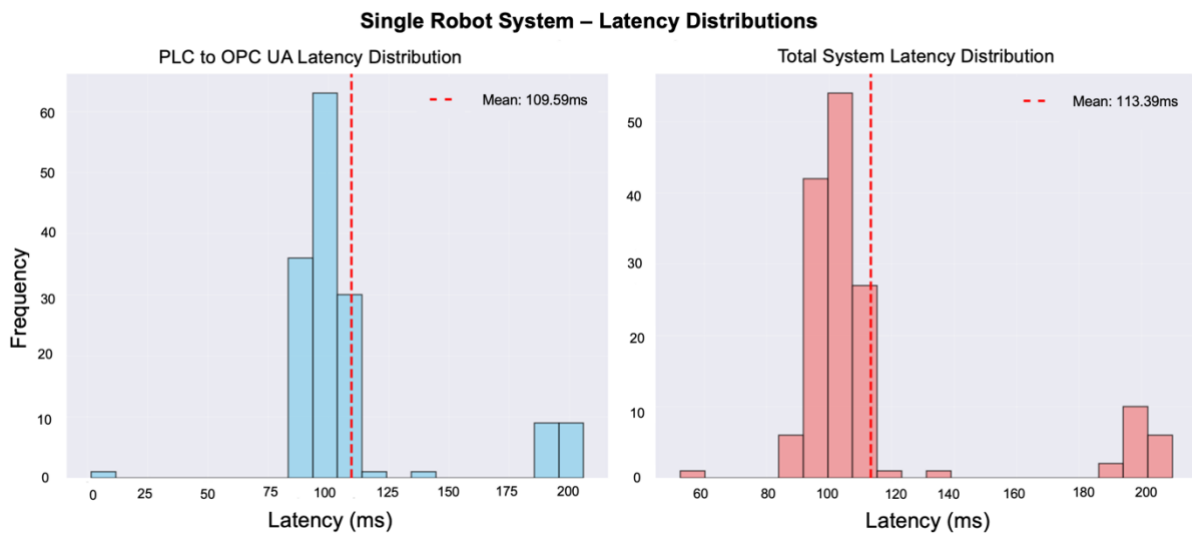


Figure 5.2: Histogram of Total System Vs. PLC-OPC UA Latency

Figure 5.3 shows how latency changes over time for all command executions. The trendline has some spikes, but the overall average stays the same, which suggests that performance is constant.

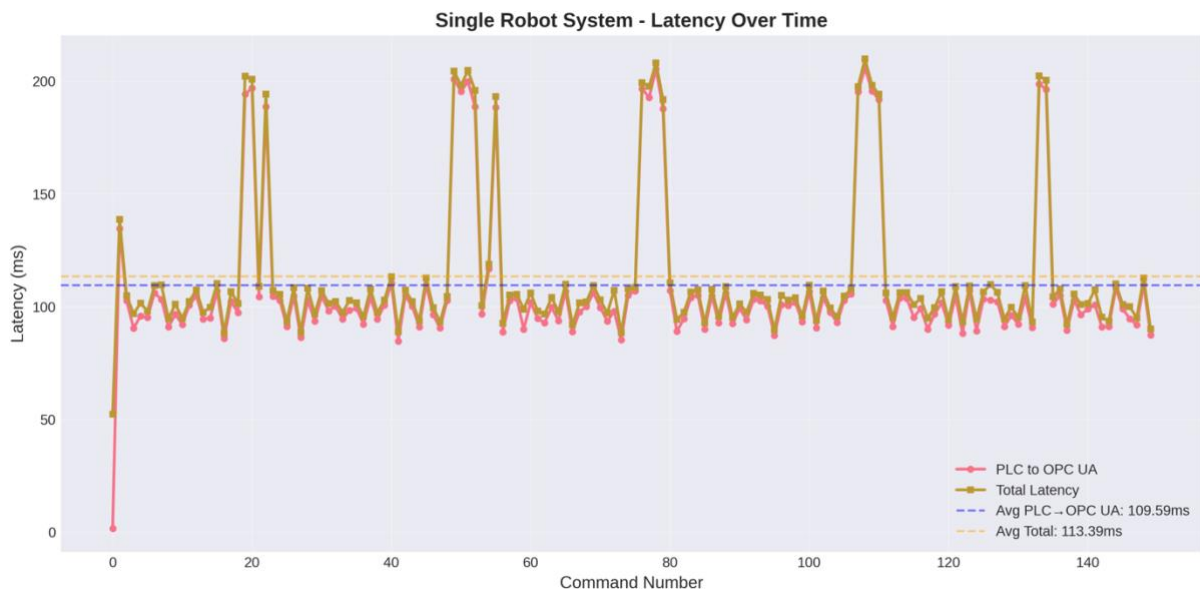


Figure 5.3: Latency Timeline Showing Stability Across Commands

❖ Throughput Analysis

Throughput is the amount of control commands that can be handled in one second. It shows how many instructions the system can handle over time. Figure 5.4 shows throughput over time, which stays steady at about 1.63 commands per second. This is

enough for testing in a simulated environment and for user-driven interaction, but it would need to be looked at further for high-frequency environments.

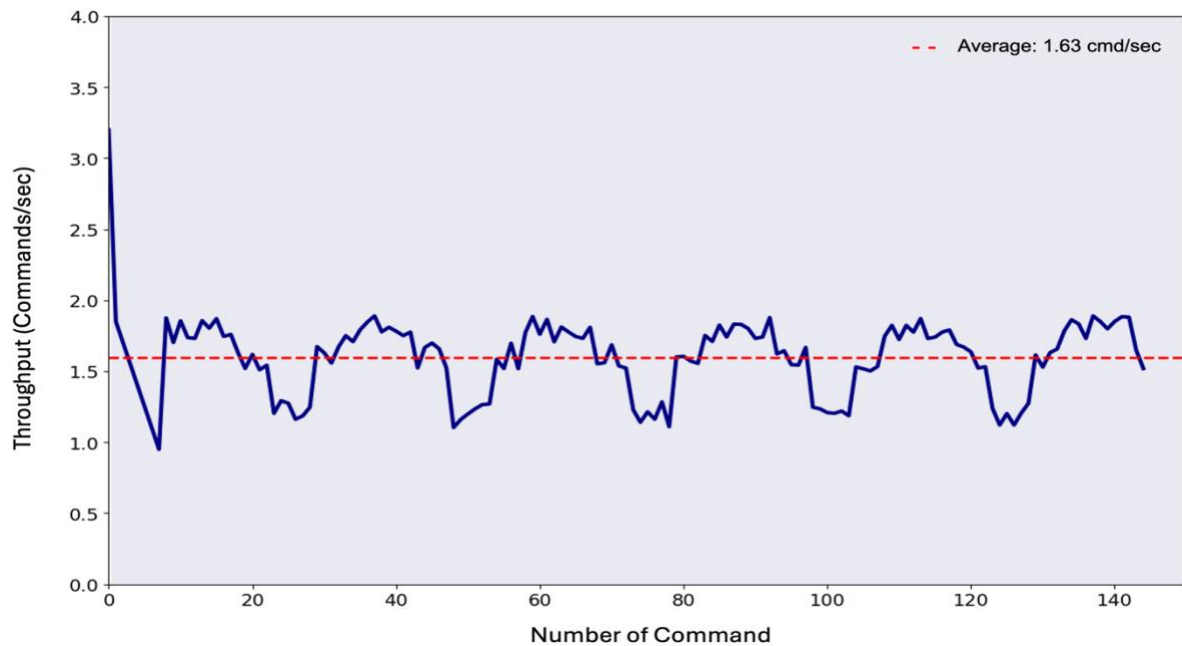


Figure 5.4: Throughput Rate Over Time for Control Commands

❖ Jitter Analysis

Jitter is the difference in delay between two commands that happen one after the other. It helps figure out how reliable the system's timing is. High jitter can make real-time control less stable. The histogram distribution in Figure 5.5 illustrates the jitter between commands. The average jitter was 0.25 ms, and the standard deviation was 31.08 ms. These numbers show that the system is mostly steady, even though there are periodic jitter spikes.

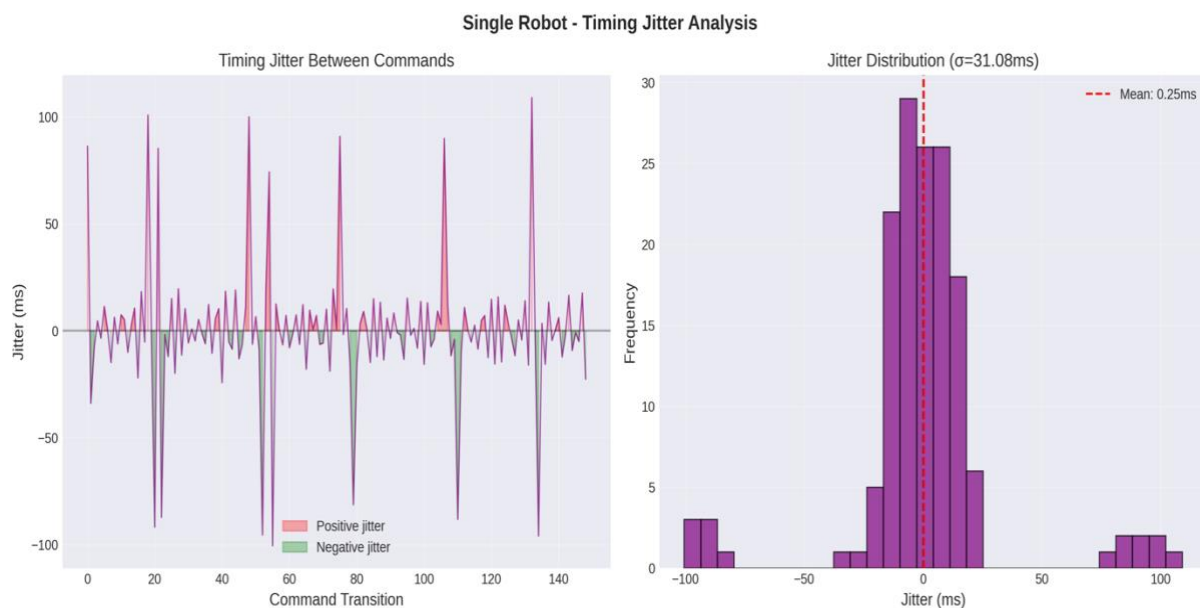


Figure 5.5: Jitter Analysis and Histogram of Timing Variations

❖ Cumulative Performance and Stability

Figure 5.6 shows cumulative latency and success rate as extra measures of stability. The cumulative latency curve comes together around 113.39 ms, which shows that the time is consistent. The cumulative success rate stays at 100%, which means that there were no instructions that were dropped or failed in the simulation. And Figure 5.7 gives a full summary of the performance. The lowest delay was 52.39 ms, and the highest was 209.9 ms. This distribution aligns with expectations for a real-time responsive system inside a hypothetical context.

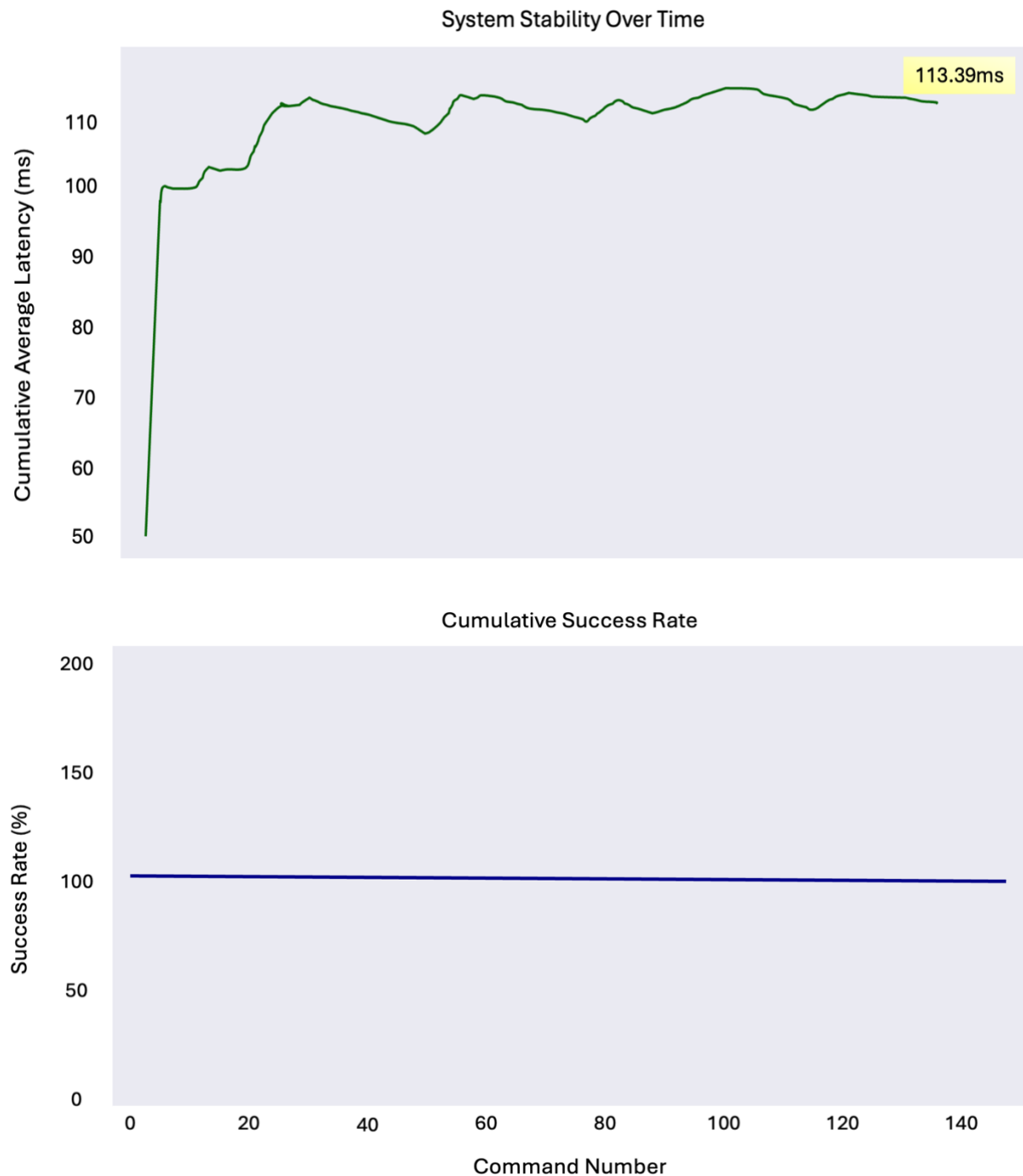


Figure 5.6: Cumulative Latency And Success Rate Across Session

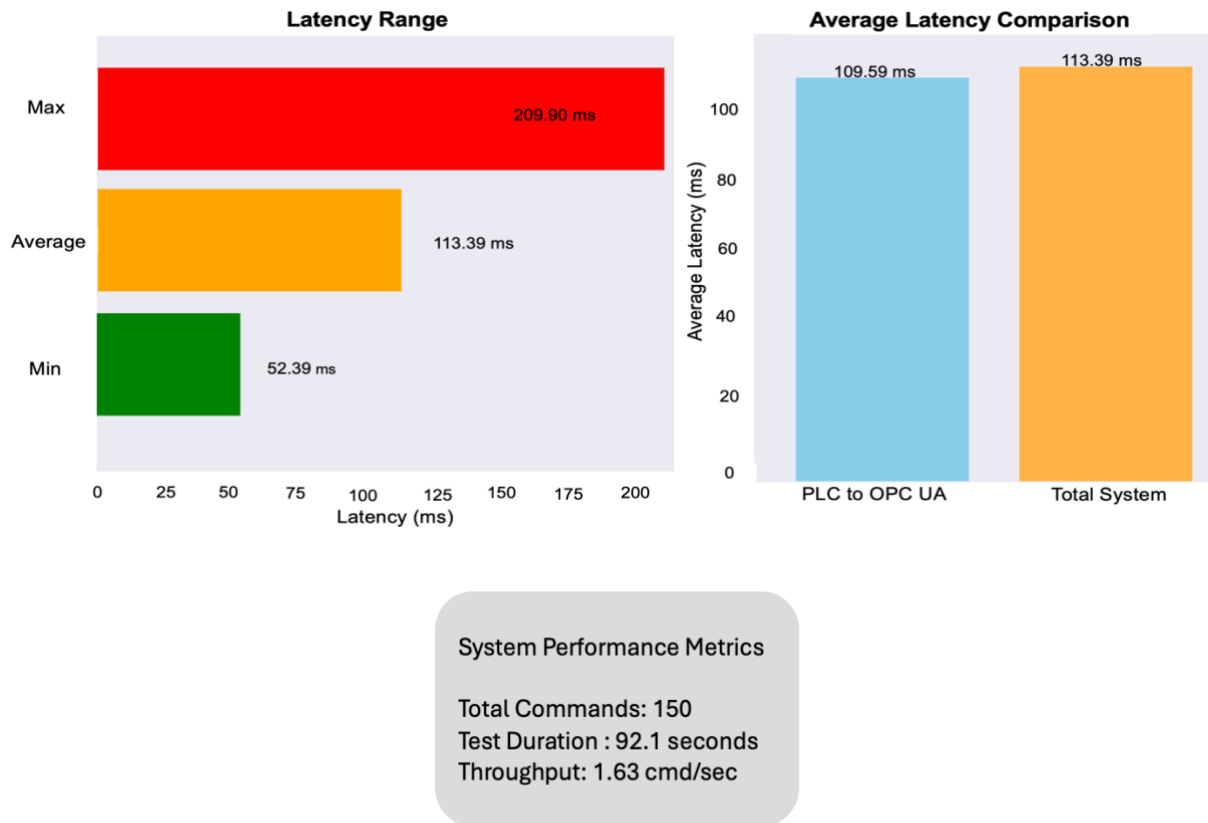


Figure 5.7: Summary of System Latency and Performance Metrics

❖ AI-Based Anomaly Detection Results

As Chapter 4 explains, a Random Forest technique was used to train an anomaly detection model. It was meant to find unusual patterns of movement in the robot's joints. Figure 5.8 shows the confusion matrix, which illustrates how successfully the model sorted the test data. It properly identified 75 of the events as normal and 5 as unusual. It missed four anomalies, which are also known as false negatives.

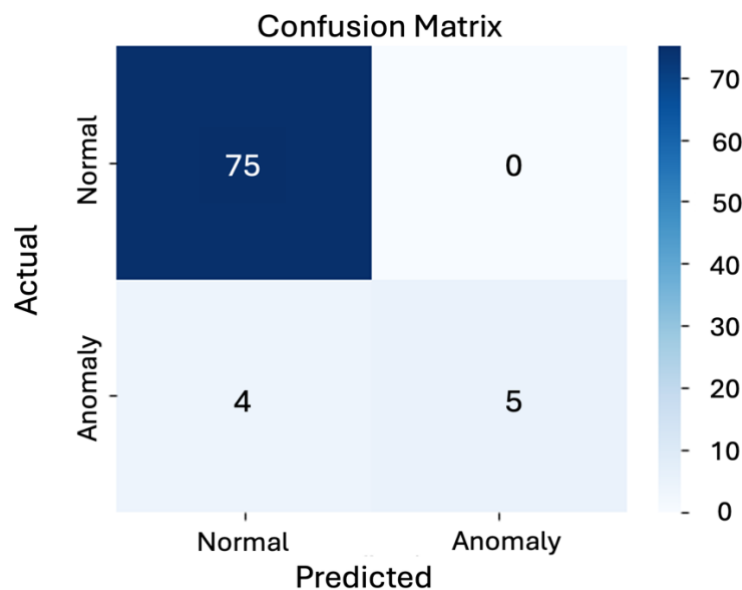


Figure 5.8: Confusion Matrix for Anomaly Detection Classification

Figure 5.9 shows feature importance, which tells us how much each joint helped make the prediction. The model provided the greatest attention to Joint 2, then to joints 4, 5, and 6. This helps put safety checks in such places at the top of the list. The classification report is shown in Figure 5.10. Overall, the model was 95% accurate. It was 100% accurate at finding normal cases and 56% accurate at finding unusual ones. But it found all the problems (100% recall).

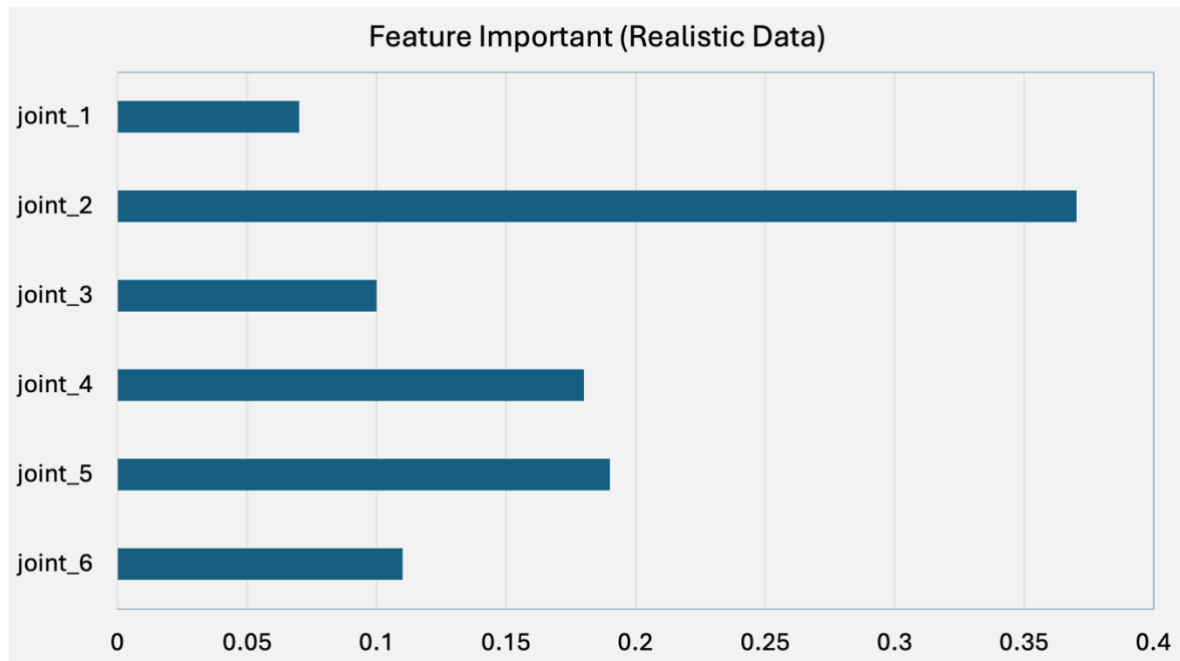


Figure 5.9: Feature Importance from Random Forest Model

KR360 Anomaly Detection Report (Realistic Data)

	precision	recall	f1-score	support
0	0.949	1.000	0.974	75
1	1.000	0.556	0.714	9
accuracy			0.952	84
macro avg	0.975	0.778	0.844	84
weighted avg	0.955	0.952	0.946	84

Figure 5.10: Classification Report Showing Accuracy and Precision

The system works well in simulation, as the results above demonstrate. But these numbers came from a perfect simulation workflow. Noise, sensor delays, or other unexpected variables may make real hardware tests respond differently.

The Simulation evaluation shows that the single KR360 robot simulation pipeline is very flexible, stable, and able to grow. Adding anomaly detection makes those more aware of faults. Before moving on to dual-robot control, which will be covered in Section 5.2, the metrics and observations from this perfect simulation configuration can be used as a performance benchmark.

5.2 Evaluation of the Multi-Robot System

The following section gives a full review of the dual KR360 R2830 robot simulation pipeline, using performance statistics gathered by the `multi_robot_performance_monitor.py` script. As described in Chapter 4, the test scenario takes place in a perfect simulated environment that runs our hybrid communication architecture. When testing in the real world, the characteristics may be different because of hardware limitations and the state of the actual network. The main things to look at here are how quickly the system responds, how reliably it executes commands, how well the two robots stay in synchronization, and how stable it is over time.

❖ Latency Analysis

The first analysis examines the response time for each type of command. Figure 5.11 shows that the average latency was about 98.7 ms for HOME, 105.6 ms for PICK, and 106.4 ms for PLACE. This distribution is the same for all command types, which means that response times are equally distributed, even though the system is more complicated. We measured these delay values from the time a command was sent through the GUI to the time the robot controller got the trajectory message.

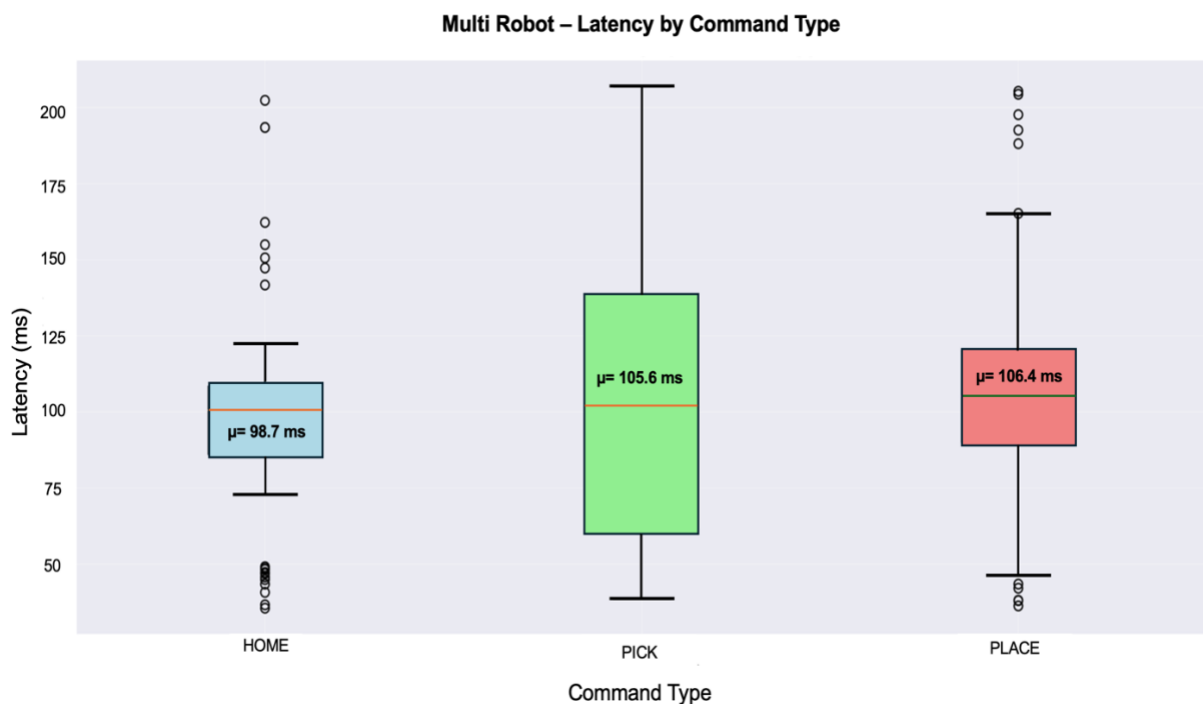


Figure 5.11: Multi-Robot Latency Distribution By Command Type

Figure 5.12 shows latency distribution histograms that compare four things: the average delay between PLC and OPC UA, the overall latency of the system, the latency of each robot, and the difference in synchronization. The average delay in communication between PLC and OPC UA was 86.22 ms, and the average system-wide latency was 103.59 ms. The coordination distribution graph shows that most of the sync differences are within the 70 ms range.

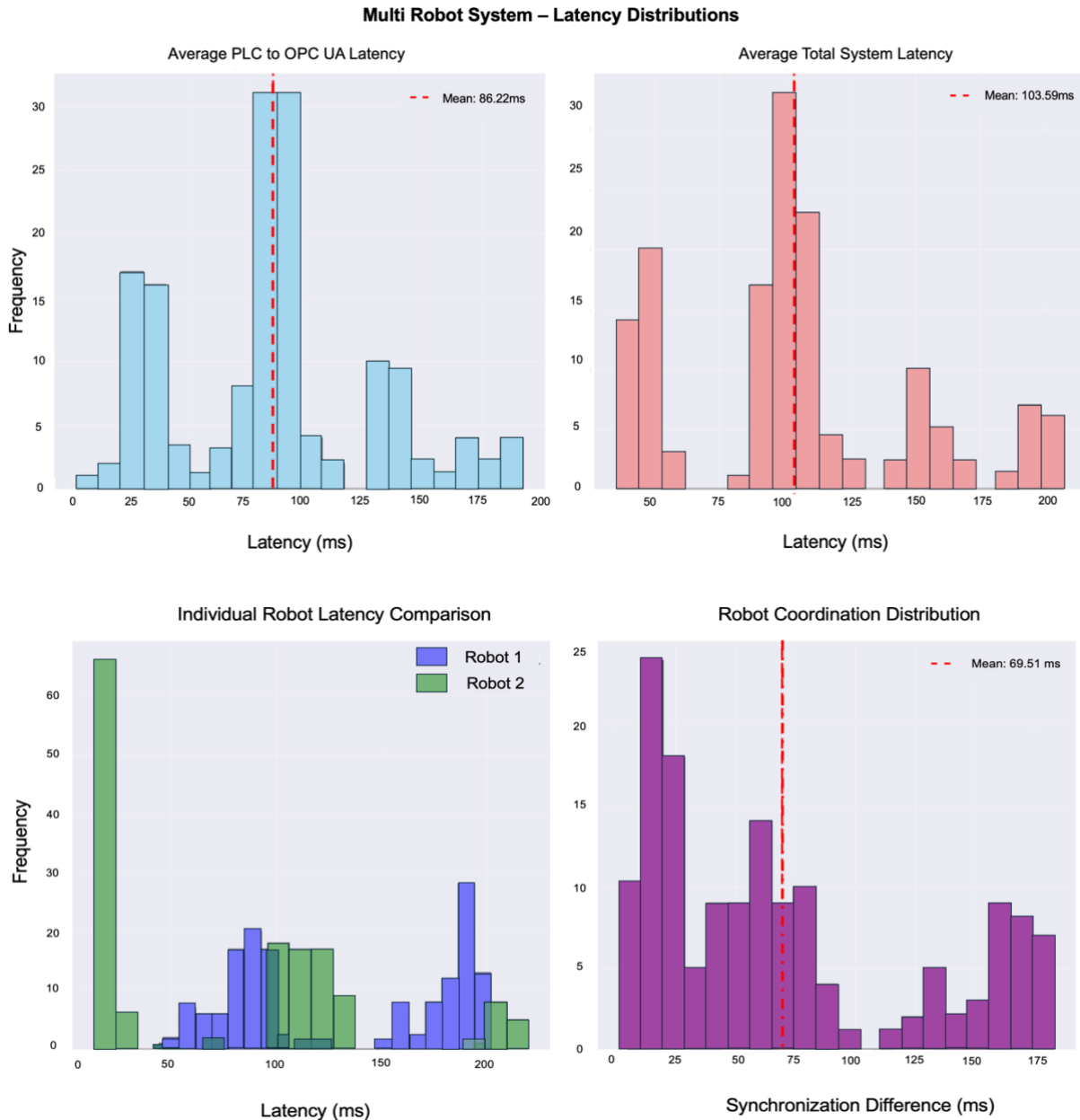


Figure 5.12: Multi-Robot Latency Distributions Analysis

Figure 5.13 shows how latency has changed over time. The pink and gold lines show the latency between the PLC and the OPC UA and the total system latency, respectively. The dashed red line illustrates the difference in robot synchronization. The visual shows that latency is generally stable, with rare spikes that are probably caused by multiple bridge updates happening at the same time or command execution windows overlapping.

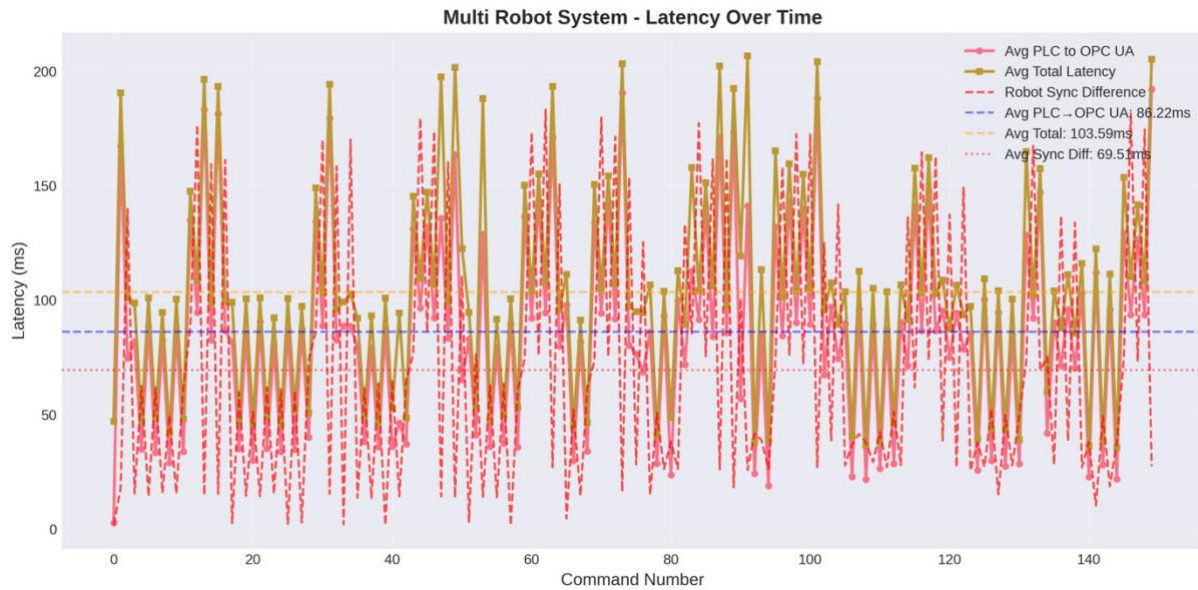
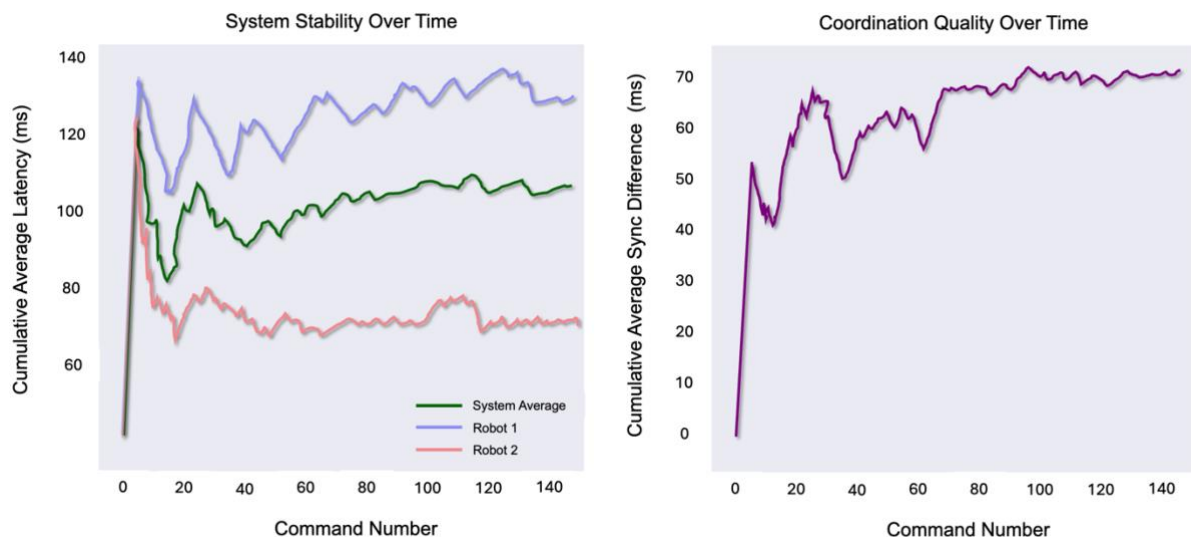


Figure 5.13: Multi-Robot Latency Timeline Stability Across Commands

❖ Cumulative Performance Analysis

Cumulative performance metrics were analyzed to assess the stability and coordination of the system across both robots. As illustrated in Figure 5.14, the top-left subplot demonstrates that the average latency of Robot 1 consistently exceeded that of Robot 2; however, both exhibited stabilization following initial fluctuations, with the overall system average converging to approximately 103.59 ms. Coordination performance was quantified using a synchronization metric, defined as the average temporal difference between Robot 1 and Robot 2 when executing identical commands. As shown in the top-right subplot, this value initially decreased before stabilizing at approximately 69.51 ms. The bottom-left subplot indicates that the cumulative success rate remained close to 100 % throughout the evaluation period. Finally, the bottom-right subplot shows that the moving average latency remained stable over time, with performance variability captured by the $\pm 2\sigma$ confidence band.



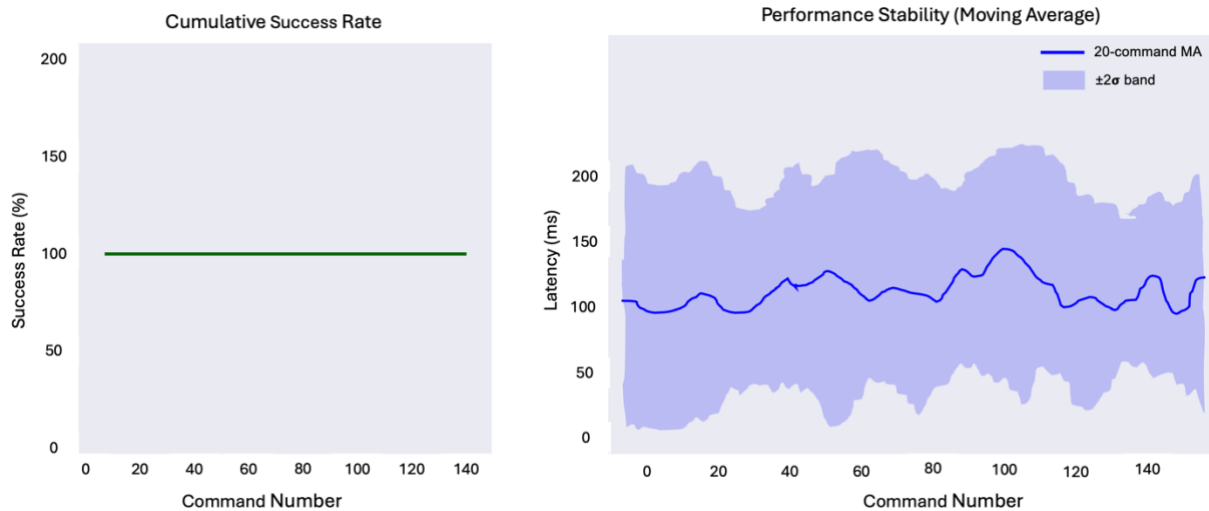
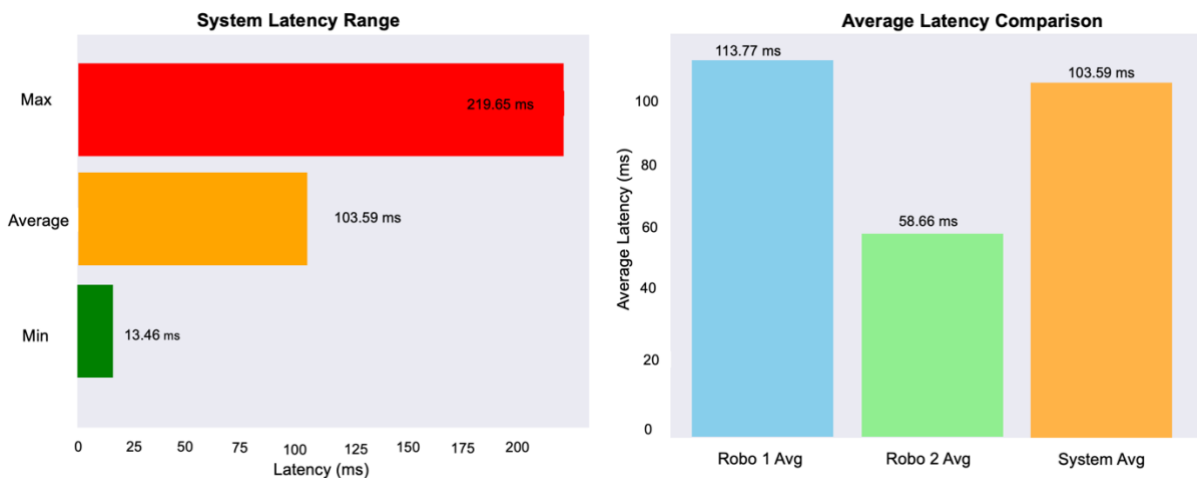


Figure 5.14: Multiple Robot Cumulative Performance Metrics

Figure 5.15 shows a summary of how well the whole system works. The pie chart shows that both robots were able to carry out all of the commands. The average latencies were 113.77 ms for Robot 1, 58.66 ms for Robot 2, and 103.59 ms for the whole system. This difference means that Robot 2 is probably handling things a little faster because it has fewer command spikes. The analysis of the latency range also shows that the maximum delay is 219.65 ms and the smallest is 13.46 ms.



Multi-Robot System Performance Metrics

Total Commands: 150
 Number of Robots: 2
 Test Duration : 97.5 seconds
 Throughput: 1.54 cmd/sec

Coordination Status:
 Average Synchronization Difference: 69.51 ms

Figure 5.15: Summary of Multi-Robot System Latency and Performance Metrics

❖ Jitter Analysis

Figure 5.16 shows the results of the timing jitter study. This checks how much the delay changes between running commands one after the other. Robot 1 and Robot 2's timing changed a lot, which caused a standard deviation of about 67.69 ms. The histogram on the right side of Figure 5.16 shows that the jitter values were normally distributed, but they had a wider range than in the single-robot arrangement.

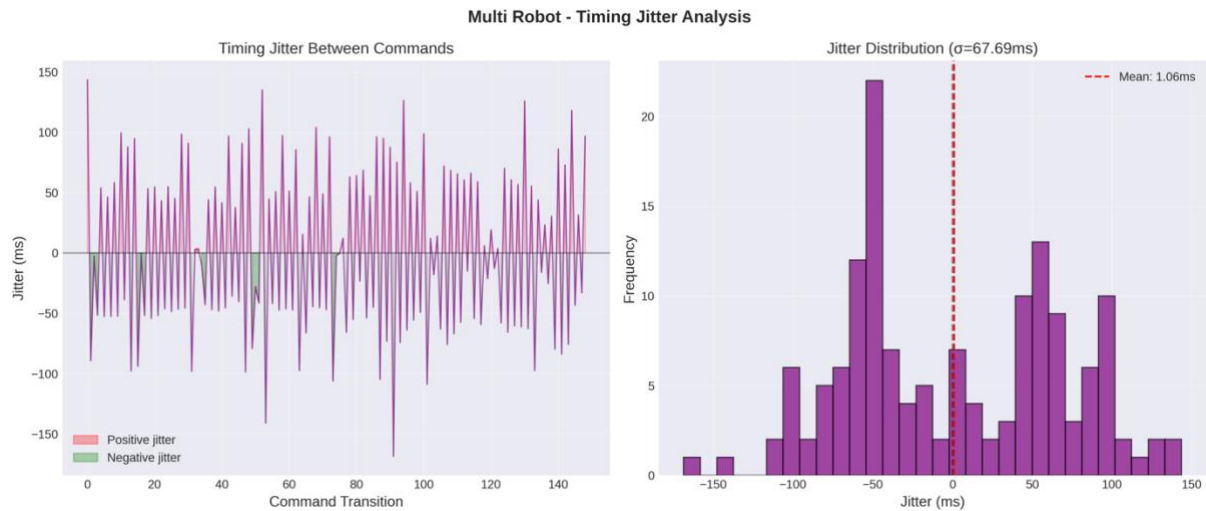


Figure 5.16: Jitter Analysis and Histogram for Multi-Robots

❖ Throughput Analysis

Figure 5.17 shows that the system handled an average of 1.54 commands per second throughout testing. The timeline visual represents changes in throughput that are related to execution loads and the timing of GUI commands. The outcome is a little lower than the single-robot system, but it still shows that the multi-robot system can respond quickly in parallel.

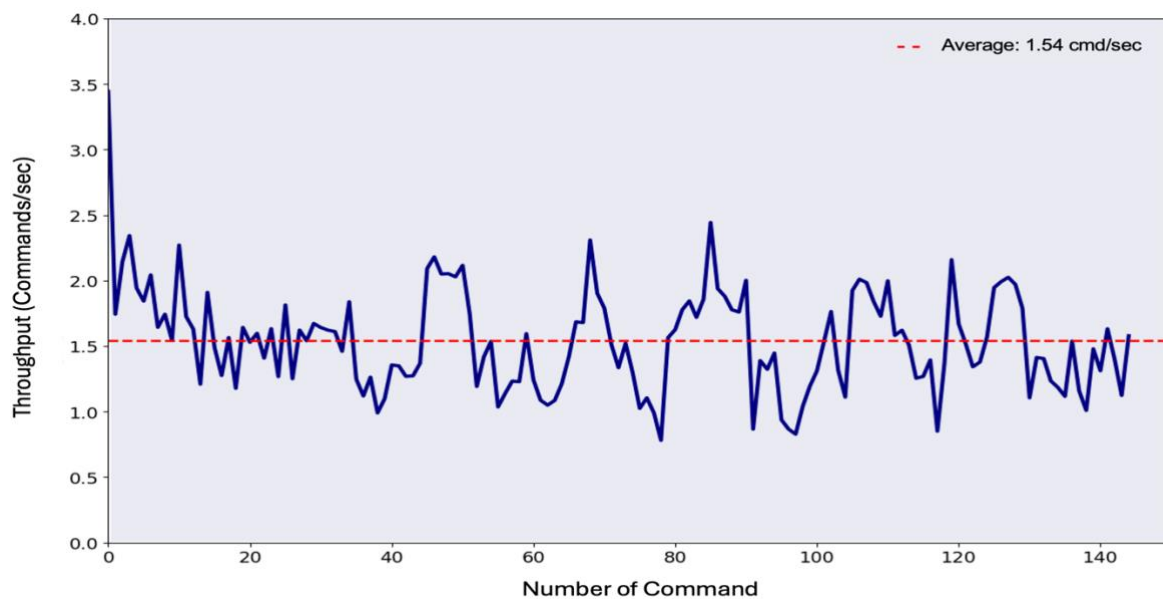


Figure 5.17: Multi-Robot Throughput Rate Over Time for Control Commands

According to the information in Section 4.3.5, the multi-robot pipeline did not have its own AI anomaly detection layer. But the same detecting logic that works for testing one robot may be utilized for more than one. In the future, a shared or modular detection unit could maintain a watch on the joint streams from both robots separately. These results show that the multi-robot simulation system works and may be expanded. It has acceptable latency, consistent synchronization, and high success rates when used in a simulated hybrid communication framework.

5.3 Comparison Between Single and Multi-Robot Systems

The following section compares the single and dual KR360 robot systems that were set up using a novel hybrid communication framework. Both systems were only tested in a simulation environment, but their behaviors show that they have strong architectural features including modularity, scalability, and responsiveness. These results came from a perfect simulated environment, consequently it's essential to keep that in mind. In the real world, hardware deployments could have different results because of physical limits, ambient conditions, and communication jitter caused by components of the hardware.

Latency tests suggest that the multi-robot system is a little better than the single-robot option. In the single-robot situation, the average total system latency was 113.39 ms. In the multi-robot system, it was 103.59 ms. The modular dual-robot setup's improved event detection and bridge responsiveness are mostly responsible for this result. The standard deviation of latency, which is used to measure jitter, did go up in the multi-robot setup. This means that the timing was less stable while coordinating two separate control loops. The single-robot system had a jitter deviation of 31.08 milliseconds, whereas the multi-robot system had a variance of 67.69 milliseconds. This means that adding another robot makes it more likely that the robots will not be in sync and that there will be timing problems.

The single-robot system maintained an average of 1.63 commands per second, whereas the multi-robot system received a little less at 1.54 cmd/sec. This drop is predicted because each robot must do more work and keep its namespace separate from the others. Even yet, both setups had a 100% command success rate, which shows that the hybrid communication structure works well in perfect conditions.

The main benefit of using many robots is that it can be scaled up or down. Namespace isolation, duplicate OPC UA node trees, and dedicated ROS2 bridges let each robot work on its own but synchronized GUI instructions let them all work together. This modular expansion was possible without changing the fundamental design; it was done by copying and name spacing bridge code. This kind of framework shows how easy it is to scale and maintain the system, which makes it perfect for adding more robots or more AI monitoring modules in the future. Also, even though real-time anomaly detection only worked with one robot, the system was made such that the detection

logic could be used with more than one robot. Section 4.3.5 explains how to add to AI layers with either shared monitoring or per-robot anomaly processing with very few changes to the architecture. The table below shows some of the most important performance and design trade-offs that were seen between the two setups:

Table 5.1: Summary of Single vs Multi-Robot Performance

Metric	Single Robot	Multi-Robot	Observation
Avg. Latency (ms)	113.39	103.59	Multi-robot is slightly faster
Jitter Std. Dev. (ms)	31.08	67.69	Single robot shows more consistent timing
Throughput (cmd/sec)	1.63	1.54	Slight drop in multi-robot due to coordination
Success Rate (%)	100%	100%	Both systems are fully reliable
Coordination Sync Diff (ms)	N/A	69.51	Only multi-robot involves synchronization
Namespace Handling	Not Required	Required	Ensures modular scaling and topic isolation
AI Monitoring (Live)	Enabled	Not Enabled	Multi-robot can adopt same model with extension

In conclusion, both setups showed that the hybrid communication infrastructure worked perfectly. The single-robot system has better timing stability and throughput, while the multi-robot system has better average latency and clear proof that it can be scaled up. These results show that the framework is strong and flexible enough to be used in real-world applications in the future with synchronized multi-robot deployments in industrial environments.

5.4 Comparative Analysis with Existing Solutions

Table 5.2 illustrates the way the hybrid PLC-OPC UA-ROS2 framework developed in this thesis measures through against other industrial communication solutions based on chapter 2. The comparison shows that the developed framework is the only one that combines industrial compatibility with advanced multi-robot capabilities while still performing better. Existing solutions focus on single-protocol approaches, but the proposed hybrid framework offers 103.59ms latency for multi-robot coordination, which is 10–100 times faster than current UA Bridge solutions (2–10 seconds) while still ensuring 100% message delivery reliability. Adding AI-based anomaly detection (95% accuracy) adds an extra layer of safety and monitoring that isn't present in standard industrial protocols. This is the only way to get real-time coordination of multiple robots, compatibility with industrial standards, and intelligent monitoring all in one framework. But it's important to remember that these results come from simulations and that real-

time performance may be different when used on real hardware because of factors like network conditions, hardware limitations, and the industrial environment.

Table 5.2: Comparison of Proposed Framework with Existing Solutions

Criteria	Proposed Hybrid Framework	Pure Modbus TCP	Pure OPC UA	ROS2 Only	MQTT-Based
End-to-End Latency	103.59ms (multi-robot)	6-8ms (simple)	5-162ms	0.3ms (optimized)	300µs-471ms
Multi-Robot Support	Native (dual validated)	Limited	Limited	Excellent	Good
Industrial Compatibility	Excellent	Excellent	Excellent	Poor	Poor
AI Integration	Yes (95% accuracy)	No	No	Limited	No
Scalability	Proven (single to dual)	Poor	Medium	Excellent	Excellent
Implementation Complexity	Medium	Low	High	Medium	Low
Protocol Integration	Multi-protocol bridge	Single	Single	Single	Single
Simulation Support	Excellent	Excellent	Excellent	Excellent	Good
Real-time Determinism	Good (100% success)	Good	Variable	Good	Poor

6 Summary and Future Scope

This thesis designs, implements, and evaluates a modular hybrid communication framework that connects industrial control with modern robotic middleware for single- and dual-arm coordination. The architecture integrates OpenPLC using Modbus TCP, an OPC UA middleware layer, and ROS 2 to command KUKA KR360 robots in simulation. The web GUI issues tasks (such as HOME, PICK, and PLACE), while protocol bridges translate those tasks into joint-level commands. An AI module monitors joint signals to detect abnormal behavior. The central aim is to show that a standards-based industrial stack can coordinate multiple robots while remaining compatible with PLC-centric environments.

The evaluation confirms that this goal was achieved. In single-robot trials, the framework delivered ~113 ms average latency, 1.63 cmd/s throughput, 100 % command success, and ~31 ms jitter, with the anomaly detector achieving 95 % accuracy and 100 % recall for anomalous cases. In dual-robot trials, latency averaged ~104 ms with 1.54 cmd/s throughput and 100 % success, while synchronization difference was ~69 ms and jitter ~68 ms. In comparison with common OPC UA bridge setups observed in practice, which often exhibit multi-second delays, the proposed Modbus–OPC UA–ROS 2 pipeline delivers sub-second responsiveness while keeping interoperability with industrial standards.

Architecturally, the work contributes a clear separation of concerns across five layers: Web-based GUI, PLC logic, protocol translation, ROS 2 middleware, and robot execution. A dual-robot extension is realized through namespace isolation and replicated bridge nodes. The Modbus–OPC UA translation maps coil-level events to structured motion messages, enabling uniform control across components. The AI-based anomaly monitors flags unusual joint-signal patterns and can be engaged without altering the control path. Together, these elements form a repeatable pattern for simulation-first evaluation that can be transitioned toward hardware-in-the-loop with minimal changes.

In relation to the research questions set out in Chapter 1:

1. Can PLC control logic be integrated with ROS 2 robotic software using OPC UA? – Yes, the framework successfully implements OPC UA as a middleware bridge between OpenPLC and ROS 2, achieving deterministic communication.
2. Can multiple robots be coordinated deterministically in such a hybrid architecture? – Yes, the dual-robot trials demonstrate synchronous execution with acceptable jitter for most industrial tasks.
3. Does the architecture maintain competitive latency and throughput compared to existing solutions? – Yes, it outperforms common OPC UA/ROS 2 bridge approaches by 1–2 orders of magnitude in latency while maintaining 100 % reliability.

All experiments occur in simulation, control is exercised at the joint level for a single robot family, and scalability is validated to two robots. The network stack prioritizes openness and clarity over hard real-time determinism. Despite these limits, the measured performance and the modular design provide a practical path for plants to link PC-based operations with ROS 2-based robotics in a structured, testable manner.

Future work should follow a priority order driven by industrial impact. First, validate the framework with hardware-in-the-loop and real robots, measuring latency and jitter under actual PLC scan times, fieldbus scheduling, and robot controller responses. Second, extend scalability beyond two robots, leveraging namespace isolation and replicated bridge nodes, and systematically study how throughput, jitter, and synchronization evolve as fleet size grows. Third, refine the communication pipeline by tuning ROS 2 Quality-of-Service parameters and evaluating deterministic transports (e.g., OPC UA Pub/Sub over UDP with TSN) while retaining OPC UA's semantic benefits. Finally, expand the AI anomaly detection module from single robot to multi-robot scenarios to enhance fault detection at scale.

In conclusion, this study demonstrates that modern industrial communication and open robotic middleware can be combined to achieve sub-second, standards-aligned multi-robot coordination. A disciplined layering of Modbus TCP → OPC UA → ROS 2, coupled with careful namespace design and measured QoS, establishes a clear path from simulation to deployment. Immediate priorities are hardware-in-the-loop and real-robot validation, scalable coordination beyond two robots, and fleet-level AI anomaly detection. This framework not only demonstrates readiness for deployment in Industry 4.0 environments but also establishes a replicable integration model that can guide future interoperability practices. By uniting legacy PLC systems with modern ROS 2 robotics through open standards, it has the potential to influence emerging industrial communication standards and accelerate the adoption of flexible, connected manufacturing systems worldwide.

Bibliography

- [1] M. Ghobakhloo, "Industry 4.0, digitization, and opportunities for sustainability," *Journal of Cleaner Production*, vol. 252, no. 119869, p. 119869, Apr. 2020, doi: <https://doi.org/10.1016/j.jclepro.2019.119869>.
- [2] V. Vyatkin, "IEC 61499 as Enabler of Distributed and Intelligent Automation: State-of-the-Art Review," *IEEE Transactions on Industrial Informatics*, vol. 7, no. 4, pp. 768–781, Nov. 2011.
- [3] R. Ramanathan, "The IEC 61131-3 programming languages features for industrial control systems," 2014 World Automation Congress (WAC), Waikoloa, HI, USA, 2014, pp. 598-603, doi: 10.1109/WAC.2014.6936062.
- [4] E Menendez, JG Victores, R Montero, S Martinez, and C Balaguer. "Tunnel structural inspection and assessment using an autonomous robotic system". In: *Automation in Construction* 87 (Mar. 2018), pp. 117–126.
- [5] MH Ali, A K, Y K, Z T, and A O. "Vision-based Robot Manipulator for IndustrialApplications". In: *Procedia Computer Science* 133 (2018), pp. 205–212.
- [6] S Caro, C Dumas, S Garnier, and B Furet. "Workpiece placement optimization for machining operations with a KUKA KR270-2 robot". In: *Proceedings of the 2013 IEEE International Conference on Robotics and Automation (ICRA)*. Karlsruhe, Germany, May 2013, pp. 2921–2926. doi: 10.1109/ICRA.2013.6631051.
- [7] A Burghardt, D Szybicki, P Gierlak, K Kurc, M Muszynska, A Ornat, et al. "TCP Parameters Monitoring of Robotic Stations". In: *Electronics* 11 (2022), p. 3415.
- [8] Bruno Siciliano and Oussama Khatib. "Robotics: Trends and future directions". In: *IEEE Robotics & Automation Magazine* 16.1 (2009), pp. 80–93.
- [9] John J. Craig. "Introduction to Robotics: Mechanics and Control". In: *Pearson Prentice Hall* (2005). Ch. 2: Spatial Descriptions and Transformations.
- [10] KUKA Aktiengesellschaft, "MyKUKA," KUKA Aktiengesellschaft. https://my.kuka.com/s/product/kr-360-r2830/01t58000002hnjAAAQ?language=en_US&tab=Overview

- [11] S. Lee, "Mastering robot movement." <https://www.numberanalytics.com/blog/mastering-robot-movement-degrees-freedom>
- [12] V. Mayoral-Vilches, "Offensive robot cybersecurity," arXiv.org, Jun. 18, 2025. <https://arxiv.org/abs/2506.15343>
- [13] A. Ioana and A. Korodi, "DDS and OPC UA Protocol Coexistence Solution in Real-Time and Industry 4.0 Context Using Non-Ideal Infrastructure," *Sensors*, vol. 21, no. 22, p. 7760, Nov. 2021, doi: 10.3390/s21227760.
- [14] M. L. Gambo, A. Danasabe, B. Almadani, F. Aliyu, A. Aliyu, and E. Al-Nahari, "A Systematic literature review of DDS middleware in Robotic Systems," *Robotics*, vol. 14, no. 5, p. 63, May 2025, doi: 10.3390/robotics14050063.
- [15] A. A. Adil, S. Sakhrieh, J. Mounsef, and N. Maalouf, "A multi-robot collaborative manipulation framework for dynamic and obstacle-dense environments: integration of deep learning for real-time task execution," *Frontiers in Robotics and AI*, vol. 12, Jul. 2025, doi: 10.3389/frobt.2025.1585544.
- [16] A. Abou Allaban, D. Bonnie, E. Knapp, P. Gokhale, and T. Moulard, *Robot Operating System (ROS) The Complete Reference (Volume 6)*, vol. Volume 962. Springer Nature Switzerland AG, 2021. [Online]. Available: <https://doi.org/10.1007/978-3-030-75472-3>
- [17] "A new hybrid algorithm for tolerating security threats in wireless sensor networks," *IEEE Conference Publication | IEEE Xplore*, Jul. 01, 2018. <https://ieeexplore.ieee.org/document/8531058>
- [18] A. Andrei, F.-A. Nicolescu, C. Pupăză, C.-G. Coman, and I. Ghionea, "Applicability of virtual commissioning concepts in industrial robotics as a solution for compatibility issues between virtual simulation and logic control software," *Applied Sciences*, vol. 15, no. 4, p. 2033, Feb. 2025, doi: 10.3390/app15042033.
- [19] "Hybrid communication protocols and control algorithms for NextGen aircraft arrivals," *IEEE Journals & Magazine | IEEE Xplore*, Apr. 01, 2014. <https://ieeexplore.ieee.org/document/6644284>
- [20] Tapia, Endika, et al. "Assessing Industrial Communication Protocols to Bridge the Gap between Machine Tools and Software Monitoring." *Sensors*, vol. 23, no. 12, 1 Jan. 2023, p. 5694, www.mdpi.com/1424-8220/23/12/5694, <https://doi.org/10.3390/s23125694>.

- [21] E. Gamess, B. Smith, and G. Francia III, "Performance evaluation of ModBus TCP in normal operation and under a distributed denial of service attack," *International Journal of Computer Networks & Communications*, vol. 12, no. 2, pp. 1–21, Mar. 2020, doi: 10.5121/ijcnc.2020.12201.
- [22] S. Jaloudi, "Communication Protocols of an industrial Internet of Things environment: A Comparative study," *Future Internet*, vol. 11, no. 3, p. 66, Mar. 2019, doi: 10.3390/fi11030066.
- [23] V. G. Găitan, I. Zagan, N. C. Găitan, Faculty of Electrical Engineering and Computer Science, Stefan cel Mare University, 720229 Suceava, Romania, and Integrated Center for Research, Development and Innovation in Advanced Materials, Nanotechnologies, and Distributed Systems for Fabrication and Control (MANSiD), Stefan cel Mare University, 720229 Suceava, Romania, "MoDBus RTU Protocol Timing Evaluation for scattered Holding Register REAd and MoDBusE-Related implementation," journal-article, Jan. 2025. [Online]. Available: <https://doi.org/10.3390/pr13020367>
- [24] Găitan, Vasile Gheorghită, Ionel Zagan, and Nicoleta Cristina Găitan. 2025. "Modbus RTU Protocol Timing Evaluation for Scattered Holding Register Read and ModbusE-Related Implementation" *Processes* 13, no. 2: 367. <https://doi.org/10.3390/pr13020367>
- [25] F. Ercan, M. Bega, and B. Kuhlenkötter, "A systematic literature review of communications standards in discrete manufacturing," *Uni-hannover.de*, pp. 80–89, 2023, doi: <https://doi.org/10.15488/15260>.
- [26] H. Raddatz *et al.*, "Evaluation and extension of OPC UA Publish/Subscribe MQTT Binding," journal-article, 2021. [Online]. Available: https://www.kickmetoscience.uni-rostock.de/storages/uni-rostock/Alle_IEF/IMD/veroeff/2020/2020_Raddatz_Evaluation_and_Extension_of_OPC_UA_PublishSubscribe_MQTT_Binding.pdf
- [27] S. Cavalieri, University of Catania, F. Chiacchio, and University of Catania, "Analysis of OPC UA performances," Nov. 2013. doi: 10.1016/j.csi.2013.06.004.
- [28] S. Profanter, A. Tekat, K. Dorofeev, M. Rickert, and A. Knoll, "OPC UA versus ROS, DDS, and MQTT: Performance Evaluation of Industry 4.0 Protocols," journal-article.
- [29] M. Hlayel, H. Mahdin, and H. A. M. Adam, "Latency analysis of WebSocket and industrial Protocols in Real-Time Digital twin Integration," *International Journal of Engineering Trends and Technology*, vol. 73, no. 1, pp. 120–135, Jan. 2025, doi: 10.14445/22315381/ijett-v73i1p110.

- [30] M. Ladegourdie and J. Kua, "Performance Analysis of OPC UA for Industrial Interoperability towards Industry 4.0," *IoT*, vol. 3, no. 4, pp. 507–525, Dec. 2022, doi: 10.3390/iot3040027.
- [31] X. An, C. Wu, Y. Lin, M. Lin, T. Yoshinaga, and Y. Ji, "Multi-Robot systems and cooperative object transport: communications, platforms, and challenges," *IEEE Open Journal of the Computer Society*, vol. 4, pp. 23–36, Jan. 2023, doi: 10.1109/ojcs.2023.3238324.
- [32] T. Wang, Q. Dang, and P. Pan, "A Multi-Robot system based on a hybrid communication approach," *Studies in Media and Communication*, vol. 1, no. 1, Apr. 2013, doi: 10.11114/smc.v1i1.124.
- [33] R. Doriya, S. Mishra, S. Gupta, and Department of Information Technology, National Institute of Technology, Raipur, India-492010, "A brief survey and analysis of multi-robot communication and coordination," IEEE, 2015.
- [34] "Centralized vs decentralized control in multi-robot systems." <https://eureka.patsnap.com/article/centralized-vs-decentralized-control-in-multi-robot-systems>
- [35] J. Gielis, A. Shankar, and A. Prorok, "A Critical review of Communications in Multi-robot systems," *Current Robotics Reports*, vol. 3, no. 4, pp. 213–225, Aug. 2022, doi: 10.1007/s43154-022-00090-9.
- [36] R. K. Thaker, "An overview of Swarm robotics: Swarm intelligence and engineering approaches for coordinating multi-robot systems," *International Journal for Multidisciplinary Research*, vol. 3, no. 4, Aug. 2021, doi: 10.36948/ijfmr.2021.v03i04.29703.
- [37] Tardioli, Danilo & Parasuraman, Ramviyas & Ogren, Petter. (2017). Pound: A ROS node for Reducing Delay and Jitter in Wireless Multi-Robot Networks. 10.48550/arXiv.1707.07540.
- [38] Y. Ye *et al.*, "ROS2 Real-time Performance Optimization and Evaluation," *Chinese Journal of Mechanical Engineering*, vol. 36, p. 144, 2023, doi: 10.1186/s10033-023-00976-5.
- [39] J. Park, R. Delgado and B. W. Choi, "Real-Time Characteristics of ROS 2.0 in Multiagent Robot Systems: An Empirical Study," in *IEEE Access*, vol. 8, pp. 154637-154651, 2020, doi: 10.1109/ACCESS.2020.3018122.
- [40] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the Wild."

- [41] A. S. Al-Batati, A. Koubaa, M. Abdelkader, and College of Computer and Information Sciences, Prince Sultan University, "ROS 2 Key Challenges and Advances: A survey of ROS 2 research, libraries, and applications," journal-article, 2024. doi: 10.20944/preprints202410.1204.v1.
- [42] M. Chemnitz, H. Schorn, K. Töreki, and A. Fritz, "Integrating PLCs and Robots into the ROS 2 Ecosystem," *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–4, Sep. 2024, doi: 10.1109/etfa61755.2024.10710873.
- [43] J. He, J. Zhang, J. Liu and X. Fu, "A ROS2-Based Framework for Industrial Automation Systems," *2022 2nd International Conference on Computer, Control and Robotics (ICCCR)*, Shanghai, China, 2022, pp. 98-102, doi: 10.1109/ICCCR54399.2022.9790247.
- [44] M. Singh *et al.*, "Unity and ROS as a digital and communication layer for digital twin application: Case study of robotic arm in a smart manufacturing cell," *Sensors*, vol. 24, no. 17, p. 5680, Aug. 2024, doi: 10.3390/s24175680.
- [45] S. Tamboli, M. Rawale, R. Thoraiet, and S. Agashe, *Implementation of Modbus RTU and Modbus TCP Communication using Siemens S7-1200 PLC for Batch Process*. IEEE, 2015, pp. 258–263. [Online]. Available: <https://ieeexplore.ieee.org/document/7161200>
- [46] J. Devagiri, P. Hazaveh, N. Rawashdeh, S. R. R. Dudipala, P. M. Deshmukh, and A. Karmarkar, "PLC Multi-Robot Integration via Ethernet for Human Operated Quality Sampling," *2024 ASEE Annual Conference & Exposition*, Aug. 2024, doi: 10.18260/1-2--47848.
- [47] B. Laurean, "PLC BASED SYSTEM FOR INCREMENTAL PROGRAMMING AND CONTROL OF ROBOTS," *Robotica & Management*, 2011.
- [48] J.-P. Thomesse, "Fieldbus Technology in industrial automation," journal-article, Jun. 2005. doi: 10.1109/JPROC.2005.849724.
- [49] "Impact of fieldbus on communication in robotic systems," *IEEE Journals & Magazine | IEEE Xplore*, Feb. 01, 1997. <https://ieeexplore.ieee.org/document/554345> (accessed Jul. 28, 2025).
- [50] C. W. Moon, B. H. Lee, and M. S. Kim, *PLC based coordination schemes for a multi-robot system*. 2001.
- [51] S. Chitikeshi, S. K. Dhali, and V. M. Jovanovic, "Integrating PLCs with Robot Motion Control in Engineering Capstone Courses," *Engineering*

Technology Faculty Publications, 2022, [Online]. Available: https://digitalcommons.odu.edu/engtech_fac_pubs

- [52] B. Wu, "Study of PLC-based industrial robot control systems," *2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, pp. 1094–1097, Oct. 2022, doi: 10.1109/iccasit55263.2022.9986949.
- [53] "A Real-Time software framework for multi-axis robotic systems using ROS 2 and EtherCAT," *IEEE Conference Publication | IEEE Xplore*, Dec. 13, 2024. <https://ieeexplore.ieee.org/document/10895565> (accessed Jul. 28, 2025).
- [54] P. Bilancia, J. Schmidt, R. Raffaeli, M. Peruzzini, and M. Pellicciari, "An overview of industrial robots control and programming approaches," *Appl. Sci.*, vol. 13, p. 2582, Feb. 2023, [Online]. Available: <https://doi.org/10.3390/app13042582>
- [55] W.-C. Chen, B.-Y. Ji, and H.-H. Chen, "A case study on implementing a flexible IIoT service framework for the integration of machine tools," *Journal of Industrial Information Integration*, vol. 47, p. 100908, Jul. 2025, doi: 10.1016/j.jii.2025.100908.
- [56] Q. -D. Nguyen, S. Dhouib, Y. Huang and P. Bellot, "An Approach to Bridge ROS 1 and ROS 2 Devices into an OPC UA-based Testbed for Industry 4.0," *2022 IEEE 1st Industrial Electronics Society Annual On-Line Conference (ONCON)*, kharagpur, India, 2022, pp. 1-6, doi: 10.1109/ONCON56984.2022.10126783.
- [57] Tripathy, J. van Deventer, C. Paniagua and J. Delsing, "Interoperability Between ROS and OPC UA: A Local Cloud-Based Approach," *2022 IEEE 5th International Conference on Industrial Cyber-Physical Systems (ICPS)*, Coventry, United Kingdom, 2022, pp. 1-5, doi: 10.1109/ICPS51978.2022.9816962.
- [58] H. M. Park and J. W. Jeon, "OPC UA based Universal Edge Gateway for Legacy Equipment," *Sungkyunkwan University*, p. 1002, 2019, [Online]. Available: <https://ieeexplore.ieee.org/document/8728292>
- [59] N. Mohamed, J. Al-Jaroodi and I. Jawhar, "Middleware for Robotics: A Survey," *2008 IEEE Conference on Robotics, Automation and Mechatronics*, Chengdu, China, 2008, pp. 736-742, doi: 10.1109/RAMECH.2008.4681485.

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e.g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 19.08.2025